SPECIFICATIONS AND EXPLANATIONS IN AUTOMATIC PROGRAMMING

HAIFENG RUAN

(B.Eng., Tsinghua University)

A THESIS SUBMITTED
FOR THE DEGREE OF DOCTOR OF PHILOSOPHY
DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF SINGAPORE

2026

Thesis Advisor:

Professor Abhik Roychoudhury

Examiners:

Dr Umang Mathur
Dr Manuel Rigger

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Haifeng Ruan
Nov 2026

Acknowledgments

I am deeply grateful to my advisor, Abhik Roychoudhury, for his tireless discussions and advice throughout my PhD. He has been a role model in his taste for imagination, insight, and vision over quantitative metrics, and in his proactive and fearless attitude towards work and life.

I thank the members of my thesis committee, Manuel Rigger and Umang Mathur, for their time and for the feedback that sharpened this thesis.

The chapters of this thesis grew out of joint work. Chapter 2 is based on joint work with Hoang Lam Nguyen, Ridwan Shariffdeen, Yannic Noller, and Abhik Roychoudhury; Chapter 3 with Zhiyu Fan, Sergey Mechtaev, and Abhik Roychoudhury; Chapter 4 with Yuntong Zhang and Abhik Roychoudhury; and Chapter 5 with Sungmin Kang and Abhik Roychoudhury. I am grateful for their ideas, their hard work, and the discussions that shaped each project.

I also thank my colleagues in the TSS group for the conversations and company over the past years.

The work in Chapter 4 has been further developed into a commercial product, Sonar Remediation Agent, after Sonar's acquisition of our startup AutoCodeRover, co-founded by Abhik Roychoudhury, Ridwan Shariffdeen, Yuntong Zhang, Martin Mirchev, and myself. I thank Sonar for the opportunity to bring this research into production and to serve a large user base, and my co-founders for the meaningful journey we shared. This thesis shares AutoCodeRover's broader theme of automated program improvement. Chapters 4 and 5 are particularly aligned with it: the former presents SpecRover, a direct successor of AutoCodeRover, and the latter presents AutoCodeSherpa, a technique for vetting code changes produced by coding agents such as AutoCodeRover and Sonar Remediation Agent.

Finally, I gratefully acknowledge the financial support of the NUS Research Scholarship and the Singapore Ministry of Education Tier 3 grant "Automated Program Repair" (MOE-MOET32021-0001), which made this work possible.

Contents

Title Page i

Declaration ii

Acknowledgments iii

List of Figures viii

List of Tables x

List of Algorithms xi

1 Introduction 1

1.1 The Specification Problem in Automatic Programming 1

1.2 Pre-LLM Era: Reducing Overfitting with Coevolution 1

1.3 LLM Era: Program Selection from LLMs 2

1.4 Agentic Era: From Global to Local Specification 3

1.5 From Specifications to Explanations 4

1.6 Contributions 5

1.7 Thesis Organization 6

2 Evolutionary Testing for Program Repair 7

2.1 Introduction 7

2.2 Overview 8

2.2.1 Workflow 8

2.2.2 Illustrative Example 10

2.3 Test Oracles 12

2.3.1 Oracle Writing Methodology 12

2.3.2 Failed Attempts 13

2.3.3 Oracle Types 13

2.4 Coevolutionary Repair 14

2.4.1 Search Space 15

2.4.2 Fitness Functions: Patch Generation 16

2.4.3 Fitness Functions: Test Generation 16

2.5 Implementation 17

2.5.1 Test Generation 17

2.5.2 Patch Generation 18
2.6 Evaluation 18
2.6.1 Evaluation Subjects 19
2.6.2 Evaluation Tools 19
2.6.3 Experimental Setup 19
2.6.4 RQ1: Efficacy of EvoRepair 20
2.6.5 RQ2: Impact of Coevolution on Patch Generation 21
2.6.6 RQ3: Impact of Coevolution on Test Generation 22
2.7 Threats to Validity 22

3 Oracle-Guided Program Selection 24
3.1 Introduction 24
3.2 Motivating Example 25
3.3 Methodology 27
3.3.1 Differential Testing of Program Samples 28
3.3.2 Specification Selection with LLM 30
3.4 Experimental Setup 31
3.5 Evaluation 33
3.5.1 RQ1: Effectiveness of Test Generation 33
3.5.2 RQ2: Effectiveness of Specification Selection 34
3.5.3 RQ3: Overall Effectiveness 35
3.5.4 RQ4: Reasons for Failure 39
3.5.5 Discussion of Selected Specification 42
3.6 Threats to Validity 43

4 Code Intent Extraction via LLMs 44
4.1 Introduction 44
4.2 Motivating Example 45
4.3 Methodology 47
4.3.1 Overview 47
4.3.2 Function Summary: Specification from Program 49
4.3.3 Reviewer Feedback: Reconciling Specifications 50
4.3.4 Patch Selection 51
4.3.5 Evidence 52
4.4 Experimental Setup 53
4.5 Evaluation 54
4.5.1 RQ1: Overall Efficacy of Task Resolving 54
4.5.2 RQ2: Utility of Autonomous SE, Confidence in Results 57
4.5.3 RQ3: Quality of Specifications Produced 58
4.6 Threats to Validity 63
4.7 Discussion 64

5 Symbolic Explanations in Coding Agents 66
5.1 Introduction 66
5.2 Background 67
5.2.1 Bug Characterization 67
5.2.2 Hoare Triple 68
5.2.3 Property-Based Testing 68
5.2.4 Coding Agents 69
5.3 Overview 70
5.4 Symbolic Explanations: Definition 72
5.5 Symbolic Explanations: Construction 73
5.5.1 PBT Generation 74
5.5.2 Infection Condition Generation 75
5.6 Experimental Setup 77
5.7 Evaluation 80
5.7.1 RQ1: Accuracy of Conditions 80
5.7.2 RQ2: Patch Validation Capability 81
5.7.3 RQ3: Efficacy Gains in Issue Resolution 83
5.7.4 RQ4: Generalizability across LLMs 85
5.7.5 RQ5: Scalability across Repositories and Issue Complexity 87
5.8 Case Studies 88
5.9 Threats to Validity 88
5.10 Discussion 89

6 Related Work 91
6.1 Automatic Program Repair and Test-Based Specifications 91
6.1.1 Coevolution of Tests and Patches 91
6.1.2 Evolutionary Repair 92
6.1.3 Evolutionary Testing 92
6.1.4 Test Generation for Patch Testing 93
6.2 LLM-Based Code Generation and Program Selection 93
6.3 LLM for Specification Inference 95
6.4 Repository-Level Coding Agents 95
6.4.1 Coding Agents 96
6.4.2 Patch Review under Unreliable Tests and Patches 96
6.4.3 Code Intent Extraction via LLMs 96
6.5 Symbolic Explanations in Coding Agents 96
6.5.1 Issue Reproduction 97
6.5.2 Input Conditions 97
6.5.3 Automatic Bug Explanation 97

7 Conclusion 98

7.1 Summary 98
7.2 Future Work 100
7.2.1 Seeding Differential Testing with LLM-Generated Inputs 100
7.2.2 Resolving Ambiguities Revealed by Conflicting Candidate Outputs 100
7.2.3 Patch Correctness Estimation 101
7.2.4 Executable Explanations in Patch Review 101

References 103

List of Figures

2.1 Overview of EvoRepair's workflow. 8

2.2 Illustration of the oracle generation from a developer provided bug-report and the buggy function, based on MATH-227 in Defects4J benchmark. 10

2.3 Evolution of patches captured in p_1, p_2, p_3 and p_4, generated through multiple iterations of coevolution. 11

2.4 (a) Venn diagram of bugs for which repair tools found a plausible patch in every repetition. Omitted from the figure are jGenProg and ARJA-e. They consistently fixed 13 and 6 bugs, with 8 and 4 bugs overlapping with EvoRepair. The other tools only fixed a subset of the bugs fixed by jGenProg. (b) Venn diagram of bugs for which repair tools found a correct patch in at least one repetition. (c) Distribution of experimental trials by the number of generations where an overfitting patch is removed. The left axis shows the absolute number of trials while the right shows the relative number. 20

2.5 (a) Change in the survival rate of plausible patches over coevolution generations in three representative trials. (b) Percentage of patches of each repair tool killed by additional test cases. (c) Probability of a triggering test and an average test to kill patches on 20 bugs (y-axis on a log scale). 21

3.1 The workflows and selection results of LLMCodeChoice, AlphaCode, and CodeT on task 44 of HumanEval. Selection results are marked in bold. While AlphaCode and CodeT use LLM-generated tests to do program clustering and some form of majority vote, LLMCodeChoice obtains distinguishing inputs by differential testing and prompts LLM to select the correct outputs for these inputs based on natural language problem description. 25

3.2 Overview of Oracle-Guided Program Selection 27

3.3 Prompt for Specification Selection 31

3.4 Comparison between direct specification inference and specification selection on an example from HumanEval. 36

3.5 Example of ambiguous task description from HumanEval benchmark. The ambiguous point is the interpretation of "distinct elements". Lines 6-7 show the specific implementation of the misinterpreted concept. 41

3.6 Example of contradictory explanation process for 'triples_sum_to_zero'. We first use the LLM to derive an explanation for each input-output pair. Then we take those explanations and query LLM again to identify the reasons for producing different explanations. 42

4.1 scikit-learn-15535 description and SpecRover artifacts. 46
4.2 Overall Workflow of SpecRover. 47
4.3 Context retrieval in SpecRover. 49
4.4 Template prompt for the reviewer agent. 51
4.5 Number of uniquely resolved issues by the top performing open-source tools on SWE-bench Lite. 56
4.6 Comparison of patch precision on SWE-bench Lite. If SpecRover does not use regression tests for patch validation, precision reduces only slightly to 48.3%. 59
4.7 pytest-7432 description and SpecRover artifacts. 60
4.8 sympy-21614 description and SpecRover artifacts. 61
4.9 Issue django-11163. The issue statement contains the ground-truth patch. 63

5.1 An example PBT 69
5.2 Overview of AutoCodeSherpa with a real example simplified for clarity; PBT is for property-based test. 70
5.3 The buggy function involved in our running example. 71
5.4 Output from the generalization phase of PBT generation. 73
5.5 Detailed condition generation flowcharts for AutoCodeSherpa. 74
5.6 A plot of generation ratio and condition accuracy. 81
5.7 Effect of LLM calls and refinement iterations on explanation accuracy. 81
5.8 PBT and infection condition accuracy over different repositories and issue complexities. The bar charts are shown with 95% confidence intervals. 86
5.9 A schematic of a correct and helpful explanation. 87

List of Tables

3.1 Average number of identified program clusters and average number of generated test cases of each tool on HumanEval+ and MBPP+. 34
3.2 Accuracy of Oracle Inference Strategies: specification selection via LLMCodeChoice, specification inference from scratch from LLM, and majority vote. 35
3.3 Overall pass@1 (%) and average token spent on the HumanEval+, HumanEval, MBPP+, and MBPP benchmarks with GPT-4 for all tools, excluding all tasks that cannot make correct program selection. The numbers $X/(Y)$ in the pass@1 (%) columns represent the pass@1 result for enhanced / (original) benchmarks respectively. 37
3.4 Four Reasons for Incorrect Specification Selection in LLMCodeChoice. 39
3.5 Causes of the 21 wrong choices by LLMCodeChoice. 39

4.1 Comparison of efficacy/efficiency/cost on SWE-bench and SWE-bench Lite. 55
4.2 Reviewer decisions on SWE-bench Lite. 58
4.3 Number of resolved instances, and number of resolved instances with generated patches being syntactically equivalent to the ground-truth patch. Results are shown for each tool in SWE-bench Lite. 64

5.1 Confusion matrices of AutoCodeSherpa and baselines, where positive means correct patch. 82
5.2 Reasons for FP patch categorization 82
5.3 Agentless fault localization efficacy, with and without explanations 83
5.4 Agentless plausible patch and bug resolve rate, with and without explanations 84
5.5 Evaluation of AutoCodeSherpa with different LLMs 85
5.6 Effect of refinement in PBT agent. Refinement brings the correctness rate of different LLMs close. *Rectified* means the first PBT was incorrect but was changed into a correct one by the refinement step, while *Degraded* means a correct first PBT was changed into an incorrect one. *Discarded* means a PBT was initially generated but eventually not submitted, as it could not pass the refinement step. 85
5.7 Comparison of the original and rerun results of AutoCodeSherpa, showing its robustness. 89

List of Algorithms

2.1 Coevolutionary Repair 15

3.1 Oracle-Guided Program Sample Selection 30

Chapter 1

Introduction

1.1 The Specification Problem in Automatic Programming

Automatic programming is a long-running aspiration in software engineering: the goal is to translate developer *intent* into executable programs with minimal manual coding. Such intent is captured by a *specification*, which can take many forms with varying degrees of completeness and rigor. Automatic programming techniques then consume the specification to produce a program.

The concrete automatic programming techniques have evolved over the years, from program synthesis, program repair, to LLM or even agentic program generation. However, a key gap has always persisted: a high-quality specification is usually unavailable for guiding the automatic programming. Specifications that are readily available in practice, e.g., test cases or natural language, tend to be incomplete or ambiguous, while the more rigorous specifications, e.g., formal specifications, are rarely written. This raises the question that motivates this thesis: how can more rigorous specifications be derived and effectively utilized in automatic programming?

In this thesis, we examine this question across three eras: pre-LLM, LLM, and agentic. We also envision the future software development workflows that go beyond specifications: in order for agents to collaborate effectively with human developers, the agent has to communicate back the actual behavior of the code it produces. In other words, agents should provide *explanations* in the future.

1.2 Pre-LLM Era: Reducing Overfitting with Coevolution

In the pre-LLM era, the main lines of work were program synthesis and program repair. Program synthesis aims to construct a program from a given specification. The specification can be a logical formula [SLTB+06, ABJ+13], a set of input-output examples [Gul11], or a reference implementation [Mas87]. The target program is then obtained either by solving an SMT constraint [JGST10] or by enumerative search [ARU17]. Among these forms, a logical formula is the most rigorous, but is challenging to write and rarely available in practice.

Input-output examples are easier to provide but inherently incomplete: a program that satisfies the examples can still diverge from the intended one.

Automated program repair (APR) [LGPR19] aims to produce a program change, or *patch*, that fixes a bug in an existing program, using search-based, semantic-based, or learning-based techniques, amongst others. The most commonly used specification is a test suite: the bug manifests as a failing test, and a patch is considered *plausible* if the patched program passes every test. A test suite, however, is an incomplete specification, so APR techniques can produce *overfitting* patches that are plausible yet still *incorrect*, i.e., they do not fix the underlying bug [SBLGB15]. A straightforward intuition would be to add more test cases to improve the specification, but generating more test cases is non-trivial due to the test oracle problem (i.e., knowing the expected output for the generated tests). For example, while software fuzzing can generate many test inputs, it is restricted to fairly simple, pre-defined oracles like crash freedom or differential metrics like those used for regression testing. It is hard to formulate complex functional specifications without domain knowledge, and formal specifications are usually not available in practice.

In Chapter 2, we address the test oracle problem by referring to other sources of specifications: in practice, we often have developer-written bug reports [AHM06, KLB+19]. We can extract lightweight specifications from these bug reports and make them available as assertions by instrumenting the buggy program. In Chapter 2, we study how additional tests can improve patch generation and how test generation and patch generation can aid each other. Thereby, we formulate a *coevolutionary* generation of tests and patches to tackle the test overfitting problem and improve the quality of generated patches. Each coevolution cycle includes the generation of patches and tests while both generation processes guide each other. This leads to the incremental augmentation of the test suite with more tests and refines the patch space with high quality patches.

1.3 LLM Era: Program Selection from LLMs

The pre-LLM setting above still requires manually extracting a specification from a natural-language bug report to guide test generation. In recent years, large language models (LLMs) [CTJ+21, LCC+22, Ope23] have become the dominant approach to automatic programming instead, because they can consume natural-language specifications directly. An LLM can synthesize a program from scratch or improve an existing one, as embodied in AI programming assistants such as GitHub Copilot and Amazon CodeWhisperer (now Amazon Q Developer) [Git23, Ama23], which generate multiple code suggestions from a natural-language query for the developer to choose from.

However, natural language is inherently ambiguous and rarely complete, so the resulting program does not always match user intent [FGM+23]. Moreover, even given a perfect specification, LLM generation is non-deterministic, and there is no reliable way to check

generated code against natural language. For instance, Codex's own evaluation under the *pass@k* metric (the probability of a correct solution within k attempts) reports pass@1 = 28.81% and pass@10 = 46.81% [CTJ+21]. While a correct solution can be generated with more attempts, the right one must be selected from the many generated solutions.

A common approach to mitigating this problem is selecting programs among multiple sampled LLM outputs based on the consistency of these outputs, e.g. via the majority vote [CZN+23, LCC+22]. Techniques following this paradigm first group LLM-generated programs into clusters so that the programs in each cluster share the same execution results on a given test suite. Then, they use the size of a program cluster as a proxy for the likelihood of being correct. Specifically, AlphaCode [LCC+22] trains a separate LLM for natural language based test-input generation and groups programs with the same output together. CodeT [CZN+23] prompts an LLM to generate a test suite and applies so-called dual-execution agreement to assign correctness scores for each program cluster. Although these approaches have shown promising improvements over the default random selection strategy in LLMs, there are certain limitations. One of the key limitations is the low quality of the LLM-generated tests. These tests are often incorrect, contain many duplicates, or cannot reveal the discrepancies between various generated programs.

To address the above limitations, in Chapter 3, we propose a new perspective on increasing the quality of LLM-generated code via program selection using the LLM as a test oracle [BHM+15]. Our method is based on our experimentally confirmed observation that LLMs serve more effectively as oracles when tasked with selecting the correct output from multiple choices, rather than predicting the right outcome for a given input from scratch. Relying on this observation, we first find *distinguishing inputs* that capture the semantic discrepancies between LLM-generated programs through differential testing instead of randomly sampling test cases with LLMs. Then, we group programs that produce the same results on distinguishing inputs into program clusters. We also record outputs produced by the programs on these inputs for each cluster. An LLM is then prompted to select the output that is most likely to be correct, guided by the natural language problem description. We also apply chain-of-thought (CoT) prompting [WWS+22] to increase the chance of selecting the right output. The selected outputs, together with the distinguishing inputs, form a new specification. Finally, the LLM-generated programs that satisfy this specification are returned to the user.

1.4 Agentic Era: From Global to Local Specification

While the LLM is a powerful approach to automatic programming, it still operates on a relatively small code snippet at a time. In practice, developers work with large codebases, and they have to navigate the codebase manually to decide where to apply the LLM. Since 2024, LLM-based coding agents [Cog24, YJW+24, ZRFR24] have emerged, extending program

generation from a small snippet to an entire repository. Backed by an LLM and equipped with analysis tools, an agent can search, edit, and test code across the codebase given a natural-language issue, and improve large, real-world software [JYW+24]. However, the only specification available to guide this process is still the issue itself, a single natural-language description of *global* intent, while fixing a specific component where the issue resides requires a specification of that component.

In Chapter 4, we present SpecRover, which explores the role of program specifications thoroughly in LLM-guided autonomous software engineering workflows. We take the position that the structure of the program captures a coarse encoding of the developer intent, and try to glean intent by analyzing (and searching over) the program structure; in particular, we perform code search on the project structure for fix localization. Our work also looks into various sources of specifications such as function-level code summaries and test cases, apart from program structure. The core contribution thus lies in distilling the various specifications coming from different sources into a single patch.

SpecRover conducts and exploits more powerful specification inference. Starting from a GitHub issue, it conducts code search guided by the program structure. In the process of the code search, as it visits classes/methods, it also calculates and deposits the specifications of the classes/methods which would have allowed for remediation of the observable error, thereby capturing *intended* program behavior. The specifications gathered from the code search are deposited along with generated tests to a reviewer agent. The reviewer agent studies the specifications, generated tests, and natural language requirements to guide the patching. More importantly, the reviewer agent produces evidence of confidence in the reported patch. Chapter 4 develops this direction in detail.

1.5 From Specifications to Explanations

In the automatic programming workflows discussed in the previous sections, the developer provides a specification that constrains the program produced by the automatic programming tool. Equally important is an *explanation* that the tool can return to the developer, describing the behavior of the program and helping the developer understand or improve it.

The need for such explanations is most acute for the coding agents discussed in Section 1.4: their autonomy creates greater potential for undesirable behavior, and explainability has been identified as a primary factor in human trust of such agents [RPPR26]. Patches produced by current agents, however, do not carry such a *checkable* explanation.

In Chapter 5, we present AutoCodeSherpa, a technique that, given an issue description, generates a *symbolic explanation* for why an issue occurs and whether a program change fixes the issue. The explanation consists of three interrelated conditions: an *input condition*, specifying the input space under which the bug occurs; an *infection condition*, an inter-

nal program state caused by the bug; and an *output condition*, capturing the observable symptoms.

AutoCodeSherpa is realized as a multi-agent pipeline. A first agent characterizes the input and output conditions by generating a property-based test (PBT)—a generalization of the failing execution that captures the input-output properties observed in buggy runs. A second agent then explores the repository's code, and a third agent synthesizes infection conditions: symbolic formulae that distinguish buggy states from normal ones. Each stage incorporates refinement steps to heighten the accuracy of the LLM-generated conditions.

These symbolic explanations offer two benefits over natural-language ones. First, because the input and output conditions are encoded as an executable PBT, the explanation itself can be run against any candidate patch and used to filter likely-correct ones—going beyond what natural-language explanations [KCYL25, MSR23] can offer. Second, the same artifact can be consumed by other agents, enabling agent-to-agent collaboration grounded in a checkable record of the bug rather than free-form prose.

AutoCodeSherpa shows that the three conditions can be generated with high accuracy, and the resulting executable explanations substantially improve patch validation and assist other agents in resolving issues.

1.6 Contributions

- **Coevolutionary repair under incomplete specifications**. In the pre-LLM setting where tests are the only available specification, we show that additional oracles can be extracted manually from natural-language bug reports, and that jointly evolving patches and tests by leveraging the extracted oracles can mitigate test-overfitting in automated program repair.
- **Oracle-guided program selection from LLM samples**. In the LLM setting where the specification comes from a natural-language requirement, we show that the LLM serves as a more accurate oracle over concrete input-output pairs than over the programs directly. We leverage this insight to achieve more accurate program selection with lower token usage than self-consistency baselines.
- **Iterative specification inference for agentic program improvement**. In the agentic issue resolution setting, we show that with a natural-language issue as a global specification, different forms of specifications, including tests and natural-language specifications of local code components, can be effectively inferred and reconciled to achieve more effective issue resolution.
- **Symbolic explanations for coding agents**. We introduce symbolic issue explanations—input, infection, and output conditions encoded through property-based tests and program-internal symbolic expressions—as a layer of trust above the patches that

coding agents produce. We show that such explanations can validate suggested patches and guide other agents.

1.7 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 presents EvoRepair, the pre-LLM study of coevolutionary program repair under incomplete test specifications. Chapter 3 presents LLMCodeChoice, the LLM-era study of oracle-guided program selection over discrepant LLM-generated samples. Chapter 4 presents SpecRover, the study of agentic iterative specification inference for autonomous program improvement. Chapter 5 presents AutoCodeSherpa, the study of symbolic, executable explanations for coding agents. Chapter 6 surveys related work for each of the four directions. Chapter 7 concludes by summarizing the four directions and envisioning the role of specifications and explanations in future LLM-driven automatic programming workflows and proposing future works.

Chapter 2

Evolutionary Testing for Program Repair

2.1 Introduction

As discussed in Section 1.2, repairs generated based on tests may suffer from the test overfitting problem, and generating additional tests to improve the test suite quality is generally difficult because the oracle or expected observable behavior of the generated tests is unknown. In this chapter, we first construct additional oracles by instrumenting buggy programs from the Defects4J benchmark with the knowledge obtained from the available bug reports. Then, we formulate a coevolution approach that generates tests and repairs in a unified workflow.

We note that Arcuri and Yao [AY08] explored coevolutionary program repair based on genetic programming. However, their strategy is based on the assumption that a formal specification of the intended program behavior (i.e., a formalization of the software requirements) is available. As it is widely known, formal specifications capturing software requirements are often unavailable in real-life software projects.

Our patch generation employs a coevolutionary search for patches *and* tests, initially driven by the developer-provided test suite. The resulting plausible patches are used as targets for the test generation so that the new tests should reach the corresponding fix locations and establish the incorrectness of these patches. Using the test execution results, we select seed patches for the next cycle of patch generation. Note that the fitness score of a patch can change over time, as new tests are generated. We do not discard patch candidates from the process as long as they pass the initial test suite. Our approach, however, only reports patches that pass all available tests, including the newly generated tests.

We implement our proposed technique, EvoRepair, on top of the popular Java unit testing framework EvoSuite [FA11]. EvoSuite provides an extensible implementation of a variety of evolutionary testing algorithms, which we re-purpose for program repair. EvoSuite also supports mutation testing to assess different properties of test suites, where the mutants can be re-purposed as patch candidates for program repair. We evaluate EvoRepair using 39 subjects derived from Defects4J [JJE14], and compare it with several state-of-the-art tools including ARJA-e [YB20b], TBar [LKKB19], and RewardRepair [YMM22]. Our evaluation

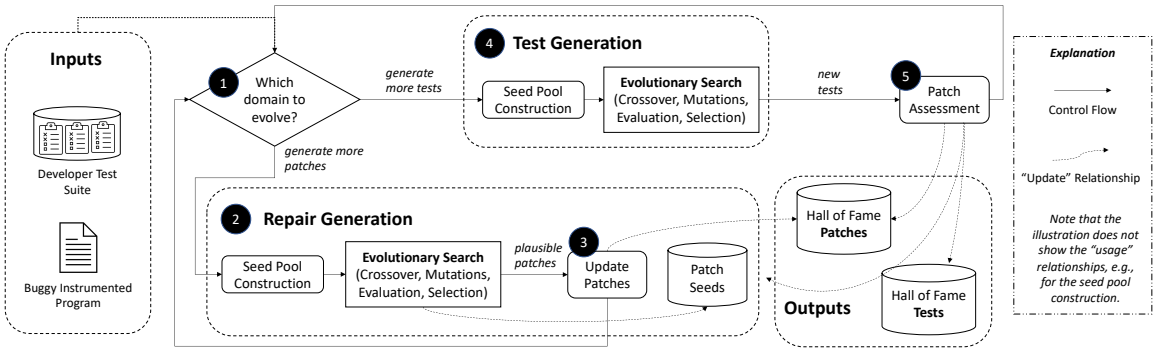


Figure 2.1: Overview of EvoRepair's workflow.

demonstrates that a *coevolutionary* process allows us to fix a larger set of bugs. The test cases generated by EvoRepair can prune the patch pool of EvoRepair by 69%, and also prune the patch pool of other tools by at least 33.3% in our experiments.

Overall, we make the following core contributions:

- We propose a coevolutionary program repair approach integrated into the popular unit testing framework EvoSuite. We re-purpose EvoSuite's evolutionary testing search to generate patches. The tests also serve as evidence of the trustworthiness of the patches reported.
- Using Defects4J subjects, for which ARJA can generate plausible patches, we extracted 39 test oracles from the corresponding bug reports. These will also be useful for other researchers in testing and repair.
- We evaluate EvoRepair against evolutionary search based program repair tools. Experimental results demonstrate the increased efficacy of EvoRepair in reducing overfitting patches from a pool of patch candidates.

2.2 Overview

Figure 2.1 shows the overall workflow of our coevolutionary generation of patches and tests. It takes in a buggy program and a developer provided test-suite. The developer provided test-suite contains at least one failing test. It outputs a set of candidate patches as well as a set of tests which provide evidence of veracity of the patches.

2.2.1 Workflow

In our workflow, the initial pool of patches is generated via mutations. Test generation with the goal of "killing" these patches is conducted. Thus, during the course of the repair algorithm - the test pool increases while the patch pool shrinks. Coevolutionary

algorithms typically maintain a collection of individuals referred to as an *archive* or *hall of fame*, which is a collection of fittest individuals selected from multiple generations of coevolution [PBWDJ12]. Throughout the workflow, we maintain three such archives:

$\mathbb{H}_{patches}$ the *Hall of Fame* for *patches* including all the patches that pass the currently considered tests.

$\mathbb{H}_{tests}$ the *Hall of Fame* for *tests* including the additionally generated test cases that were able to kill patches.

$\mathbb{V}_{patches}$ the *patch seeds*, i.e., the generated patches that pass the failing developer tests but fail on at least one other test (either initially passing developer tests or additionally generated tests). These patches are helpful because they fix the initial problem but fall short in other aspects. They serve as additional seeds for patch generation.

The central aspect of coevolutionary repair is the interaction between testing and repair, which is driven by the decision to evolve the test or patch domain (Step 1). In the beginning, since we assume there is a developer test suite, the coevolution starts from the patch domain and attempts to find plausible patches. Later, the workflow dynamically decides which domain to evolve. As long as the size of $\mathbb{H}_{patches}$ is below a pre-defined threshold, we will keep generating patches. If $\mathbb{H}_{patches}$ has reached this target size, we will keep generating tests to kill the patches in $\mathbb{H}_{patches}$. For practical usability, the size of $\mathbb{H}_{patches}$ should be small [NSGR22], so that a developer can examine the patches in $\mathbb{H}_{patches}$. On the other hand, $\mathbb{H}_{patches}$ should have enough patches to provide accurate feedback for test generation. In our experiment, we set the target size to 20.

The patch generation (Steps 2 and 3) starts by constructing the seed patch pool by combining the two sets $\mathbb{V}_{patches}$ and $\mathbb{H}_{patches}$. In the beginning, it is randomly initialized. Intuitively, we aim to refine the patches in $\mathbb{V}_{patches}$ by using the patches from $\mathbb{H}_{patches}$. During the evolutionary search, patches that pass the failing developer tests but fail any other test will be kept in $\mathbb{V}_{patches}$ as they can serve as helpful patch ingredients for future generations, while plausible patches will be kept in $\mathbb{H}_{patches}$. Patch generation ends when $\mathbb{H}_{patches}$ reaches a predefined upper-limit size, or when timeout occurs.

The test generation and patch assessment (Steps 4 and 5) aim at killing overfitting patches. It generates new test cases guided by the current patches in $\mathbb{H}_{patches}$. Patches that fail the additional tests are moved to $\mathbb{V}_{patches}$, while patches that pass all new tests remain in $\mathbb{H}_{patches}$.

The overall workflow ends after a predefined timeout or when the user stops the search. EvoRepair generates the sets $\mathbb{H}_{patches}$ (the most promising repair candidates that survived all generated test cases) and $\mathbb{H}_{tests}$ (the augmented test suite as an additional artifact for the user).

```
Topic:
denominatorDegreeOfFreedom in FDistribution leads
to IllegalArgumentsException in
UnivariateRealSolverUtils.bracket

Description:
The problem is the 'initial' parameter to that
function, wich is POSITIVE_INFINITY and therefore not
within the boundaries. I already pinned down
the problem to the FDistributions
getInitialDomain()-method, wich goes like:

return getDenominatorDegreesOfFreedom() /
(getDenominatorDegreesOfFreedom() - 2.0);

Obviously, in case of  denominatorDegreesOfFreedom
is 2, this must lead to a division-by-zero, resulting
in POSTIVE_INFINITY.
```

(a) bug report for MATH-227

```
1 double getInitialDomain(double p, double d) {
2    double ret;
3    ret = d / (d - 2.0);
4    return ret;
5 }
```

(b) buggy code in MATH-227

```
1  T wrapper_method(Parameters p ...) {
2    if (instrumentation_enabled) {
3      T result = original_method(p);
4      if (<condition_for_buggy_behavior>) {
5        throw new RuntimeException("[
               Defects4J_BugReport_Violation]");
6      }
7        return result;
8    } else {
9        return original_method(p);
10   }
11 }
```

(c) general oracle instrumentation template

```
1  double getInitialDomain(double p, double d) {
2     if (Boolean.parseBoolean(System.getProperty("defects4j.
          instrumentation.enabled"))) {
3        double init = getInitialDomain_original(p, d);
4        double lowerBound = 0;
5        double upperBound = Double.MAX_VALUE;
6        if (init < lowerBound || init > upperBound) {
7           throw new RuntimeException("[
                 Defects4J_BugReport_Violation]");
8        }
9        return init;
10    } else {
11       return getInitialDomain_original(p, d);
12    }
13 }
```

(d) instrumentation for MATH-227

Figure 2.2: Illustration of the oracle generation from a developer provided bug-report and the buggy function, based on MATH-227 in Defects4J benchmark.

2.2.2 Illustrative Example

To further illustrate our proposed workflow, we show an example that is inspired by the bug report for MATH-227 [Hen08], and the corresponding subject in Defects4J *math-95*. The original bug is in the class `FDistributionImpl` inside the method `getInitialDomain`, which accesses the initial domain value for an F-distribution. In our example, we kept the nature of the bug but simplified the context. Figure 2.2a depicts a snippet of the bug report for MATH-227, describing the flawed behavior and the expected behavior.

The buggy program (see Figure 2.2b) takes the desired probability p and the denominator degrees of freedom d. In particular, there are two incorrect behaviors: (1) for $d = 2$ there is a divide by zero error, because the divisor in line 2 is zero, and (2) for $d = 1$ the return value is negative. Both of these are unexpected behavior, which was reported by a user in the bug report (Figure 2.2a). Therefore, we create an oracle (see Figure 2.2d) in Section 2.3, which makes the boundary check explicit and throws a custom exception, which is recognized by our test generation.

We assume that there is one failing developer test t_f with $p = 0.49$, $d = 2.0$, and an assertion that checks $ret < M$ for large constant M. In the execution of EvoRepair, Step 2 generates

```
1 double getInitialDomain
2 (double p, double d) {
3   double ret = 0.0;
4   if (p > 0.5)
5     ret = d /(d - 2.0);
6   return ret;
7 }
```

(a) Patch p_1

```
1 double getInitialDomain
2 (double p, double d) {
3    double ret;
4    ret = d /(d - 1.0);
5    return ret;
6
7 }
```

(b) Patch p_2

```
1 double getInitialDomain
2 (double p, double d) {
3   double ret = 1.0;
4   if (d != 2.0)
5     ret = d /(d - 2.0);
6   return ret;
7 }
```

(c) Patch p_3

```
1 double getInitialDomain
2 (double p, double d) {
3   double ret = 1.0;
4   if (d > 2.0)
5     ret = d /(d - 2.0);
6   return ret;
7 }
```

(d) Patch p_4

Figure 2.3: Evolution of patches captured in p_1, p_2, p_3 and p_4, generated through multiple iterations of coevolution.

two plausible patches so that we have

$$\mathbb{H}_{patches} = \{p_1, p_2\}$$

Both p_1, p_2 pass the initially failing test case t_f. After returning to Step 1 and deciding that new tests should be generated, Step 6 produces a new test t_1 with $p = 0.6$ and $d = 2.0$ that *kills* p_1. Note that t_1 (as all additionally generated tests) does not define an assertion inside the test method but instead uses our custom oracle $ret < M$. After the patch assessment in Step 5, the patch sets are updated to

$$\mathbb{H}_{patches} = \{p_2\} \quad \mathbb{V}_{patches} = \{p_1\}$$

Inspired by the edits in p_1 and the updated $\mathbb{H}_{tests} = \{t_1\}$, a new iteration of repair generation produces the patch p_3 that passes all current tests. Another round of test generation produces t_2 with $p = 0.49$ and $d = 1.0$ that *kills* p_2 and p_3 so that after patch assessment we have

$$\mathbb{V}_{patches} = \{p_1, p_2, p_3\} \quad \mathbb{H}_{patches} = \emptyset \quad \mathbb{H}_{tests} = \{t_1, t_2\}$$

Figure 2.3 shows all generated patches and the resulting final patch and test sets are:

$$\mathbb{V}_{patches} = \{p_1, p_2, p_3\} \quad \mathbb{H}_{patches} = \{p_4\} \quad \mathbb{H}_{tests} = \{t_1, t_2\}$$

We illustrate the concept of the coevolution of tests and patches, and that using bug-report-derived oracles, the test generation was able to kill overfitting patches. Overfitting patches

are not completely discarded but re-purposed as seeds and ingredients for future repair generations, and finally, are evolved to the resulting correct patch.

2.3 Test Oracles

One of the fundamental challenges in software testing is the *test oracle* problem [Wey82, ABC+13, BHM+15]. It is concerned with determining the expected behavior of a program. In our case, we have to reason about the additionally generated tests, and hence, we also need an oracle. There are learning-based test oracle construction techniques like SEER [IVTJ22], TOGA [DRML22], and ODS [YGM+22], and other heuristics-based techniques [TYPR16, XLZ+18, GM22] to identify likely overfitting or likely incorrect patches. However, all these techniques only provide an approximation. While they can be used for patch assessment and ranking, we cannot use them to confidently prune patches in our coevolution context. Therefore, we decided to use the natural language specifications available in the bug reports in the Defects4J benchmark to create *user-originated* oracles. Techniques like JDoctor [BGK+18] and MeMo [BGE+21] can be used to extract specifications from Javadoc comments, but they strongly depend on the quality and structure of code comments. Preliminary results showed that they do not perform well on the bug reports included in Defects4J, which is why we opted to *manually* extract the oracles from the bug reports. We envision that the bug reporters could also write such oracles. Therefore, we only use the information provided by the reporter rather than the follow-up discussion of the maintainers, which is often also included in the online bug reports. For creating these oracles, we focused on the 59 subjects for which the state-of-the-art search-based APR technique ARJA [YB20a] can generate plausible patches. We instrument the original buggy program to check for the buggy behavior mentioned in the bug report. If the buggy behavior is detected, the instrumentation will throw a custom exception, which will cause the abortion of the original program execution detected by our framework. To some extent, this represents a hand-written, bug-report-derived, functionality based oracle.

2.3.1 Oracle Writing Methodology

Using the bug report's description, we first identify the program location (i.e., class and method) where the incorrect behavior is observed. Then, we extract the condition under which the incorrect behavior is described. The instrumentation usually involves writing a wrapper function around the original code, which checks for a specific condition and throws a custom exception if the condition is satisfied. In general, we applied the template shown in Figure 2.2c.

Some bug reports mentioned unexpected exceptions, so we need to add a try-catch-block around the original method call. Note that our instrumentation can be dynamically enabled (line 2 in Figure 2.2c), which we enable by checking whether the system property

`"defects4j.instrumentation.enabled"` is set. Therefore, we can enable the oracle only for the execution of the newly added test methods. This becomes necessary because our additional oracles can cause initially passing test cases to fail. The reason is that our oracles make the buggy behavior explicit and throw a custom runtime exception, which might not be handled by the existing test cases. However, a correct patch should also pass our oracle. In our overall approach, we enable the oracles for newly generated tests only and disable them for the existing tests.

For example, let us have a look at the report for MATH-227 mentioned in Section 2.2. In the title of the report, the reporter mentions an unexpected IllegalArgumentsException. Further in the description, the reporter specifies that the problem is caused by parameter values being outside of expected bounds. Therefore, our instrumentation checks for these bounds and throws a custom runtime exception as shown in Figure 2.2d.

Note that our instrumentation is applied at the location where the bug reporter expects a program failure and makes the incorrect behavior explicit by throwing an exception. In contrast to other Information Retrieval (IR)-based techniques [KLB+19, MB23], this usage of the bug reports does not serve the fault or fix localization but solely has the purpose of guiding the test generation. Further, we believe that creating such oracles needs a minor additional effort so that a bug reporter could provide them. Such an oracle also could be simplified to conventional assertions.

2.3.2 Failed Attempts

From the 59 subjects, we successfully created 39 oracles. We could not formulate a meaningful oracle for the other 20 subjects because of missing information about the expected behavior (14/20) or because no bug report was available (6/20). In general, we require information about the condition under which we can observe the failure and the observation location. In most of the cases with insufficient information (7/14), the report only provided a failing test case that did not allow for generalization beyond the specific test. For six other reports, there was no precise description of the *condition* under which the program behaves incorrectly. For example, the report for MATH-949[1] describes that a method incorrectly always returns zero, but did not specify *when* this is incorrect. Further, one bug report only suggested a fix without mentioning the buggy behavior. More information on these bug reports can be found in our artifact.

2.3.3 Oracle Types

During the oracle creation, we observed a few patterns that can be categorized as follows. For 20/39 subjects, the bug report described that a particular exception, e.g., a null pointer exception, should not be observed, indicating incorrect behavior. For these subjects, we throw

[1] https://issues.apache.org/jira/browse/MATH-949

our custom exception when catching the described exception to make the error explicit. For 12/39 subjects, the reports mentioned a general condition on the output or an intermediate value. In particular, for 6/39 subjects, the reports described specific expected value bounds. Additionally, one bug report described a property that can be used for differential testing: a Java class was found to be buggy by checking its output against another class that implements the same functionality (see MATH-631[2]).

2.4 Coevolutionary Repair

We propose a program repair framework based on coevolution. Coevolution is defined as the simultaneous evolution of multiple populations that belong to different domains; the respective fitness functions guide the evolution of the populations. In our case, we would evolve two populations: a population of *patches* and a population of *test cases*. Specifically, given a buggy program, an initial test suite, and an oracle, we first search for some plausible patches to form the initial patch population. The test suite and the patches are then evolved alternately in search of test cases that invalidate the current patches and of patches that pass the growing test suite.

The key observation behind the framework is that program repair can be seen as a competitive coevolution problem. Competitive coevolution is one where the fitness of individuals from different populations are inversely related; the populations thus compete against each other. In the context of program repair, a better patch can pass more test cases, while a better test case can fail more patches. This duality makes a natural competitive coevolution problem.

The benefit of the coevolutionary approach is also dual. On the one hand, evolving the population of test cases improves the specification of the buggy program and helps generate less overfitting patches [SBLGB15]. On the other hand, evolving the population of patches is conducive to better test cases that prune the patch pool more effectively.

Evolution of either population is performed by means of an evolutionary search. An evolutionary search is guided by a fitness function f. In each generation, the fitness of each individual in a population is computed with f, based on which some individuals are selected as parents. The parents then undergo mutation and crossover to yield the next generation. Note that patches and tests have their respective mutation and crossover operators.

In our coevolutionary repair, the fitness of a patch is related to its execution result on a selected set of test cases that have failed some patch before. We denote this set with $\mathbb{H}_{tests}$ and denote the resulting fitness function with $f_{\mathbb{H}_{tests}}$. Likewise, we have $\mathbb{H}_{patches}$ and $f_{\mathbb{H}_{patches}}$ for guiding the evolution of test cases. We summarize the workflow of coevolutionary repair in Algorithm 2.1. The patch population is initialized with $\mathbb{H}_{patches}$, $\mathbb{V}_{patches}$ and some random patches (to escape possible local optima). The population then undergoes evolution under

[2] https://issues.apache.org/jira/browse/MATH-631

Algorithm 2.1: Coevolutionary Repair

Input: buggy program, developer test suite
Output: plausible patches $\mathbb{H}_{patches}$
Output: additional test cases $\mathbb{H}_{tests}$

$\mathbb{H}_{patches} \leftarrow \emptyset$; $\mathbb{V}_{patches} \leftarrow \emptyset$; $\mathbb{H}_{tests} \leftarrow \emptyset$
while *within timeout* **do**
  if *evolve patch domain* **then**
    $P \leftarrow \mathbb{H}_{patches} \cup \mathbb{V}_{patches} \cup randomPatches()$
    $P_{surviving}, P_{partial} \leftarrow evolve(P, f_{\mathbb{H}_{tests}})$
    $\mathbb{H}_{patches} \leftarrow \mathbb{H}_{patches} \cup P_{surviving}$
    $\mathbb{V}_{patches} \leftarrow \mathbb{V}_{patches} \cup P_{partial}$
  else
    $T \leftarrow \mathbb{H}_{tests} \cup randomTests()$
    $T_{killing} \leftarrow evolve(T, f_{\mathbb{H}_{patches}})$
    $\mathbb{H}_{tests} \leftarrow \mathbb{H}_{tests} \cup T_{killing}$

    $\mathbb{H}_{patches}, P_{killed} \leftarrow update(\mathbb{H}_{tests}, \mathbb{H}_{patches})$
    $\mathbb{V}_{patches} \leftarrow \mathbb{V}_{patches} \cup P_{killed}$
  end
end
return $\mathbb{H}_{patches}$, $\mathbb{H}_{tests}$

the guidance of $f_{\mathbb{H}_{tests}}$ to yield plausible patches $P_{surviving}$ that pass the developer test suite and the tests in $\mathbb{H}_{tests}$. We also keep *partial* patches that pass the developer test suite but fail on other tests in $\mathbb{H}_{tests}$; these are stored in $\mathbb{V}_{patches}$, while $P_{surviving}$ are added to $\mathbb{H}_{patches}$. Evolution of test cases is similar: the tests in $\mathbb{H}_{tests}$ and some random tests are evolved with the goal to find new tests ($T_{killing}$) that kill patches in $\mathbb{H}_{patches}$. These tests are added to $\mathbb{H}_{tests}$. Afterwards we update $\mathbb{H}_{patches}$ and $\mathbb{V}_{patches}$ accordingly. We provide greater details of our approach in the subsections below.

2.4.1 Search Space

We construct the search space of patch generation with ARJA-e, which uses the statements in the buggy program as well as a set of predefined templates as fix ingredients. The search space of test generation is composed of all possible sets of statements of sizes from 1 to N (i.e. $n \in [1, N]$) where each test case can have a size from 1 to L (i.e. $l \in [1, L]$).

The search space of EvoRepair is the combination of the program repair space and the program test space. Such a large search space requires an optimized navigation strategy [NSGR22]. Previous work has shown that a coevolutionary search is much more effective for navigating such a large search space than an evolutionary search [PM02]. In addition, such algorithms

can adaptively focus on relevant areas in the search space due to the mutability of the fitness landscape.

2.4.2 Fitness Functions: Patch Generation

Compared to most coevolutionary algorithms, which use a single objective for the selection of the population, the two populations in our framework both require multiple objectives. There are several benefits of using multi-objective fitness functions to search for plausible patches in the context of program repair [YB20a]. An additional helper function (i.e., minimizing the patch size) can alleviate the problem of convergence to local minima. In addition, generating a plausible patch is necessary but not sufficient for the developer. We leverage the same fitness functions as defined in previous work [YB20b] that uses the *patch size* [YB20a, WSKLG21, YB20b] and the *failure rate* on tests [YB20a, WNGF09, YB20b], and formulate the optimization problem as follows:

$$\begin{cases} min\ f_1(p) = \sum_{j=1}^{n} b_j \\ min\ f_2(p) = \frac{\sum_{t \in T_{pos}} h(p,t)}{|T_{pos}|} + w \times \left(\frac{\sum_{t \in T_{neg}} h(p,t)}{|T_{neg}|} \right) \end{cases} \tag{2.1}$$

where $f_1(p)$ represents the size of the patch p by counting the number of edit operators in the patch, and $f_2(x)$ combines the failure rate of the positive tests T_{pos} with the weighted failure rate of the negative tests T_{neg}. $w \in (0, 1]$ controls the bias towards negative test cases. $h(p, t)$ indicates how badly a test failed on the patch by computing how far an assertion failure was from the expected value (see details in [YB20b]). We employ NSGA-II [DPAM02] to minimize both objective functions.

2.4.3 Fitness Functions: Test Generation

For test cases, the fitness is computed using many objectives: (1) the ability to cover the fix locations, (2) the ability to cover the oracle location, (3) the ability to cover a target location (i.e., fix or oracle location) through different contexts, and (4) the ability to trigger our oracle's custom exception during mutation testing. The third and fourth objectives are inspired by the intuition that a test case is more likely to invalidate a patch if it tests the patch in multiple different ways. The optimization problem can be formalized as follows:

$$\begin{cases} min\ g_1(t,p) = d(p_{loc}, t) \\ min\ g_2(t,p) = d(o_{loc}, t) \\ min\ g_3(t,p) = d_{context}(\{p_{loc}, o_{loc}\}, t) \\ min\ g_4(t,p) = d_{mutant}(p_{loc}, t) \end{cases} \tag{2.2}$$

- $g_1(t, p)$ computes the distance of test t from covering the fix location $p._{loc}$ using approach level and normalized branch distance [McM04]; similarly, $g_2(t, p)$ targets the oracle location o_{loc}.
- $g_3(t, p)$ computes the distance to a target location (i.e., fix or oracle location) *through* a particular context (i.e., call stack): We want to explore the different contexts where each control dependency of a target location is indirectly covered through invocation of other (public) methods. The set of possible contexts can be (statically) derived from the call graph of the class under test. Given a test that covers a particular context, the fitness is computed as the target location distance. If a test does not trigger this context, it is assigned maximum distance instead. This objective is a variation of the *direct branch coverage* criterion [RCV+15].
- $g_4(t, p)$ computes the infection and propagation distance [FA15] w.r.t. a fix location mutant: To assess the sensitivity of test cases to changes at the fix locations, we perform *strong mutation testing* [JH11, FA15] by applying small mutants at all fix locations and guide test generation towards "killing" these mutants. A mutant is considered to be killed if it leads to a state infection that propagates to an assertable output difference. In patch testing, we are particularly interested in killing mutants by throwing the custom oracle exception. By targeting only the fix locations, we further tackle the scalability issue of traditional mutation testing and control the budget allocation.

We employ DynaMOSA [PKT18] to optimize the selection of test cases and objectives for this many-objective problem over all patches $p \in \mathbb{H}_{patches}$. In particular, we compute structural dependencies between the individual objectives in the same manner as for the traditional test case generation setting (i.e., based on the control dependency hierarchy). Note that for the distance calculation we execute the original program.

2.5 Implementation

Our approach is implemented on top of EvoSuite [FA11], a testing framework based on multi-objective evolutionary algorithms. It has interfaces for numerous evolutionary algorithms such as MOSA [PKT15], DynaMOSA [PKT18], and NSGA-II [DPAM02], and allows easy addition of new fitness functions. Below, we explain our extensions to EvoSuite, which have allowed it to perform coevolution of patches and tests.

2.5.1 Test Generation

We base our test generation on the DynaMOSA algorithm [PKT18]. Originally, DynaMOSA has been introduced in the context of the many-objective coverage problem for automated test case generation. Our goal is not to produce a general test suite but to generate test cases that can identify overfitting patches. Therefore, we adapt the DynaMOSA implementation in EvoSuite as follows:

- *Population Initialization:* Instead of *randomly* initializing the population, we use seeds from $\mathbb{H}_{tests}$.
- *Test Case Evaluation:* Our search employs several customized fitness functions as introduced in Section 2.4.3.
- *Extended Coverage Archive:* The original coverage archive in DynaMOSA contains test cases that reach a target location not reached by previous test cases. We have added to the archive test cases that kill program mutants at the fix locations as well as any test case that covers one of the target locations (i.e., fix or oracle location). These additional test cases are helpful because a single test reaching a location is usually not sufficient to detect program behavior changed by a patch.

2.5.2 Patch Generation

The evolution in the space of program edits is based on ARJA-e [YB20b], which represents patches as chromosomes, defines crossover and mutation operators of the chromosomes, and evolves chromosomes by the NSGA-II algorithm. For better usability, we have ported the patch representation and the operators to EvoSuite, which are interfaced with EvoSuite's NSGA-II implementation. In addition, we make the following changes to EvoSuite and ARJA-e:

- *Avoiding Changes to the Oracles:* For generated test cases to rule out overfitting patches, an injected oracle should remain in the program after a patch is applied. Therefore, we identify the oracles from abstract syntax trees and prevent generating patches that alter or remove them.
- *Avoiding Using Oracles in Repair:* We prevent using statements from the oracles as fix ingredients, though these statements can serve as fix ingredients in principle. This is because we need to compare EvoRepair with other tools, which may generate patches that change the oracle and thus cannot handle a program with an oracle.
- *Taking Seed Patches:* Just as we provide seed test cases for test generation, we also provide seed patches for patch generation. A portion of the seeds are "best-so-far" patches coming from $\mathbb{H}_{patches}$. Another portion comes from $\mathbb{V}_{patches}$. By combining the two seed sources, we hope to breed offspring patches that pass more test cases while avoiding falling into local optima.

2.6 Evaluation

In this section, we present the results of our evaluation for the proposed coevolutionary repair process. We first show the general effectiveness of EvoRepair in repairing bugs. We then show that patch and test generation benefit each other, confirming the usefulness of the coevolutionary approach. Overall, we investigate the following research questions.

RQ1 How effective is EvoRepair in repairing bugs compared to the state of the art of Java program repair?

RQ2 What is the impact of coevolution on patch generation?

RQ3 What is the impact of coevolution on test generation?

2.6.1 Evaluation Subjects

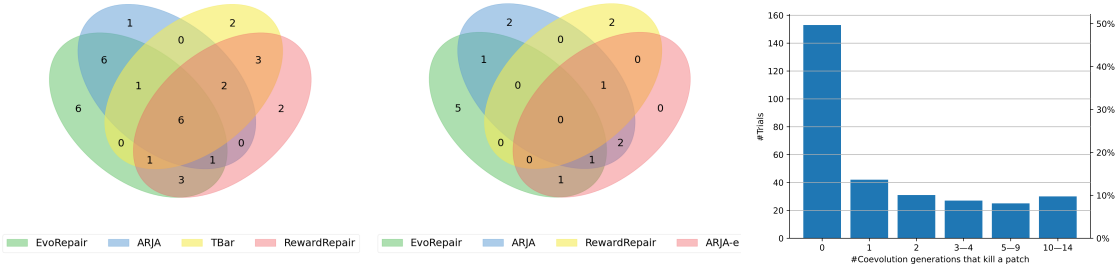
For our evaluation we use a subset of subjects from the popular Defects4J [JJE14] benchmark v1.0.1. We selected subjects for which we were able to generate a test oracle using the bug report as discussed in Section 2.3. Namely, we consider four projects from Defects4J: `Chart`, `Lang`, `Math` and `Time`. In total, we include the 39 subjects for which we were able to generate a test oracle (see Section 2.3). Given the large amount of CPU time required to run the experiment, we focus on this limited subject set that can be fixed by our baseline tool ARJA [YB20a]. We do not consider the other 20 subjects that do not have a test oracle because we focus on the coevolution of tests and patches.

2.6.2 Evaluation Tools

We compare our proposed technique EvoRepair with several state-of-the-art tools for Java program repair. We evaluate the repairability for our subset of defects using ARJA [YB20a], ARJA-e [YB20b], jGenProg [MM16], Cardumen [MM18], jKali [QLAR15], jMutRepair [DW10], Nopol [XMD+17], TBar [LKKB19], and RewardRepair [YMM22]. We use the Astor framework [MM16], which implements the techniques Cardumen, jKali, jMutRepair, and jGenProg. Similarly, we reuse the ARJA [YB20a] framework for the implementations of ARJA and ARJA-e. Since RewardRepair assumes perfect fault localization, we compute a list of fix locations prior to invoking the model, using the Ochiai algorithm [Och57] via the GZoltar [CRPA12] framework. Note that only EvoRepair runs on the instrumented version; all other tools are executed on the original buggy program. EvoRepair does not use the instrumentation for fix localization and instead relies on the built-in spectrum-based fault localization in ARJA-e.

2.6.3 Experimental Setup

All our experiments were conducted on servers with 32 vCPU and 64GB memory using Docker containers. Additionally we provide an NVIDIA GeForce RTX 4090 GPU for RewardRepair. We set a 2h timeout for all our experiments per repair task following previous empirical studies [DMMA19]. Based on previous test runs, we set the internal timeout for patch generation to 10 minutes and for test generation to 1 minute in each coevolution iteration. The experiment on each subject is repeated 10 times. For each tool, we report the number of bugs for which it can consistently generate a plausible patch in all 10 runs. We also report the cumulated total number of bugs fixed in at least one of the 10 runs. For the number of



(a) Plausible Patches **(b)** Correct Patches **(c)** Distribution of Patch Killing

Figure 2.4: (a) Venn diagram of bugs for which repair tools found a plausible patch in every repetition. Omitted from the figure are jGenProg and ARJA-e. They consistently fixed 13 and 6 bugs, with 8 and 4 bugs overlapping with EvoRepair. The other tools only fixed a subset of the bugs fixed by jGenProg. (b) Venn diagram of bugs for which repair tools found a correct patch in at least one repetition. (c) Distribution of experimental trials by the number of generations where an overfitting patch is removed. The left axis shows the absolute number of trials while the right shows the relative number.

correctly fixed bugs (i.e., semantically equivalent to the developer fix), we only report the total over 10 runs in this chapter since correct patches are sparse. However, we include all experimental results in our artifact.

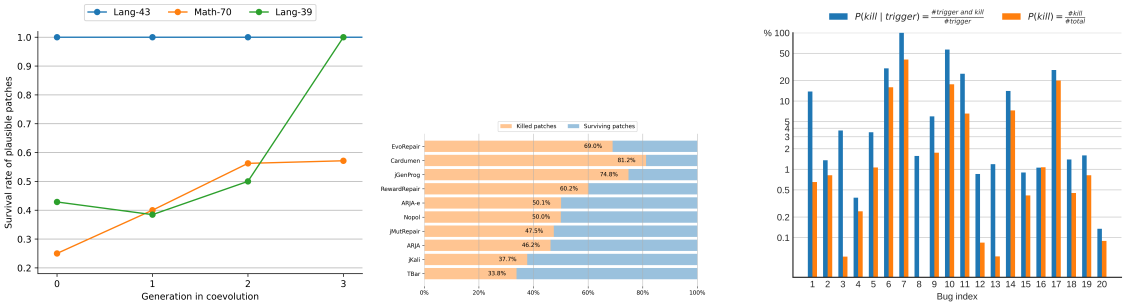
Seed Selection Parameters

EvoRepair seeds patch generation with patches from $\mathbb{H}_{patches}$ and $\mathbb{V}_{patches}$. In the experiment, we form $\frac{1}{2}$ of the initial population from $\mathbb{H}_{patches}$ and $\frac{1}{4}$ from $\mathbb{V}_{patches}$, and the remaining $\frac{1}{4}$ is randomly initialized. When $\mathbb{H}_{patches}$ and $\mathbb{V}_{patches}$ do not have enough patches, random patches are used instead. For test generation, $\frac{1}{2}$ of the initial population is from $\mathbb{H}_{tests}$, and the other $\frac{1}{2}$ is randomly initialized.

2.6.4 RQ1: Efficacy of EvoRepair

Figure 2.4a depicts the number of bugs each tool could plausibly fix in every one of the 10 runs. For brevity, we omit numbers from Cardumen, jKali, and jMutRepair as they are a subset of jGenProg. For 24 bugs, EvoRepair finds a plausible patch in every repetition, and there are 33 bugs for which EvoRepair was able to find a plausible patch in at least one repetition. In comparison, RewardRepair consistently fixes 18 bugs; ARJA finds a plausible patch for 17 bugs in every repetition and for 21 bugs in at least one repetition; TBar consistently fixes 15 bugs, and ARJA-e finds a plausible patch for 6 bugs in every repetition and for 19 bugs in at least one iteration. None of the other tools have fixed more than 15 bugs.

EvoRepair also produced more correct patches. As shown in Figure 2.4b, over the 10 repetitions, EvoRepair correctly fixed 8 of the 39 bugs. This is followed by ARJA, ARJA-e,



(a) Survival Rate **(b)** Killed Patches **(c)** Probability of Killing a Patch

Figure 2.5: (a) Change in the survival rate of plausible patches over coevolution generations in three representative trials. (b) Percentage of patches of each repair tool killed by additional test cases. (c) Probability of a triggering test and an average test to kill patches on 20 bugs (y-axis on a log scale).

and RewardRepair, which correctly fixed 7, 5, and 3 bugs. The other tools correctly fixed no more than 2 bugs each. Note that, although ARJA and ARJA-e had similar performance with EvoRepair, they generated more than 200 plausible patches on average for each bug. In contrast, EvoRepair maintains no more than 20 patches in the hall of fame. We also notice from Figure 2.4b that EvoRepair, ARJA, and RewardRepair have several uniquely fixed bugs, due to the different search strategies of the tools.

2.6.5 RQ2: Impact of Coevolution on Patch Generation

Intuitively, patch quality is improved because the additional test cases can remove overfitting patches during the search for a correct patch. As will be shown below, this intuition is supported by our experiments.

We have collected the number of plausible patches generated and the number of patches killed in our experiments. Overall, about 14,500 patches were generated, of which over 10,000 were killed, making a significant reduction of 69.0% to the patch pool. We further investigate how these reductions are distributed among experimental trials and among the coevolution generations within the trials. Of our 390 trials (39 subjects $\times$ 10 repetitions), 308 have generated plausible patches. In these trials, it is desirable if overfitting patches have been detected in different generations, which would mean that the test cases were constantly pruning the patch pools. We plot in Figure 2.4c the distribution of the 308 trials by the number of coevolution generations where an overfitting patch was detected. A total of over 35% of the trials have plausible patches killed in more than one generation, and over 25% of the trials have plausible patches killed in three or more generations. This means that in a reasonable number of trials, the test cases are steadily removing overfitting patches.

The steady pruning of patch pools has led to a gradual improvement in the quality of the generated patches. The quality of patches found in a certain generation can be reflected by *survival rate*, which is the fraction of these patches that survive until the end of coevolution. From our 390 trials, we have collected the survival rate of patches for the 162 trials that have both plausible patches found in multiple generations and at least one surviving patch. These trials can be further divided into three categories, each represented by a line in Figure 2.5a. In 71 (43.8%) trials, survival rates were always one, represented by the Lang-43 line in Figure 2.5a, which means no test case was generated that could kill a patch. In 62 (38.3%) trials, survival rates have monotonically increased throughout the coevolutions, represented by the Math-70 line. Only in 29 (17.9%) trials, the survival rate dropped at some point; however, the survival rate may still go back up (see Lang-39).

In order to see whether coevolution can help improve patch quality of other repair tools, we have also pruned patch pools of the other tools with our test cases. Figure 2.5b shows the fraction of overfitting patches by each repair tool detected by the additional test cases generated by EvoRepair. While the test cases had not been generated to target these patches, more than 60% of the plausible patches generated by RewardRepair, jGenProg, and Cardumen are removed, and a minimum of 33% of the plausible patches generated by the other tools are also removed. The high proportions of detected overfitting patches show the great potential of coevolution in improving patch quality of different repair tools.

2.6.6 RQ3: Impact of Coevolution on Test Generation

This section demonstrates that our coevolutionary workflow improves the quality of generated test suites. In particular, we illustrate the effectiveness of our fitness functions by showing the effectiveness of our core intuition: triggering the custom exception on the *original* program is a good predictor of whether it can kill a patch, i.e., trigger the exception on a *patched* program. This intuition directly motivates the fitness function g_2 and indirectly also g_3 and g_4 (see Section 2.4.3).

To illustrate this, we have compared in Figure 2.5c how likely it is for a triggering test and for an average test to kill at least one patch. The comparison is performed on 20 bugs for which there are both triggering tests and killing tests. Note that the y-axis is on a log scale. A test that triggers the oracle on the original program has a much higher probability (10.9x on average and 70.4x maximum) of killing a plausible patch afterwards. In other words, the fitness function performs well in selecting potential killing tests.

2.7 Threats to Validity

External Validity

To mitigate the selection bias for our comparison with the state-of-the-art APR tools, we selected the seven best-performing tools reported in previously studied large-scale empirical

evaluations [DMMA19] and RewardRepair as the more recent state of the art learning based tool [YMM22]. Since our implementation is based on ARJA, we focused the benchmark selection on the subjects in Defects4J, for which ARJA can find plausible patches. In the future, we plan to evaluate the efficacy of EvoRepair on a more diverse set of subjects in recently proposed data-sets [LKCSL17, MUMM19] of Java bugs.

Internal Validity

EvoRepair and the state-of-the-art Java repair tools in our comparison have nondeterministic components, and hence, can produce different results for different runs. To alleviate the threat of random observations, we ran each of the repair tasks 10 times and report the average values. Our artifact includes all experimental results. An additional threat to internal validity is that the manually written test oracles may not be fully correct if the referenced bug reports are incomplete or incorrect. The thesis author and a co-author independently double-checked the correctness of written oracles with respect to the available bug reports. Finally, our experiments used a two-hour timeout, and it is possible that larger timeouts can lead to other observations.

Construct Validity

Our analysis focuses primarily on identifying test-adequate (i.e., plausible) patches; however, these patches can still be incorrect due to the overfitting issue [SBLGB15]. EvoRepair mitigates the overfitting issue by generating additional tests. Further, we also manually assess and report the correctness of the identified patches.

Chapter 3

Oracle-Guided Program Selection

3.1 Introduction

As discussed in Section 1.3, current strategies for mitigating incorrect LLM-generated code select the best program from multiple LLM-generated alternatives, judged by criteria like the consistency of their execution results on an LLM-generated test suite. However, this approach has crucial limitations: (1) LLMs often generate redundant tests or tests that cannot distinguish between correct and incorrect solutions, and (2) the consistency criteria, such as the majority vote, fail to foster developer trust due to the absence of a transparent rationale behind the choices made. In this chapter, we instead use the LLM as a test oracle over distinguishing inputs: we first generate distinguishing inputs that capture semantic discrepancies of programs sampled from an LLM, and record the outputs produced by the programs on these inputs. An LLM then selects the output most likely to be correct, guided by the natural language problem description.

Our experiments on HumanEval+ and MBPP+ show that this oracle-guided selection improves *pass@1* over the state-of-the-art CodeT from 80.8% to 87.7% and from 75.0% to 78.6%, respectively, while being 60% more efficient w.r.t. token cost. In addition, the selected input-output specifications helped us uncover incompleteness and ambiguities in task descriptions, and identify incorrect ground-truth implementations in the benchmarks.

Overall, we make the following contributions:

- We propose the idea of oracle-guided program selection for LLM-generated programs, which finds distinguishing inputs and leverages the natural language descriptions to select the correct outputs of distinguishing inputs to form a high-quality distinguishing test suite as specifications.
- In our experiments on HumanEval+ and MBPP+, we show that our approach, implemented in a tool LLMCodeChoice, enhances the accuracy of code generation compared with the state-of-the-art program selection techniques.
- We demonstrate that the inferred distinguishing input-output specifications are useful for detecting incompleteness and ambiguities in the problem descriptions.

3.2 Motivating Example

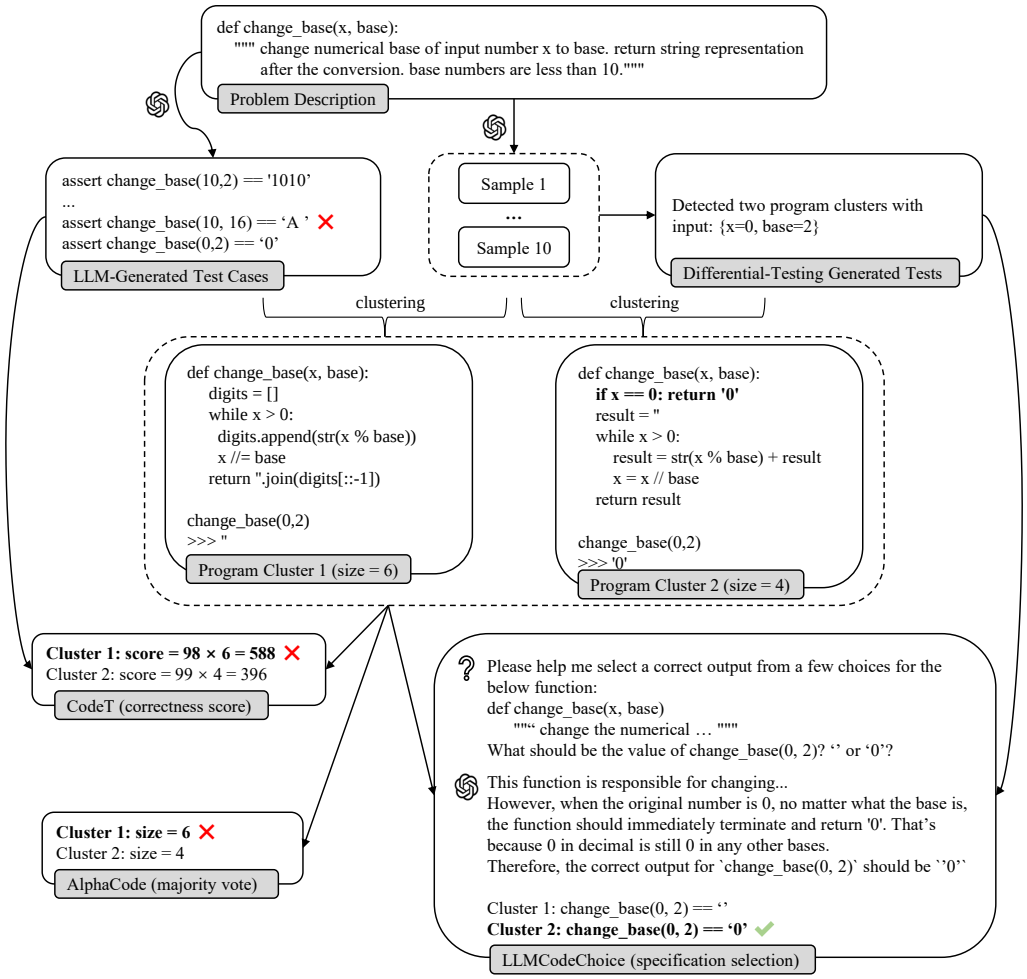


Figure 3.1: The workflows and selection results of LLMCodeChoice, AlphaCode, and CodeT on task 44 of HumanEval. Selection results are marked in bold. While AlphaCode and CodeT use LLM-generated tests to do program clustering and some form of majority vote, LLMCodeChoice obtains distinguishing inputs by differential testing and prompts LLM to select the correct outputs for these inputs based on natural language problem description.

In this section, we illustrate LLMCodeChoice with task 44 of the HumanEval benchmark. We compare our technique with CodeT [CZN+23], the state-of-the-art program selection technique, as well as AlphaCode [LCC+22]. Both CodeT and AlphaCode are based on majority vote and can improve the accuracy of program selection. However, they failed to select a correct program for this task, while LLMCodeChoice made a correct selection.

As shown at the top of Figure 3.1, the "change_base" task involves implementing a function that returns the string representation of the input number x after changing its numerical base to *base*, where *base* is less than 10, and we generated ten program samples with ChatGPT. We observe that six of the programs are equivalent, denoted with "Program Cluster 1" in Figure 3.1. Samples in program cluster 1 iteratively calculate the digits of x in the specified

base and then concatenate the digits to get the result string. Meanwhile, the other four samples take another solution, forming "Program Cluster 2" in Figure 3.1. This solution is largely the same as that of program cluster 1, except that it correctly returns the string '0' in case $x = 0$. Because the representation of the number 0 would still be 0 for any base. On the other hand, since program cluster 1 ignores this edge case, it would return the wrong result of an empty string when $x = 0$. This confusion is likely due to the incompleteness of the natural language specification of the 'change_base' task. The task description in Figure 3.1 specifically mentions that *base* is less than 10, but leaves the range of the input x unspecified, which leads to two different understandings in the LLM-generated programs. The surprising fact is that the canonical solution of this task provided by experienced programmers in HumanEval also has this mistake, which underlines the importance of providing supporting evidence for LLM-generated programs to developers.

AlphaCode and CodeT AlphaCode and CodeT perform majority vote based on program execution results on a set of LLM-generated test cases. For this example, we generated 100 test cases with GPT-4, using the problem description as the prompt. We notice that the test cases include a distinguishing input $x = 0, base = 2$, which would allow AlphaCode and CodeT to divide the programs into two clusters. There is also a test case $x = 10, base = 16$, though the problem description specifies that *base* should be less than 10. Both clusters would fail this test due to the unexpected *base* value.

AlphaCode executes all the programs with the LLM-generated test inputs and groups programs that produce the same outputs into one cluster, and then randomly selects one program from the largest program cluster as the correct program sample. In the example of Figure 3.1, AlphaCode selects a sample from program cluster 1, because this cluster is the larger one. Similar to AlphaCode, CodeT tracks the passing tests for each LLM-generated program regarding the LLM-generated test suite, and classifies programs that pass the same tests into one program cluster. CodeT then assigns a correctness score defined as (size of a program cluster) $\times$ (number of passing tests of a program cluster) to each program cluster, and selects programs from the cluster with the highest score. In Figure 3.1, cluster 1 passes 98 test cases (failing (0,2) and (10,16)) and has size 6, so its correctness score is calculated as $98 \times 6 = 588$. Cluster 2 does not fail the $x = 0$ test case and has a score of $99 \times 4 = 396$. Since cluster 1 has a higher score, CodeT also selects program cluster 1 as correct. Therefore, both AlphaCode and CodeT made the wrong selection for this example.

Our Approach. Instead of solely relying on LLMs to generate test cases based on incomplete natural language specification, we propose to search for a small yet precise distinguishing test suite. The small size of the test suite makes it affordable to be shown to developers as correctness evidence for selected LLM-generated programs. We illustrate the construction of the distinguishing test suite through the example in Figure 3.1. We set a postcondition for all program samples from LLM that their outputs must be equivalent

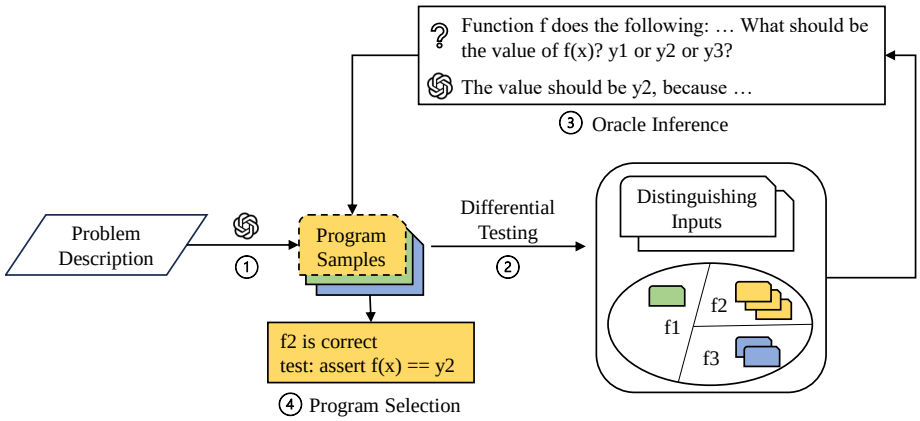


Figure 3.2: Overview of Oracle-Guided Program Selection

for any given input. Taking the postcondition as a guard and leveraging testing techniques, we can easily find one counter-example $x = 0, base = 2$ that violates the postcondition as a distinguishing input and thus produces two program clusters. This counter-example shows that the natural language specification is incomplete for the test input $x = 0, base = 2$. We provide insight into the possible outputs by executing the two program variants of 'change_base' and get two possible specifications.

$$\texttt{change_base(0,2)=="0"}\ OR\ \texttt{change_base(0,2)==""}.$$

Then we leverage LLM to select one correct output from the two specification options based on the problem description. Experimentally, we show that LLM is more accurate in selecting the output from several options rather than predicting the output from scratch, which we believe explains a better success rate of our approach compared to CodeT. Combining the distinguishing input and the selected output, we curate an input-output specification, which not only improves program selection accuracy but may also enhance the trust of developers in LLM-generated code, since input-output examples are easy to interpret for humans.

3.3 Methodology

LLMCodeChoice accepts a programming task description in natural language, which must satisfy the following assumptions:

- the description must contain information on how outputs of the program relate to its inputs,
- there must be a method of testing the generated code to find distinguishing inputs and the corresponding outputs,

- it must be possible to represent the input and the output as text that multiple outputs for distinguishing inputs fit into the LLM's context window.

LLMCodeChoice first samples a set S of programs from an LLM (step ① in Figure 3.2). Then, it performs differential testing to obtain *distinguishing inputs* witnessing behavioral differences among samples (step ② in Figure 3.2). Specifically, we say an input i is a distinguishing input, if there exist two samples s_1, s_2 such that $s_1(i) \neq s_2(i)$, where $s_1(i)$ and $s_2(i)$ represent the respective outputs of both samples on i. The set of distinguishing inputs, which we write $\mathcal{I}$, partitions the samples into clusters. Two samples fall into the same cluster if they produce the same output on every input.

With the distinguishing inputs and the program outputs on these inputs, we try to infer an input-output oracle for the problem. For every distinguishing input, we query the LLM about the corresponding correct output and then rule out all clusters incorrect on this input (step ③ in Figure 3.2). A query takes the form of a multiple choice question, choices consisting of all different outputs of the samples that have not been ruled out. In this way, we are able to gradually zoom in on and finally select the correct equivalence class (step ④ in Figure 3.2).

3.3.1 Differential Testing of Program Samples

We employ differential testing to generate distinguishing inputs. The generated distinguishing inputs would allow us to select the correct program more accurately, with less computational cost, and also facilitate the maintenance of the LLM-generated program in the long run if the program is integrated into a codebase.

The distinguishing input can help increase the accuracy of our program selection. This is because a distinguishing input clearly indicates when two programs would have different behavior. To make a choice between two different programs, it is enough to get the expected output of a distinguishing input and compare the actual outputs of both programs with the expected output. Intuitively, it is often an easier task to find out the one expected output than to compare the full semantics of two programs.

The distinguishing input also makes program selection more cost-effective, as the number of distinguishing inputs is often small. As shown in our experiment, ten LLM-generated programs on average represent no more than three kinds of different semantics, so two distinguishing inputs would be enough to differentiate the programs. Therefore, only a modest number of tokens need to be used to predict the expected outputs of the inputs using an LLM. In comparison, CodeT requires sampling many test cases from the LLM and thus incurs high computational cost.

An additional benefit of distinguishing inputs is that they facilitate the understanding and maintenance of the generated program. The inputs, together with their corresponding outputs, effectively form a test suite that specifies the desired program behavior. In case the generated program is integrated into a codebase, this test suite can be integrated for quality

assurance as well. As we will illustrate later in our case study, these inputs have helped identify ambiguous descriptions or even wrong ground truths in the evaluation datasets.

These benefits motivate us to aid program selection with distinguishing inputs. There is more than one way to generate these inputs. An obvious option is to use the LLM to generate test cases [CZN+23]. However, as discussed in the example in Section 3.2, distinguishing inputs tend to be rare in LLM-generated test cases. Also, the queries to LLM would incur significant computational costs. In light of these observations, we have opted to generate distinguishing inputs using differential testing. Specifically, we employ two test generation techniques, fuzzing and symbolic execution for differential testing. To differentiate two program samples, we search for an input violating an assertion we make that the program samples yield the same output on every input. First, we use a property based fuzz testing tool to find tests that produce different outputs in the two programs. This is done by differential fuzzing the two programs $P1, P2$ where for any input we check the outputs in $P1, P2$. If fuzzing fails to produce a distinguishing input within a time bound T, we employ a more systematic symbolic execution tool to find distinguishing inputs within the same time budget T. The code for both programs $P1, P2$ is renamed apart from input variables[1] and composed into a new program $Q_{P1,P2}$ which also contains the assertion $\varphi_{P1,P2}$ about equivalence of outputs of $P1, P2$

$$\varphi_{P1,P2} \equiv \text{out}(P1) == \text{out}(P2)$$

where $\text{out}(P1), \text{out}(P2)$ denotes the output variable for $P1, P2$ respectively. The symbolic execution engine analyzes $Q_{P1,P2}$ with the goal of finding an input violating $\varphi_{P1,P2}$.

We show our program selection algorithm in Algorithm 3.1, where lines 1—5 correspond to the differential testing process. The process of differentiating two samples is represented with the procedure **DiffTest** in line 4, which returns a singleton set of distinguishing inputs for two samples (or empty set if not found). To distinguish between more than two samples, we perform this pairwise differentiation for the multiple pairs made up from the samples (lines 3–5 of Algorithm 3.1). Although one can also differentiate all samples at once by making an assertion about all of them, pairwise comparison makes LLMCodeChoice compatible with any differential testing technique. The process of differential testing across samples produces a set of distinguishing program inputs. These distinguishing inputs effectively define an equivalence relation over the samples, where two samples are equivalent if they produce the same output on all the distinguishing inputs. We denote the equivalence relation with $\sim$ in line 5 of Algorithm 3.1; the resulting set of program clusters is the quotient set $C = S/\sim$. We then conduct oracle inference to select the correct program cluster from C.

[1]The only common variables across the two programs $P1, P2$ are the input variables, the rest are renamed apart.

Algorithm 3.1: Oracle-Guided Program Sample Selection

Input: Task description $task$, LLM $\mathcal{L}$, program samples S
Output: One selected cluster of program samples

1 $\mathcal{I} \leftarrow \{\ \}$
2 **for** (s_1, s_2) in $S \times S$ **do**
3 **if** $\neg\exists\, i \in \mathcal{I}.\, s_1(i) \neq s_2(i)$ **then**
4 $\mathcal{I} \leftarrow \mathcal{I} \cup \textbf{diffTest}(s_1, s_2)$
5 **end**
6 $C \leftarrow S/\sim$, where $s_1 \sim s_2 \iff \forall\, i \in \mathcal{I}.\, s_1(i) = s_2(i)$
7 **for** i in $\mathcal{I}$ **do**
8 $outputs \leftarrow \{\, c(i) \mid c \in C \,\}$
9 $o \leftarrow \textbf{oracleInference}(\mathcal{L}, task, i, outputs)$
10 $C \leftarrow \{\, c \in C \mid c(i) = o \,\}$
11 **if** $|C| = 1$ **then**
12 **break**
13 **end**
14 **return** the only remaining cluster in C

3.3.2 Specification Selection with LLM

In this section, we explain the key specification selection component of LLMCodeChoice, which uses the LLM to select the correct program specification for distinguishing inputs found in Section 3.3.1.

Given the distinguishing inputs and corresponding outputs from program clusters, our goal is to select the correct output for all distinguishing inputs and form the input-output program specification, thereby selecting the correct program cluster. Algorithm 3.1 details the specification selection and cluster selection process. LLMCodeChoice first selects the distinguishing input i having the highest entropy (line 6). The entropy of a distinguishing input i is defined as $-\sum_k \frac{n_k}{N} \log \frac{n_k}{N}$, where n_k is the size of the k-th program cluster, and N is the total number of programs. The intuition is that by using the input-output specification of a high-entropy distinguishing input to filter the programs, more programs can be ruled out with a single input, so that the number of queries made to the LLM can be minimized. For the selected input i, LLMCodeChoice then collects the different outputs $c(i)$ of each program cluster c (line 7), which have been recorded during differential testing. With the output options, we invoke the LLM to select the correct output (line 8) with the specification selection prompt shown in Figure 3.3. The prompt has three parts. First, the prompt frames the specification selection task for LLM as a multiple choice question. We provide the function signature and problem description for the LLM to have an initial understanding of the programming problem. Second, the prompt takes a distinguishing input and asks LLM to select the correct answer from the recorded output options. Our intuition is that the

```
# Specification selection as a multi-choice question
Please select a correct output from a few choices for below function.
# function signature and problem description
def foo(args):
  """problem description"""
# multi-choice to select correct output
What should be the value of ‘foo(i)‘? ‘output 1‘ or ‘output 2‘?
# Explain the choice
Please explain step by step and answer.
```

Figure 3.3: Prompt for Specification Selection

provided output options prune the search space of LLM, so selecting a correct output from multiple choices should be more effective than predicting the right execution result for a given input from scratch. This intuition was confirmed by our experiment. Moreover, we also integrate the popular chain-of-thought prompting [WWS+22] which has been shown effective in improving LLM's accuracy through a detailed reasoning step, by adding an "explain step by step" request at the end of the prompt. This step enables the LLM to explain the possible reasons that produce each output for a distinguishing input before the LLM selects answers. Afterwards, we keep only the clusters that produce the expected output and filter out the other clusters (line 9). We iteratively perform the specification selection and cluster filtering with each distinguishing input, until only one cluster satisfying all the selected input-output specifications is left (line 10—11). This one cluster is then considered the most likely to be correct and proposed to the user (line 12).

3.4 Experimental Setup

We implement the idea of oracle-guided program selection in a tool LLMCodeChoice and demonstrate its effectiveness by addressing the following research questions:

RQ1: How effective are distinguishing inputs in finding semantic discrepancies of LLM-generated programs?

RQ2: How effective is specification selection in LLMCodeChoice?

RQ3: How effective is LLMCodeChoice in the overall program selection?

RQ4: What are the reasons for failure of LLMCodeChoice?

In RQ1, we first compare the effectiveness of distinguishing inputs generated by LLMCodeChoice against the LLM-based test generation used in AlphaCode and CodeT on HumanEval and MBPP subjects. In RQ2, we evaluate LLMCodeChoice's specification selection accuracy for the generated distinguishing inputs to show the effectiveness of inferring specification with LLM. We then demonstrate the overall improvement of

pass@1 (probability of LLM-generated program being correct with only one sample) by applying LLMCodeChoice in selecting correct LLM-generated programs on HumanEval and MBPP benchmarks in RQ3. Finally, we discuss the reasons for incorrect specification selection in our method through more detailed analysis (RQ4). Additionally, we discuss the use of our input-output specifications in enhancing the trust of developers in LLM-generated programs (Section 3.5.5).

Benchmarks We evaluate LLMCodeChoice on two widely-used code generation benchmarks HumanEval [CTJ+21] and MBPP [AON+21]. HumanEval and MBPP are manually curated datasets that consist of 164 and 399 basic Python programming tasks. We removed the example input-output test cases in the problem descriptions of HumanEval, because the examples overlap with some test cases in the validation test suite. We modified the prompt of MBPP to the same format as HumanEval. Each prompt consists of a function signature and a natural language problem description in the docstring. However, a recent study [LXWZ23] shows that the validation test suite of HumanEval and MBPP benchmarks is relatively weak. This may cause the acceptance of plausible LLM-generated programs, which pass the original test suite but are actually incorrect. To mitigate such risks, we also evaluate LLMCodeChoice on the enhanced versions of HumanEval+ and MBPP+ [LXWZ23] equipped with a much more comprehensive test suite.

Implementation We use ChatGPT (snapshot gpt-3.5-turbo-0613) to generate solutions for all tasks in HumanEval and MBPP benchmarks. Specifically, we set the `max_token` parameter to 1024 which provides sufficient space for ChatGPT to complete all tasks in benchmarks, and we keep other parameters as default (e.g., temperature=1). Our differential testing engine (refer to Section 3.3.1) is built upon two Python automated testing frameworks Hypothesis [MHD19] and CrossHair [Sch23]. Hypothesis is a property-based testing tool using a random testing strategy to find counterexamples for postconditions, whereas CrossHair attempts to find counterexamples by exploring feasible execution paths with symbolic inputs. We set a 3-minute time bound for both tools. For the specification selection step, we use the GPT-4 (snapshot gpt-4-0613) model from OpenAI.

Metrics and Baselines We use the pass@1 metric [CTJ+21] to evaluate program selection accuracy. For each task, we use LLMCodeChoice to select one program cluster from all program samples. We then randomly select one program from the selected program cluster. The pass@1 of LLMCodeChoice on this task is the probability that the selected program passes all test cases in the benchmark. Suppose the selected cluster contains N programs, of which n programs pass all test cases, the pass@1 is then calculated as n/N. For both benchmarks, we report the average pass@1 across all tasks. We compare LLMCodeChoice against four baselines, (1) random selection which randomly selects one program from all LLM-generated programs, (2) AlphaCode [LCC+22], (3) AlphaCode with LLM-based specification inference,

and (4) CodeT [CZN+23]. The pass@1 of the baselines are calculated similarly. Since Codex (code-davinci-002) model used in the original CodeT has been deprecated by OpenAI, we replicated the test case generation process in AlphaCode and CodeT with the most powerful GPT-4 model by reusing CodeT's artifact and reported the latest result to ensure a fair comparison.

3.5 Evaluation

In this section, we first separately demonstrate the effectiveness of both components of LLMCodeChoice. We then show our improvement over the baselines in the overall selection accuracy. Finally, we discuss the limitations and unique advantages of LLMCodeChoice.

3.5.1 RQ1: Effectiveness of Test Generation

We measure the effectiveness of our test-generation component with the number of different clusters it can identify from the samples. The number of test inputs generated is also measured. It is desirable that the component can partition the samples into more clusters with fewer distinguishing inputs. This would mean that each input is of higher quality, and would make it easier for a human to examine these inputs and the resulting clusters.

We compare these two metrics of LLMCodeChoice with AlphaCode and CodeT. Note that, although AlphaCode and CodeT use the same set of LLM-generated tests, they can produce different clusters due to their different clustering strategies (see Section 3.2). Moreover, we compare with the "ground truth" clusters obtained by executing the validation test suites in HumanEval+ and MBPP+. Since the validation test suites contain a large number of carefully generated tests, one would expect them to find the most clusters.

Table 3.1 shows the metrics of LLMCodeChoice and the baselines. It can be seen that LLMCodeChoice outperforms all baselines on the MBPP benchmark. In particular, LLMCodeChoice finds 24.3% more program clusters than CodeT. On HumanEval+, the average number of program clusters identified by all tools is comparable. LLMCodeChoice performs slightly better than CodeT, but slightly worse than AlphaCode. Specifically, for 23 of the 164 tasks, LLMCodeChoice finds fewer clusters than AlphaCode. We have manually investigated these tasks, and found that these tasks have strict input constraints. For example, task 19 requires the input string to be space-delimited numerals ("zero" to "nine"). Such constraints are difficult for our differential testing engines based on random mutation and symbolic execution. However, there are also 21 tasks for which LLMCodeChoice found more clusters than AlphaCode. This implies that the LLM and traditional differential testing techniques can complement each other, e.g., seeding random mutation with an LLM-generated input. We leave the exploration of combining both techniques to future work.

Table 3.1: Average number of identified program clusters and average number of generated test cases of each tool on HumanEval+ and MBPP+.

	HumanEval+		MBPP+	
	avg clusters	avg tests	avg clusters	avg tests
Ground Truth	2.54	740.9	2.18	108.6
AlphaCode	2.56	89.2	2.56	85.3
CodeT	2.31	89.2	2.11	85.3
LLMCodeChoice	2.48	1.2	2.71	1.3

A noteworthy special situation during the testing is when no distinguishing input can be found. In this case, even if two programs actually have different semantics, LLMCodeChoice would place them in the same cluster and select them with equal probability. In the 563 tasks in both benchmarks (164 from HumanEval+ and 399 from MBPP+), the "ground truth" identifies 251 tasks as having semantically different programs, but LLMCodeChoice failed to find a distinguishing input for 35 of these. On the other hand, among the other 312 (=563-251) tasks whose programs are all deemed equivalent by the "ground truth", LLMCodeChoice managed to find a distinguishing input for as many as 119 tasks. This finding indicates that even a carefully-constructed test suite may fail to differentiate semantically different samples.

With regard to the number of test cases, LLMCodeChoice uses no more than 1.3 test cases for each task, which is about 60x fewer than AlphaCode and CodeT. With fewer test cases, LLMCodeChoice is able to identify a similar or larger number of program clusters, highlighting the usefulness of our test cases. The small number of distinguishing inputs allows them to be presented to developers as evidence before developers accept LLM-generated code, which we discuss further in Section 3.5.5.

3.5.2 RQ2: Effectiveness of Specification Selection

Specification selection is a key component of LLMCodeChoice, deciding whether the correct program cluster can be chosen. We measure its effectiveness by the frequency with which it makes a correct choice throughout our experiment. In total, our experiment includes 448 specification selection attempts made on 327 tasks of HumanEval and MBPP. Note that for the other 236 tasks from both benchmarks, as LLMCodeChoice has identified only one program cluster, there is no selection to be made. From the 448 selections, we count out the ones without a correct option. The correct option can be missing, either because all program samples are wrong, or because the correct programs have been excluded from the

Table 3.2: Accuracy of Oracle Inference Strategies: specification selection via LLMCodeChoice, specification inference from scratch from LLM, and majority vote.

	LLMCodeChoice	Direct Inference	Majority Vote
HumanEval+	87.6%	77.0%	71.3%
MBPP+	82.7%	68.2%	66.8%
Total	84.3%	70.3%	68.2%

options due to a previous wrong selection. This leaves 381 selections with the correct option available, and in these we count the number of times the correct option is actually chosen. The correct options have been obtained by executing the ground-truth program from the benchmarks on the inputs.

We compare our specification selection with two baseline selection strategies, by applying the strategies to the test inputs involved in the above-mentioned 381 selections. One baseline is majority vote, i.e., selecting the option that is the output of the majority of the samples. This is similar to AlphaCode, but AlphaCode takes all inputs into account to identify the majority, while here we only consider a single input at a time. The other baseline, denoted with direct inference, is where we remove the options from the prompts of LLMCodeChoice and use the LLM to predict the output directly. By comparing with direct inference, we demonstrate the benefit of hinting the LLM with program outputs.

We present the result in Table 3.2. As shown in the table, LLMCodeChoice has selected the correct output in 321 (84.3%) of the 387 attempts, representing the highest accuracy among all the strategies. This is 14.0% higher than direct inference, highlighting the benefit of providing options for the LLM. Majority vote has an even lower accuracy than direct inference. The fact that the majority is often incorrect shows the necessity of oracle inference beyond simple majority vote.

We further investigate the reason LLMCodeChoice outperforms direct inference. It turns out that the LLM struggles to handle complex inputs, especially in math problems, while including possible options in the prompt mitigates this problem. For example, direct inference did not correctly solve the problem in Figure 3.4, while LLMCodeChoice managed to do so.

3.5.3 RQ3: Overall Effectiveness

We have evaluated the two components of LLMCodeChoice in RQ1 and RQ2 separately. In this section, we focus on the overall program selection accuracy of LLMCodeChoice, measured by the pass@1 metric. This pass rate has been calculated both on HumanEval and MBPP, and on their enhanced versions HumanEval+ and MBPP+. We count out the

Please help me find the correct output for the below function.
Please help me select a correct output from a few choices for the below function
def largest_prime_factor(n: int):
"""Return the largest prime factor of n. Assume n > 1 and is not a prime."""
What should be the value of `largest_prime_factor(25191)`? `311` or `933`?

Please explain step by step and answer.

----------Response of Direct Specification Inference ----------
Let's apply this to `largest_prime_factor(25191)`:
25191 is not divisible by 2
25191 is divisible by 3 --> 8397 remains (25191/3 = 8397)
8397 is not divisible by any of the prime numbers 2, 3, 5, 7, 11...
So, the output should be: `8397`

---------- Response of Specification Selection ----------
The process to find out the largest prime factor of a number `n` is as follows:
1. ... 2. ... 3. ...
Consider `n=25191`, ... will find that the prime factors of 25191 are 3, and 311.
Among them, 311 is the largest prime factor.
So, the function `largest_prime_factor(25191)` should return `311`.

Figure 3.4: Comparison between direct specification inference (marked in green) and specification selection (marked in blue) on an example from HumanEval.

tasks for which the LLM-generated programs are all correct or all wrong, in which case program selection technique makes no difference. This leaves 89 tasks in HumanEval and 162 tasks in MBPP. Apart from pass@1, we also count the number of LLM tokens used and measure the time usage to show the cost-effectiveness of different program selection techniques. The baselines we compare against are CodeT and AlphaCode. We present the results in Table 3.3. Note that in the two pass@1 columns, the results are of the form X/(Y), where X represents the pass@1 on the enhanced benchmarks and Y represents the pass@1 on the original benchmarks.

Pass@1. As shown in Table 3.3, LLMCodeChoice achieves pass@1 of 87.7% and 78.6% on HumanEval+ and MBPP+ respectively, making it the most accurate of the techniques. This is followed by CodeT, and AlphaCode has the lowest accuracy. We further analyzed the reason AlphaCode and CodeT are not as accurate as LLMCodeChoice. For AlphaCode, this is due to the limited ability of the LLM to generate correct programs. Specifically, the majority of the LLM-generated programs is often incorrect. The quality of the initial program samples also has an impact on the "dual-agreement selection" [CZN+23] of CodeT. This stresses the importance of a better program selection strategy beyond majority vote. With regard to CodeT, an equally important factor is the quality of the LLM-generated test cases. To evaluate the correctness of these tests, we execute the test cases with the ground-truth

Table 3.3: Overall pass@1 (%) and average token spent on the HumanEval+, HumanEval, MBPP+, and MBPP benchmarks with GPT-4 for all tools, excluding all tasks that cannot make correct program selection. The numbers $X/(Y)$ in the pass@1 (%) columns represent the pass@1 result for enhanced / (original) benchmarks respectively.

	HumanEval+ (HumanEval)		MBPP+ (MBPP)	
	pass@1 (%)	avg tokens	pass@1 (%)	avg tokens
Random Selection	57.0 (63.1)	-	58.1 (69.9)	-
AlphaCode	75.6 (80.3)	6319	71.4 (81.6)	5507
CodeT	80.8 (86.8)	6319	75.0 (83.3)	5507
LLMCodeChoice-Vote	73.1 (84.7)	-	72.0 (81.2)	-
LLMCodeChoice-Infer	79.6 (82.5)	-	76.9 (81.9)	-
LLMCodeChoice	87.7 (89.2)	1533	78.6 (83.2)	1331

programs provided in the HumanEval+ and MBPP+ benchmarks. If a ground-truth program fails a test case, this indicates that the test case is wrong. The execution results show that on both benchmarks, over 80% of the tasks have at least one wrong test case. On average, each task has more than 23 wrong test cases. Remember from Table 3.1 that there are about 85 LLM-generated test cases for each task, which means nearly 25% of the LLM-generated test cases are wrong. This shows that the LLM is prone to generate incorrect test cases, which would impact the accuracy of the dual-agreement selection of CodeT. Different from CodeT, LLMCodeChoice obtains the expected output of the test inputs by specification selection.

Ablation Study. We identify the respective contribution of both components in LLMCodeChoice through an ablation study. Specifically, we evaluate the following two variants of LLMCodeChoice:

- LLMCodeChoice-Vote: selecting the largest program cluster.
- LLMCodeChoice-Infer: replacing specification selection with direct inference, i.e., removing the output options from the prompt.

We can see that partitioning the programs into clusters brings an accuracy gain of 16.1% on HumanEval+ and 13.9% on MBPP+. On top of the partitioning, comparing LLMCodeChoice-Infer and LLMCodeChoice-Vote indicates that specification inference increases the accuracy by 6.5% and 4.9%. Finally, further providing the LLM with possible outputs increases the accuracy by 8.1% and 1.7% on the benchmarks.

Trustworthiness. Overfitting is a common challenge in many research areas, such as machine learning and automated program repair. It happens when a neural network or a program performs well on a small number of test cases but fails to generalize to more tests. A similar case can happen to LLM-generated programs, when the developer only has time to perform a few tests before accepting an actually incorrect program. To evaluate the extent to which program selection techniques can mitigate this risk, we simulate the developer test cases with the original HumanEval and MBPP benchmarks. We say an LLM-generated program is overfitting if it passes the original benchmark but fails the enhanced benchmark. For example, according to Table 3.3, AlphaCode has a pass@1 of 80.3% on HumanEval+ and 75.6% on HumanEval, indicating that AlphaCode selected an overfitting program for 4.7% (80.3%-75.6%) of the tasks. In contrast, this is only 1.5% (89.2%-87.7%) for LLMCodeChoice, showing that LLMCodeChoice is less prone to the overfitting problem. LLMCodeChoice is also the least overfitting on MBPP. This means that the developer can have greater trust for a program selected by LLMCodeChoice when only limited resource is available for validating program correctness.

Token Consumption. To apply the program selection techniques in practice, it is also important to understand their computational cost. In particular, we count the number of LLM tokens each technique consumes in the selection process. For LLMCodeChoice, the tokens are used to perform specification selection, while CodeT consumes tokens to generate test cases. As shown in Table 3.3, LLMCodeChoice has consumed 4x fewer tokens per task on average than CodeT, while still achieving a higher selection accuracy. This is because LLMCodeChoice produces only a few distinguishing inputs, on which specification selection is performed. On the other hand, to perform dual-agreement selection [CZN+23], CodeT has to sample a large number of tests to get a good estimate of the majority.

Time usage. Across the tasks in both datasets, the median time usage of LLMCodeChoice is 221.5 seconds, and that of AlphaCode and CodeT is 142.1 seconds.

Detection of Wrong Programs. When all programs generated for a task are wrong, the pass@1 would always be zero, regardless of the selection technique. Therefore, such tasks have been counted out of the pass@1 evaluation. However, LLMCodeChoice may identify such a case when the LLM chooses none of the provided output options. Overall, there are 28 such tasks in HumanEval+ and 90 in MBPP+, and LLMCodeChoice could detect 30 of these cases.

Summary Overall, our experiment shows that LLMCodeChoice improves pass@1 over random selections by 6.7% to 16.6% and outperforms all baselines. LLMCodeChoice also consumes significantly fewer computation resources (number of tokens) compared to other baselines, indicating its potential for deployment in real-world development processes.

Table 3.4: Four Reasons for Incorrect Specification Selection in LLMCodeChoice.

	Wrong Choice	Incomplete Problem	Ambiguous Problem	Wrong Ground Truth
HumanEval+	6	5	2	2
MBPP+	15	17	2	11
Total	(35.0%) 21	(36.7%) 22	(6.7%) 4	(21.6%) 13

Table 3.5: Causes of the 21 wrong choices by LLMCodeChoice.

Type of mistakes	Occurrences
Incorrect calculation	11
Misunderstanding of problem	5
Hallucinated precondition	3
Incorrect reasoning	1
Use of incorrect fact	1

3.5.4 RQ4: Reasons for Failure

In this section, we investigate the causes of incorrect specification selection made by LLMCodeChoice. While LLMCodeChoice achieves high accuracy in specification selection, it failed to make the correct choice in 60 of the 381 selection attempts throughout our experiment. We divide the failure reasons of these attempts into four categories in Table 3.4. Two annotators (the thesis author and a co-author of the corresponding publication) made the categorization independently and discussed their differences with a PhD student not involved in this work from the department to reach an agreement.

Wrong Choice This means LLMCodeChoice selects an incorrect output from all possible output options for a distinguishing input, when the expected behavior is clearly defined by the problem description. This category accounts for 35.0% of the incorrect selections in our experiment. Since this category of failures is particularly important for understanding the application scope of LLMCodeChoice, we inspected the 21 mistakes in this category and further divided them into 5 types, as shown in Table 3.5.

In Table 3.5, *incorrect calculation* is the prevailing type of errors, representing 11 wrong choices. This error type means the LLM takes the right steps to reason about the expected

output, but the concrete result of a certain step is wrong. The 11 errors can be further divided into 8 arithmetic errors, 2 string manipulation errors, and 1 logical error. As an example of arithmetic error, task 782 of MBPP calculates the sum of the odd-length subarrays of an array. When given a distinguishing input [0,-1], the LLM made a mistake by stating that "the possible odd-length subarrays are [0], [-1], and [0, -1]". It is a known issue that LLMs can often make mistakes in calculation, even when specially trained on massive mathematical content [LAD+22]. This suggests that an LLM might be less effective as an oracle when the program requirements involve complex arithmetic calculations.

The second most common type of error is *misunderstanding of problem*, roughly accounting for 1/4 of the wrong choices. For example, task 454 of MBPP involves telling whether a string contains 'z'. The quotes around the letter z mean an exact match is required (i.e., case-sensitive). However, the LLM overlooked this detail and wrongly stated that the string 'Z' also contains 'z'. This type of error suggests that the accuracy of LLMCodeChoice can be negatively impacted when the requirement is not clear or explicit enough.

The third largest error type is *hallucinated precondition*, accounting for 3 wrong choices. In this error type, the LLM "rejects" an input because the input does not satisfy a certain precondition hallucinated by the LLM. For example, task 803 of MBPP tells whether an integer is a perfect square. When given the input -1, the LLM does not respond with the correct answer "False". Instead, the LLM hallucinated that the input must be non-negative and stated that an exception should be thrown for -1, since "it is mathematically impossible for negative numbers to be perfect squares." While this type of error can also reduce the accuracy of selection, we note that a user of LLMCodeChoice can easily identify such an error by reading the LLM's response, if the selected output is an exception.

Finally, there are another two less common error types. *Incorrect reasoning* is related to a complex math problem that the LLM failed to reason about, while *use of incorrect fact* occurred when the LLM used a mistaken formula to solve a problem.

Incomplete Problem Incomplete problem represents the situation where the task description does not explicitly specify correct execution behaviors triggered by distinguishing inputs or lack of definition for specific terms. These distinguishing inputs are often edge cases for the programming tasks, which accounts for 36.7% incorrect specification selection. For example, (1) how to handle empty string, (2) what exceptions to throw when the program crashes, etc. Incomplete problems are especially common in MBPP, because of their vague descriptions. For example, task 264 in MBPP asks to implement a function to calculate a dog's age in dog's years, without specifying the definition of dog's year. However, we find that the original online description for task 264 clearly explained the relation between dog's year and human's year [2].

[2]https://www.w3resource.com/python-exercises/python-conditional-exercise-31.php

```
1  # Task 40
2  def triples_sum_to_zero(l: list):
3    """ takes a list of integers as an input. it returns True if there are three distinct
         elements in the list that sum to zero, and False otherwise.
4    """
5  - if len(l) < 3 or len(set(l)) < 3:
6  - return False
7    for i in range(n-2):
8      for j in range(i+1, n-1):
9        for k in range(j+1, n):
10         if l[i]+l[j]+l[k]==0:
11           return True
12   return False
```

Figure 3.5: Example of ambiguous task description from HumanEval benchmark. The ambiguous point is the interpretation of "distinct elements". Lines 6-7 show the specific implementation of the misinterpreted concept.

Ambiguous Problem We categorize an incorrect specification selection as an ambiguous problem if the two annotators independently propose more than one interpretation of the natural language description and LLMCodeChoice selects the output that is different from the interpretation of ground truth implementation, it accounts for 6.7% of incorrect specification selection. Figure 3.5 shows one example from HumanEval benchmark. The "triples_sum_to_zero" task checks if there are three distinct elements in the list sum to zero. There are two interpretations of the task because *distinct elements* can either refer to elements of different values, or elements of different indices. Both of the two interpretations are correct, but the ambiguity confused LLMCodeChoice and led to incorrect output when given triples_sum_to_zero[0,0,0] as the distinguishing input.

Wrong Ground Truth We have also identified 13 cases where we believe the specification selection is correct and the ground truth program from the benchmarks is wrong, which accounts for 21.6% of the incorrect specification selections. For example, task 799 in MBPP asks to "Write a function def left_rotate(n, d) to that rotate left bits by d bits a given number. We assume that the number is 32 bit". For input (n=16, d=28), the correct output should be '1', because, in 32 bits representation, there are 27 leading zeros before the number 16 (10000), therefore the 28 times left shifts should move the 1 to the rightmost bit, which represents 1 in decimal. However, the ground truth solution produces '4294967297', which represents a 33-bit binary number. This shows the practical value of LLMCodeChoice in witnessing developers' mistakes. By contemplating the input and the output proposed by LLMCodeChoice, the developer has a greater chance to be alerted and avoid such errors. We share our explanations for each wrong ground truth in our artifact.

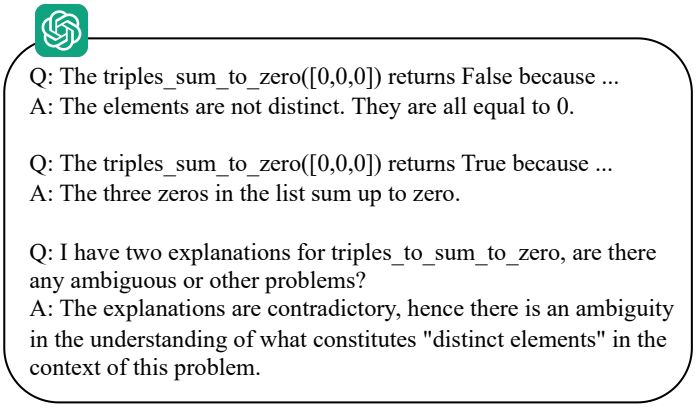


Figure 3.6: Example of contradictory explanation process for 'triples_sum_to_zero'. We first use the LLM to derive an explanation for each input-output pair. Then we take those explanations and query LLM again to identify the reasons for producing different explanations.

3.5.5 Discussion of Selected Specification

We have shown several limitations of the specification selection of LLMCodeChoice in RQ4. While most existing works focus on improving the pass@k result of LLM-generated programs, we propose the perspective that *providing explainable evidence to developers is more important than unexplained pass@k.* In this section, we discuss how we can utilize the byproduct of input-output specification from LLMCodeChoice to improve developers' confidence through a case study of tasks categorized as Ambiguity and Incomplete Specification in RQ4.

Contradictory Explanations Similar to oracle-guided program synthesis [JGST10], one advantage of LLMCodeChoice is the small-sized distinguishing input-output specification. We propose a preliminary idea called contradictory explanation generation to generate meaningful feedback for developers before they accept any suggestion from AI programming assistants. We obtain the contradictory explanation in two phases, explanation derivation and contradictory feedback. In the explanation derivation phase, for a distinguishing input i and the output o of each program cluster, we prompt the LLM with "the task returns o on input i because…", so that the LLM completes the sentence with an explanation of the output. Explanations of the different outputs may contradict each other. In the contradictory feedback phase, we put the task description and the explanations in one prompt, prompting the LLM to identify the reason that the task description allows contradictory explanations. Figure 3.6 shows task 40 of HumanEval as an example, where the LLM correctly explains the ambiguity of "distinct element" from the two contradicted explanations.

Evaluation. We implemented the contradictory explanation idea and conducted a preliminary evaluation on the tasks where LLMCodeChoice selected incorrect specifications.

We again asked the volunteer PhD student who helped categorize reasons for incorrect specification selections as an external developer to evaluate the contradictory feedback generated by LLM. If a contradictory feedback points out ambiguity or incompleteness in a problem description, and that the volunteer thinks the feedback correctly explained the problem, we mark the task as resolved. In total, we resolved 14 of the 26 tasks with incomplete specification or ambiguity, which shows the potential of applying the input-output specification as a trustworthy guard for future automatic programming. In future work, we will investigate how to create a more comprehensive technique to produce feedback for code from LLM and evaluate the technique on full datasets.

3.6 Threats to Validity

Internal Threats. A threat to internal validity is that our differential testing engine cannot guarantee to find all distinguishing inputs. Thus, the program clusters computed are not guaranteed to be fully accurate. Another threat is that our experiment compares LLMCodeChoice with AlphaCode and CodeT. However, the OpenAI Codex (code-davinci-002) model used by CodeT has been deprecated. To address this, we replaced the Codex model with GPT-4 model in the experiment for LLMCodeChoice and CodeT and we replicated CodeT's result using its artifact. While GPT-4 was one of the most powerful LLMs at the time of the experiments, we believe this replacement will also bring benefit to CodeT.

External Threats. The external threat may arise regarding the LLM we selected. Although there are many other open-source large language models [LCC+22, RGG+23], our experiment only evaluated LLMCodeChoice using GPT-4, because it is the most representative LLM we can access. To mitigate this threat, we release LLMCodeChoice's implementation to facilitate more detailed evaluation and future development in the community.

Another external threat is the type of programs LLMCodeChoice can be applied to. In our experiments, LLMCodeChoice was only applied to standalone (self-contained) programs. If the task is to generate a non-standalone program, e.g., a multi-component project or a library of utility functions, the application of LLMCodeChoice may require adjustments to reflect the particularities of the application domain. We leave investigating the selection of non-standalone programs to future work.

Finally, in terms of the modality of program output, LLMCodeChoice currently deals only with output that can be represented in text. LLMCodeChoice may not be applied directly when the output has a different modality, e.g., when the program plots an image. To mitigate this threat, a multimodal LLM can potentially be used, and LLMCodeChoice can be adapted to compare program outputs of other modalities.

Chapter 4

Code Intent Extraction via LLMs

4.1 Introduction

As discussed in Section 1.4, program improvement or program repair typically requires capturing developer *intent* to guide the process. The natural-language description of the developer intent is usually only available at a "higher level": it captures the intended behavior of the entire software system. However, to improve or repair specific components of a software system (where the error might have been localized), one needs to infer specifications of the different components. The works on semantic program repair [NQRC13, MYR16] extract such specifications via symbolic analysis of the given tests. Nevertheless, for the general problem of program improvement, the buggy program may or may not be accompanied by tests. Moreover, symbolic-analysis-based program repair has a high entry barrier for developers.

In this chapter, we examine efficient and low-cost workflows for iterative specification inference within an LLM agent. Given a GitHub issue to be resolved in a software project, our goal is to conduct iterative code search accompanied by specification inference, thereby inferring intent from both the project structure and behavior. The intent thus captured is examined by a reviewer agent with the goal of vetting the patches as well as providing a measure of confidence in the vetted patches. Our approach SpecRover is built on the open-source LLM agent AutoCodeRover [ZRFR24]. In an evaluation on the full SWE-bench consisting of 2294 GitHub issues, it shows more than 50% improvement in efficacy over AutoCodeRover.

In summary, the core contributions of this chapter are as follows.

- *Specification inference.* We examine the role of specification inference in LLM-guided autonomous software engineering. Our work suggests iterative specification inference to guide patching in LLM-oriented program repair workflows. Once the understanding of developer intent is accomplished via iterative specification inference, patch construction is a natural by-product of the inferred specification.
- *Suggesting patches with confidence.* We design a reviewer agent for code review which reconciles specifications, tests, and natural language requirements. The reviewer agent can be seen as conducting a comprehensive patch validation. The reviewer agent can

produce evidence of correctness of automatically generated fixes, such as an explanation of the patch, a reproducer test, and the accumulated specifications from different code elements. This evidence can be maintained along with the automatically generated patches, to track future regressions.

- *Experimental evidence.* Our tool shows high efficacy, solving 19.3% of issues in full SWE-bench and 31% on SWE-bench Lite. We also balance other needs from LLM agents such as low cost ($0.65 per issue) and supporting higher precision and recall. Our tool is available open-source.

4.2 Motivating Example

We now present the SpecRover approach via an example. The GitHub issue involved in this example is scikit-learn-15535[1], shown in Figure 4.1a. In the issue, two relevant code snippets are provided. According to the issue report, both snippets had worked without problem on an older version of scikit-learn, and it is expected that they continue to work on the current version. However, on the current version, the first snippet now crashes. The associated error information indicates that the crash occurred when scikit-learn mistakenly tries to convert a non-numeric array element into a float.

To resolve the issue, SpecRover first identifies buggy program locations by exploring the program and retrieving relevant code. In this example, the identified buggy method is `check_array`, shown in Figure 4.1b. The method performs two conversions of string to float. The two conversions are the root cause of the failure reported in the issue, resulting in an exception when the involved string is non-numeric. SpecRover inferred a summary of *intended behavior* of the method `check_array` (shown in Figure 4.1c), which serves as a specification of how the method should be modified.

Next, the identified buggy method and its intended behavior are passed to our patching agent, which will write patches for the method. At the first attempt, the patching agent wrote a partial patch, which only contains the first of the two hunks in Figure 4.1e, i.e., the patch only catches exceptions for the first conversion. Without further rectification of the patch, the issue could not be resolved. To vet the patch for potential mistakes, the initial patch is then passed to the reviewer agent. Apart from the patch, the reviewer agent also takes a reproducer test that reproduces the issue. The reviewer agent then runs the reproducer test on both the original program and the program repaired by the initial patch. By referring to the error information, the patch, and the issue statement, the reviewer agent is able to give the feedback as shown in Figure 4.1d. The feedback correctly indicates that the initial patch does not resolve the issue and can be rectified by catching exceptions for

[1] https://github.com/scikit-learn/scikit-learn/issues/15534

regression in input validation of clustering metrics

```
from sklearn.metrics.cluster import mutual_info_score
import numpy as np

x = np.random.choice(['a', 'b'], size=20).astype(object)
mutual_info_score(x, x)
```

ValueError: could not convert string to float: 'b'
while

```
x = np.random.choice(['a', 'b'], size=20)
mutual_info_score(x, x)
```

works with a warning?
this worked in 0.21.1 without a warning (as I think it should)

(a) Issue statement.

```
def check_array(array, ...):
    dtype_orig = getattr(array, "dtype", None)
    ...
    if dtype_orig is not None and dtype_orig.kind == "O":
        # if input is object, convert to float.
        dtype = np.float64
    ...
    if dtype_numeric and array.dtype.kind == "O":
        array = array.astype(np.float64)
    ...
```

(b) Buggy function.

Intended behavior: The check_array method should validate the input array without attempting to convert arrays with dtype object containing non-numeric strings to float. ... If the array contains non-numeric strings, it should skip the conversion to float and proceed with the validation. ...

(c) Inferred intended behavior of the buggy function.

The patch attempts to handle non-numeric strings by catching a ValueError during the conversion to float. However, it does not correctly handle the case where the input contains non-numeric strings. The patch still tries to convert the array to float later in the code, which results in the same ValueError. The patch does not resolve the issue.

(d) Reviewer feedback on initial patch.

```
--- a/sklearn/utils/validation.py
+++ b/sklearn/utils/validation.py
@@ -459,8 +459,13 @@ def check_array(array, ...
     ...
     if dtype_orig is not None and dtype_orig.kind == "O":
-        # if input is object, convert to float.
-        dtype = np.float64
+        # if input is object, check if it contains non-numeric strings
+        try:
+            array.astype(np.float64)
+            dtype = np.float64
+        except ValueError:
+            # contains non-numeric strings, skip conversion
+            dtype = None
     ...

@@ -551,7 +556,11 @@ def check_array(array, ...
     ...
-            array = array.astype(np.float64)
+            try:
+                array = array.astype(np.float64)
+            except ValueError:
+                # contains non-numeric strings, skip conversion
+                pass
     ...
```

(e) Final patch generated by SpecRover.

Figure 4.1: scikit-learn-15535 description and SpecRover artifacts.

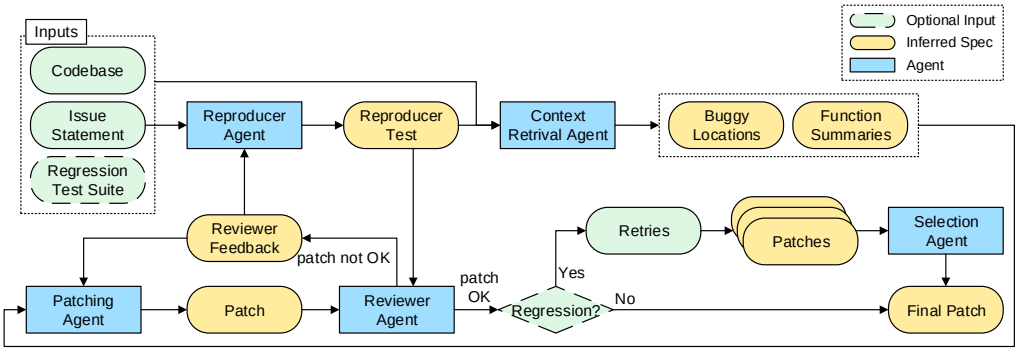


Figure 4.2: Overall Workflow of SpecRover.

the other string conversion. Finally, the feedback is passed to the patching agent, which writes the correct patch shown in Figure 4.1e.

In this example, we illustrated how our reviewer agent provides feedback on an incorrect patch for our patch-writing agent. The feedback leads to a later rectification of the patch, and explains clearly why the initial patch is incorrect.

4.3 Methodology

4.3.1 Overview

Problem setup Given a software codebase C and a natural language problem description D, the goal is to automatically derive a patch p (i.e. a set of code modifications) to C, such that the patched codebase C' satisfies the requirements in D. One example setup for D is GitHub issues, in which the issue description contains requirements for fixing a bug or adding a new feature.

In this chapter, we drive autonomous program improvement with the help of program specifications. We try to acquire an understanding of the intended program behavior (the specification), which then allows us to produce high-quality patches that successfully resolve GitHub issues. Beyond producing high-quality patches, an additional benefit of understanding the specification is that it also serves as evidence as to *why* the patch is correct. The evidence holds promise in terms of easing software maintenance and engendering trust in the code. The key novelty of our approach lies in how we infer and utilize various forms of specifications. For an overview of all the specifications involved, we depict the general workflow of our approach SpecRover in Figure 4.2. In this figure, the inferred specifications are highlighted in yellow. We also highlight in blue all the LLM agents present in the workflow. As shown in Figure 4.2, the specifications are inferred in an iterative fashion: the agents take in specifications, possibly produced by other agents, and in turn infer new forms

of specifications. This iterative process generates a variety of specifications, until a patch is generated and deemed correct by one of our agents that vets generated patches.

Specifically, as shown in Figure 4.2, the following specifications are inferred in sequence in SpecRover, which is given as input an issue statement and a software codebase.

1. The input issue statement is passed to a reproducer agent, which writes a *reproducer test* that reproduces the program fault reported in the issue.
2. The reproducer test, its execution results, along with the issue statement and the codebase, are passed to a context retrieval agent. The context retrieval agent explores the program codebase and identifies the relevant code to the issue. It eventually decides on a set of *buggy locations* that need patching.
3. The context retrieval agent also produces a *function summary* for every function encountered while exploring the program code. A function summary describes the intended behavior of a function in natural language, with respect to the current issue being solved.
4. The buggy locations, together with their corresponding function summaries, are passed to a patching agent, which tries to write a *patch* to resolve the issue.
5. The patch and the reproducer test are passed to a reviewer agent for scrutiny. The reviewer agent will produce a *reviewer feedback* if the patch is deemed incorrect; the patching agent will take in the reviewer feedback and try writing another patch. The reviewer feedback is a natural-language explanation of why the patch is incorrect and how it can be rectified. Likewise, a reviewer feedback for the reproducer test will be produced at the same time if the test is deemed incorrect.
6. If a patch is deemed correct by the reviewer agent, and there is an existing regression test suite available for the program, the patch will be checked via the regression test suite. If there is no regression, the patch will be accepted as the final patch. Otherwise, if some of the regression tests fail, we will retry the workflow up to a predefined number of times.
7. Finally, after multiple retries, there can be multiple patch candidates. A selection agent is invoked to select one final patch among the patch candidates, and give the reason why this patch is selected. The final patch, the reason for selection, and optionally the rest of the candidate patches will be sent to the user.

Among the specifications, the 3) function summary and 5) reviewer feedback are unique to SpecRover and unexplored by other LLM agents. These specifications have boosted the effectiveness of SpecRover in resolving software issues, because they fully exploit different kinds of software artifacts: the function summary exploits the program code behavior, and the reviewer feedback exploits both the code and the test.

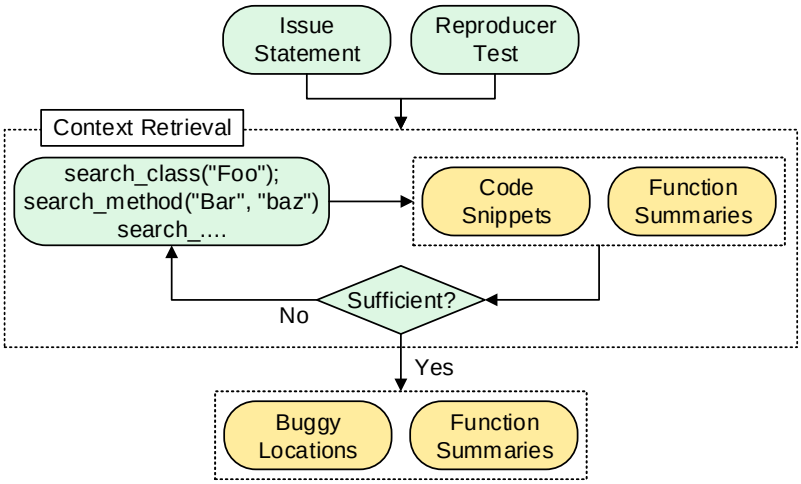


Figure 4.3: Context retrieval in SpecRover.

4.3.2 Function Summary: Specification from Program

In this section, we first describe how the context retrieval agent gathers code context for the software issue to be resolved. We then discuss how SpecRover transforms the user intent in the issue description into program specifications for shorter code elements such as functions.

Existing LLM programming agents typically employ a *context retrieval* step to collect necessary code context related to the given issue from a large codebase. SpecRover follows the general architectural design of programming agents in its context retrieval stage, as shown in Figure 4.3. SpecRover conducts context retrieval by providing a set of APIs to the LLM for exploring the codebase. The LLM agent invokes the retrieval APIs to investigate the relevant code snippets in the program. The retrieved code forms the code context for the current to-be-resolved issue, which can contain definitions of the relevant classes and methods. After each round of retrieval API invocations, the LLM agent takes the code context collected so far and decides whether the context is sufficient for understanding and resolving the problem. If the context is deemed sufficient, the retrieval process will end, and the agent will decide on a set of *buggy locations*, which are sent to the patching agent for repairing. Otherwise, the retrieval process continues until a predefined threshold count is reached.

One key novelty in SpecRover is the *explicit* extraction of *function summaries* while collecting code snippets during context retrieval. In SpecRover, whenever a new code snippet is retrieved with an API and sent to the context retrieval agent, we explicitly prompt the agent to analyze the "intended behavior" of this code snippet in the current problem context. The intended behavior (or specification) is a concise natural-language summary of how a function

should behave to meet the requirements specified in the high-level problem description. This function-level summary of intended behavior serves as a local specification to guide the patch construction. The system-level intended behavior specification given by the user (i.e. the issue description) is often on how the program should behave rather than how a unit function should behave. So we usually do not have the intended behavior of a function. Although the issue description may provide some "direction" on the intended behavior of a function - it is usually not sufficient to guide the patching agent. On the other hand, the extracted function-level specification (capturing the intended behavior of the function) serves as a more direct guide to the patching agent. Instead of giving a set of bug locations $\{L_1, L_2, ..., L_n\}$ to the patching agent to modify, SpecRover gives the pairs of bug locations and their corresponding local specification $\{(L_1, Spec_1), (L_2, Spec_2), ..., (L_n, Spec_n)\}$. The patching agent can then refer to the specifications of intended behavior and modify code at the function level (so as to achieve this intended behavior). Intuitively, our approach decomposes the repository-level issue solving task to several function-level code modification tasks, in which each function-level task has a natural language specification. LLMs have been extensively studied for function-level coding tasks and have shown promising results in function-level benchmarks such as HumanEval [CTJ+21, LXWZ23] and MBPP [AON+21]. Therefore, this task decomposition helps the patching agent of SpecRover which then has to solve smaller and more manageable tasks.

4.3.3 Reviewer Feedback: Reconciling Specifications

Another kind of specification inferred by SpecRover is the reviewer feedback. To be more precise, the reviewer feedback can be called a meta-specification: it is a reflection on the specifications inferred in previous steps. Concretely, given a patch and a reproducer test, the reviewer agent in SpecRover will provide feedback, which includes 1) a binary decision of whether the patch and the reproducer test are correct respectively; and 2) an explanation for the decisions.

The reviewer feedback contributes to our specification inference practice in two ways. First, it makes the specification inference iterative. The reviewer feedback will be passed back to the patching agent and the reproducer agent, leading to improved patches and reproducer tests. Second, it reconciles the patch and the test. In this way, errors that are not obvious when examining the two separately can be revealed and rectified. What makes the reviewer feedback important is the absence of a suitable test suite. If a test suite was available for checking whether the issue has been resolved, patch correctness could be easily decided. In reality, however, issues occur when the program already passes the accompanying regression tests, which means that a high quality test-suite to check a generated patch (for the given issue) is usually not available.

To mitigate the lack of an issue-revealing test case, SpecRover writes a reproducer test via the reproducer agent. However, this test alone is not sufficient for deciding patch correctness.

This is because the reproducer test can be *incorrect*, due to the non-determinism of the LLM, i.e., the test may fail a patch that actually conforms to the user intent. Besides, the reproducer test can also be an *incomplete* description of intent, i.e., a patch may pass the test without completely resolving the issue. The limitation of the reproducer test derives from the fact that tests are a precise yet incomplete specification. To overcome the limitation, we make the observation that the natural-language issue statement is ambiguous in nature yet often contains richer information. Therefore, supplementing the test with an understanding of the issue statement is likely to help decide patch correctness. This is accomplished in the reviewer agent of SpecRover, which considers the issue statement as well as the test to vet patch candidates.

Beyond deciding on patch correctness, the more important aspect of the reviewer feedback is an explanation of the decision made, which will help the patch agent rectify an incorrect patch. Further, for a correct patch, the explanation will help the user understand and accept the patch. The user can merge the reviewer feedback into the software together with the patch for future reference, which will help software maintenance in the long run.

```
An issue has been submitted. Engineer A has written a reproduction test for the issue.
Engineer B has written a patch for the issue. Your task is to decide whether the created
patch resolves the issue.
NOTE: both the test and the patch may be wrong.
Here is the issue: ...
Here is the test written by Engineer A: ...
Here is the result of executing the test on the original buggy program: ...
Here is the patch written by Engineer B: ...
Here is the result of executing the test on the patched program: ...
Think about
(1) whether the test correctly reproduces the issue, and
(2) whether the patch resolves the issue.
```

Figure 4.4: Template prompt for the reviewer agent.

Our reviewer agent generates the feedback in two steps. First, the original program and the patched program are run on the reproducer test. In both runs, execution information including the output and the exit code are collected. The reviewer agent then provides the LLM with this execution information, along with the issue statement and the reproducer test. The LLM is prompted to decide whether the patch and the test are correct respectively, and to provide explanations for both decisions. Figure 4.4 shows the template of the prompt.

4.3.4 Patch Selection

As shown in Figure 4.2, a patch approved by the reviewer agent is checked through a regression test suite, which serves as an oracle for whether the patch breaks existing functionality of the program. However, in the setting of resolving GitHub issues, the regression test suite can be an inaccurate oracle, meaning that it can reject correct patches which resolve the issue. This is because the correct patch may inevitably modify existing functionalities of the program while resolving the issue, thus causing some of the existing regression tests to fail.

For example, if the patch needs to modify the signature of an existing function `f` in order to resolve an issue, regression tests that invoke `f` will now fail. Since the correct patch can be rejected by the regression tests, we employ a patch *selection* process at the end of the workflow to select the most promising patches among the rejected candidate patches.

During the final patch selection phase, SpecRover goes beyond the test cases and employs a *selection agent* to choose a patch based on the natural language issue description. All candidate patches that failed some tests are presented to the selection agent, together with the issue description. The selection agent is instructed to analyze the root cause of the issue, think about how the issue can be possibly resolved, and select a patch that best addresses the issue. This natural language-guided patch selection can recover correct patches that are mistakenly filtered out by an inaccurate test suite. It exploits the natural language issue report as that captures the most up-to-date intents from users/developers. While making a choice among the candidate patches, the selection agent also explicitly states a reason why it chooses a particular patch among the candidate patches. This "reason for selection" can be given as *evidence* together with the final patch.

4.3.5 Evidence

SpecRover is designed to not only generate a patch to resolve the issues in software repositories, but also to provide the inferred specifications as evidence for why a patch was selected. Specifically, along with the final patch, the following artifacts can be the outputs of SpecRover as well:

- Buggy locations and their intended behaviors.
- The reproducer test written by the reproducer agent.
- The reason why the final patch was approved by the reviewer agent (if the patch is approved by the reviewer and the regression test suite).
- The reason why the final patch is selected by the selection agent (if there are multiple candidate patches which cannot be differentiated by the tests).

The benefits of generating evidence are threefold. First, these artifacts can guide the LLM agents in constructing higher quality patches, as discussed in Section 4.3.2 and 4.3.3. Second, the natural language artifacts can assist the developers in understanding the auto-generated patches more quickly. Before examining the actual patch, developers can gain a preliminary understanding of the changes by reviewing the reasons for approval or selection and the intended behaviors for the buggy locations. Last but not least, the evidence can be integrated into the software repository and can evolve with it. The developers can integrate the reproducer test for this issue as part of the test-suite of the program. Reasons for patch approval/selection can become parts of the commit message when the auto-generated patch is committed to the repository. Overall, we propose SpecRover as a programming agent

that not only automatically generates code improvements but also produces evidence that enriches the software system lifecycle.

4.4 Experimental Setup

We address the following research questions:

RQ1: What is the efficacy of SpecRover in resolving issues?

RQ2: What level of confidence can developers get from the patch and specifications produced by SpecRover?

RQ3: What is the quality of the specification produced by SpecRover as evidence?

Benchmark We evaluate the efficacy of SpecRover on SWE-bench [JYW+24], a widely-used benchmark for autonomous program improvement consisting of 2294 real-world GitHub issues. For each issue, the only input for SpecRover is the issue statement and the buggy codebase. Note that the regression test suite used by SpecRover is part of the *buggy* program; we do not access any code or test that is added by the developer in the fixed version of program.

Baselines and Evaluation Metrics For RQ1, we compare with the state-of-the-art systems that target the repository-level issue solving task. In our comparison, we include all the open-source software engineering agents which have reported results on SWE-bench. The baseline tools include:

- AutoCodeRover [ZRFR24]. AutoCodeRover employs a set of program structure-aware APIs to gather relevant code context. It optionally integrates debugging techniques such as Spectrum-based Fault Localization to sharpen the context.
- SWE-agent [YJW+24]. SWE-agent designs an agent-computer interface, which defines the possible actions taken by an agent to edit code, navigate the codebase, and execute tests.
- AppMap Navie [App24]. Navie uses a retrieval-augmented generation (RAG) based approach to construct the code context, and performs an explicit planning step before generating code changes [Gil24].
- OpenHands (formerly OpenDevin) [WLS+25]. OpenHands's CodeActAgent tackles the tasks by having a general action space, where the agent is allowed to execute arbitrary Python and bash commands inside a sandbox environment.
- Aider [Gau24]. Aider constructs a repository map which helps the LLM to understand the repository context. It also uses the regression test suite as a harness to retry the task.

- Moatless Tools [Örw24]. Moatless Tools builds an agentic loop that functions as a finite state machine and transitions between states. It focuses on building a set of good tools for the agent instead of relying on the agent for reasoning.
- Agentless [XDDZ25]. Agentless employs a fixed two-phase approach of localization and repair, without allowing the LLM to decide on actions or utilize tools.

We report pass@1 efficacy on SWE-bench for all tools. For each issue, SWE-bench has a set of acceptance tests written by the developers to evaluate the patch correctness. These acceptance tests are not used by the tools when generating patches. We follow the official SWE-bench evaluation criteria - if the single patch generated by a tool passes the SWE-bench acceptance tests for the issue, the issue is considered as resolved.

Implementation and Parameters We implemented SpecRover on top of the AutoCodeRover codebase and reuse its context retrieval APIs. We implemented new features unique to SpecRover such as function summary extraction as part of the context retrieval process. Other unique features such as patch reviewing and selection are implemented as new LLM agents. SpecRover supports multiple LLMs as backend. In our experiments, we used the Claude 3.5 Sonnet as the main foundation model, and only switch to OpenAI GPT-4o for a task if that task encounters an API error when invoking the Claude remote APIs. We set maximum retries on regression test suite failures to be 3.

Manual Inspection of Results For a better understanding of the experimental results, we perform manual inspection to obtain certain data, e.g., patch overfitting rate. All manual inspections were independently conducted by two annotators (the thesis author and a co-author) and subsequently cross-checked. Discrepancies were resolved through consultation with a third colleague from our group.

4.5 Evaluation

4.5.1 RQ1: Overall Efficacy of Task Resolving

We first evaluate the overall efficacy of SpecRover in resolving repository-level tasks. We report the efficacy of SpecRover on both SWE-bench (consisting of 2294 real-world GitHub issues), and SWE-bench Lite (which is a subset of SWE-bench consisting of 300 issues). For the baseline tools, we compare with their corresponding reported efficacy. If a tool supports different configurations (e.g. different LLMs as the backend), we compare with the configuration with the highest efficacy.

Result Table 4.1 shows the efficacy of issue resolving in both SWE-bench and SWE-bench Lite. Overall, at the time of evaluation, SpecRover achieved the highest efficacy among all the open-source tools in both SWE-bench and SWE-bench Lite. In SWE-bench Lite,

Table 4.1: Comparison of efficacy/efficiency/cost on SWE-bench and SWE-bench Lite.

Tool	LLM	Resolved%	Avg. Time (s)	Avg. Cost ($)
Efficacy on SWE-bench (size=2294)				
AutoCodeRover	GPT-4	12.42% (285)	248	0.45
SWE-agent	GPT-4	12.47% (286)	-	1.59
AppMap Navie	GPT4o	14.60% (335)	-	-
SpecRover	Sonnet-3.5+GPT-4o	**19.31%** (443)	362	0.72
Efficacy on SWE-bench Lite (size=300)				
SWE-agent	GPT-4	18.00% (54)	-	1.67
AutoCodeRover	GPT-4	19.00% (57)	195	0.43
AppMap Navie	GPT-4o	21.67% (65)	-	-
OpenHands	Sonnet-3.5	26.00% (78)	-	1.10
Aider	GPT-4o+Opus-3	26.33% (79)	-	-
Moatless Tools	Sonnet-3.5	26.67% (80)	71	0.17
Agentless	GPT-4o	27.33% (82)	-	0.34
SpecRover	Sonnet-3.5+GPT-4o	**31.00%** (93)	309	0.65

'-' indicates data is not publicly available.

compared to the previously top-performing group of tools which resolved approximately 26% to 27% of the issues, SpecRover improved the efficacy to 31%. This efficacy improvement is also evident in the full SWE-bench, where SpecRover improved the efficacy from 14.60% to 19.31%. Figure 4.5 illustrates the number of uniquely resolved issues by SpecRover and other tools in SWE-bench Lite. For clarity, this figure includes only the top five performing tools from Table 4.1. SpecRover uniquely resolved 12 issues, the highest number of uniquely resolved issues among all the tools. Among the 12 uniquely resolved issues, SpecRover resolved six of them by generating only one patch, demonstrating that the inferred function summary can effectively guide the LLM to generate correct patches. For the other six issues, SpecRover deemed the first generated patch as incorrect from the reviewer agent and the regression test suite. In this case, the patches are iteratively refined based on the reviewer feedback and the test results, and eventually the correct patch is selected at the end of the workflow.

Time and Cost We also report the average time taken and average costs for each issue in Table 4.1. For each tool, we include the time and cost statistics in Table 4.1 if this information was publicly reported or can be calculated from their publicly available execution traces. On average, SpecRover costs $0.65 USD to generate patches for each issue in SWE-bench Lite, achieving the highest efficacy among the compared tools with a relatively low cost. We further investigate the 93/300 issues resolved by SpecRover in SWE-bench Lite. For the resolved issues, SpecRover only costs $0.36 USD per issue to generate the correct patch,

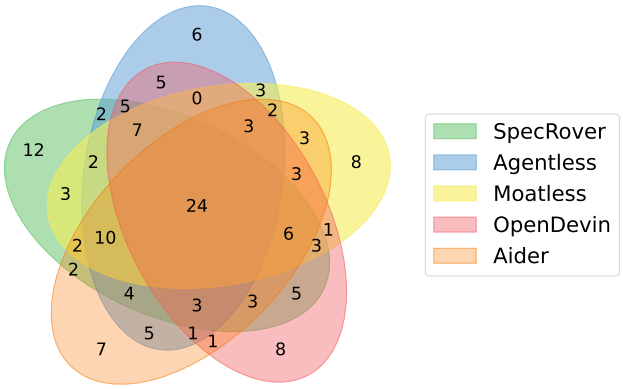


Figure 4.5: Number of uniquely resolved issues by the top performing open-source tools on SWE-bench Lite.

suggesting that the resolved issues require fewer retries and fewer API calls to the LLM in general. Time-wise, SpecRover spends an average of 309 seconds (i.e. 5.15 minutes) on each issue, which includes the time for executing the reproducer and the regression tests in the project. According to a prior study, most developers accept automated program repair tools which take less than 30 minutes [NSGR22]. SpecRover requires approximately 5 minutes, which we deem acceptable.

Patch Correctness A patch that passes a given test suite is not necessarily correct, because the test suite is an incomplete specification. This problem is known as *patch overfitting* [SBLGB15] in program repair. For a more accurate evaluation of the generated patches, we manually compared the SpecRover-generated patch with the developer patch for each resolved issue. Our manual investigation confirmed that 54.8% (51/93) of the patches that pass the test suites are semantically equivalent to the ground-truth patches. We also observed that 34 out of the 42 (=93-51) overfitting patches (i.e., the patches that pass the test suite but are not semantically equivalent to ground truth) modify the same methods as the ground-truth patch, and only the specific modification is semantically different from the ground truth. This implies that even overfitting patches produced by SpecRover are useful in indicating a fix location. Adding up the semantically equivalent patches and the overfitting patches that have the correct fix location, a total of over 90% of the test-passing patches prove useful for issue resolution.

Reasons for Failure We examined the 207 tasks that are not resolved by SpecRover in SWE-bench Lite. These failure cases break down into three cases:

1. Ambiguous issue description (as labeled by SWE-bench Verified [CAS+24]): 107 (51.7%)
2. Incorrect fix location: 61 (29.5%)

3. Incorrect code modification: 39 (18.8%)

Case 1 is "hard or impossible to solve" [CAS+24] without further information. Case 2 is more frequent than Case 3, implying that SpecRover can potentially have significant improvement by leveraging more fix localization techniques. For Case 3, in most such cases (26/39), we observed that the gist of the patch generated by SpecRover is actually correct, but some code detail is wrong. To reduce this kind of failure, more test generation techniques can be leveraged within our agent.

4.5.2 RQ2: Utility of Autonomous SE, Confidence in Results

Although the efficacy in resolving issues is an important aspect of autonomous program improvement, it is not the sole purpose of such a technique. Rather, the efficacy is a means to an end – to reduce human effort in software maintenance. To this end, a program improvement technique must not only have high efficacy, but also minimize the effort required of an end user to use the technique. The effort is related to two metrics: 1) *signal-to-noise ratio*, i.e., the ratio of correct to incorrect patches presented to a user; and 2) the difficulty of examining each auto-generated patch that is suggested.

We have designed SpecRover to reduce both of these efforts. First, to reduce the number of incorrect patches that a user may examine, we use the reviewer agent to decide the correctness of the generated patch and the reproducer test. The user can choose to examine the generated patch only when both the patch and the reproducer test are deemed correct by the reviewer agent. The accuracy of the reviewer decisions is measured in RQ2. Second, to make it easy for a user to examine each patch, SpecRover provides a variety of evidence to help understand the patch, as discussed in Section 4.3.5. The quality of the evidence will be discussed in RQ3.

There can be four different scenarios when the reviewer decision is viewed in relation to the actual correctness of the patch. For convenience, we say a patch is *accepted* when the reviewer agent decides that both the generated patch and the reproducer test are correct. With this, we discuss the following four scenarios:

- TP (true positive): Patch is accepted and correct;
- TN (true negative): Patch is not accepted and incorrect;
- FP (false positive): Patch is accepted but incorrect;
- FN (false negative): Patch is not accepted but correct.

Table 4.2 lists the frequency that each scenario occurred in our experiment. The table counts in 119 tasks in SWE-bench Lite for which a reproducer test was generated. In the table, we also calculate the accuracy, precision, and recall of the reviewer decisions. Out of the 119 tasks, there are 26 TP's and 51 TN's, i.e., as many as 64.7% (accuracy) of the reviewer

Table 4.2: Reviewer decisions on SWE-bench Lite.

Category	# Tasks
TP	26
TN	51
FP	26
FN	16
Total	119
Accuracy = (TP+TN) / Total	64.7%
Precision = TP / (TP+FP)	50.0%
Recall = TP / (TP+FN)	61.9%

decisions were consistent with the actual correctness of the patch. The recall was also over 60%, meaning that the majority of the generated correct patches were recognized by the reviewer agent.

A metric of particular interest to program improvement techniques is the precision. The precision is defined as TP/(TP+FP), i.e., the proportion of correct patches in all the patches offered by a technique. It is directly related to user effort required to examine generated patches. For SpecRover, the precision is 50.0%, as calculated in Table 4.2. To put the precision in perspective, we compare the precision of SpecRover with that of other baseline tools in Figure 4.6. The precision of the baseline tools is the same as their pass@1 efficacy reported in Table 4.1, since these tools indiscriminately present every generated patch to a user. As can be seen in Figure 4.6, the precision of SpecRover is higher than 1.8x that of Agentless, which has the second highest precision. The high precision of SpecRover indicates a much lower cognitive load imposed on the user, compared to other techniques. Moving forward, we suggest paying attention to agent precision.

4.5.3 RQ3: Quality of Specifications Produced

In this section, we investigate the quality of evidence generated by SpecRover. The high-quality evidence allows a developer to easily integrate an auto-generated patch into an existing codebase.

Function Summaries

We manually investigated the natural-language specifications generated by SpecRover for SWE-bench Lite issues. To obtain some approximation of "ground truth" for the generated specifications, we extracted the titles of Pull Requests that developers wrote when they fixed each issue. The PR title usually summarizes the fixes made in the PR in one sentence.

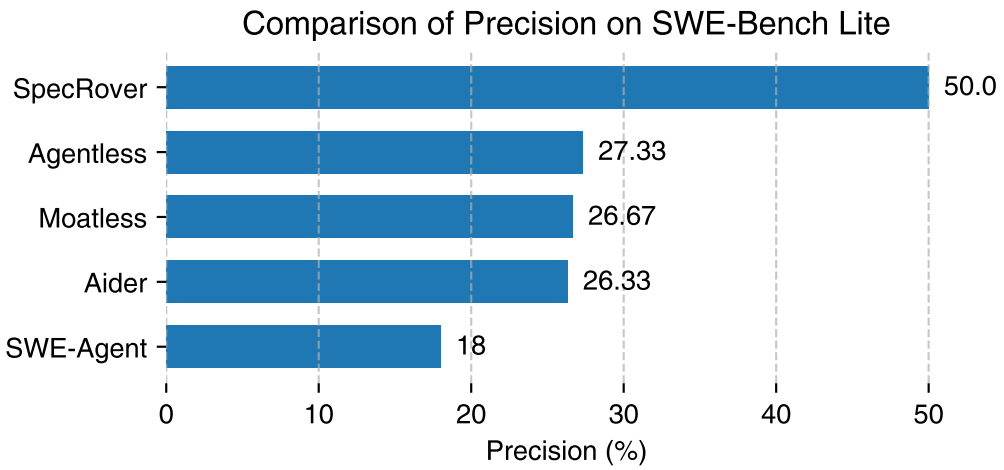


Figure 4.6: Comparison of patch precision on SWE-bench Lite. If SpecRover does not use regression tests for patch validation, precision reduces only slightly to 48.3%.

In this investigation, we include only the issues for which SpecRover generated a patch at the same methods as the developer's patch. Moreover, we exclude the issues whose developer-written PR title is too brief (e.g. "Fixes issue #..."). For the included issues, we manually compare the specification generated by SpecRover for the patched methods, with the PR title written by the developer. For 72/101 (71%) inspected issues, the specification generated by SpecRover covers similar intent to the human-written PR title, which shows that the generated specification serves as a good summary of the intended behavior. The generated specifications are generally an extended version of the PR title and additionally describe how the code should be modified.

Reviewer Feedback

We cannot conduct a quantitative examination on the quality of the reviewer feedbacks on the generated patches, since there is no "ground truth". Therefore, we illustrate the quality of the feedbacks with two examples.

Reviewer Feedback as Summary In the first example, we show that the reviewer feedback can serve as a concise summary of a generated patch. The summary describes the behavior of the patch at a high level. Therefore, a developer can understand the generated patch faster by reading the summary before examining the details of the patch. Besides, after the developer accepts the patch and decides to merge it into existing code, the summary constitutes a good commit message, so that the developer does not need to write one. From a developer's perspective, the whole process is very much like reviewing a pull request, which is already part of a developer's day-to-day workflow. The specific issue involved in this

skipping: --runxfail breaks pytest.mark.skip location reporting
pytest versions: 5.4.x, current master
When `@pytest.mark.skip` / `skipif` marks are used to skip a test, for example

```
import pytest
@pytest.mark.skip
def test_skip_location() -> None:
    assert 0
```

the expected skip location reported should point to the item itself, and this is indeed what happens when running with `pytest -rs`:

```
SKIPPED [1] test_it.py:3: unconditional skip
```

However, adding `pytest -rs --runxfail` breaks this:

```
SKIPPED [1] src/_pytest/skipping.py:238: unconditional skip
```

The `--runxfail` is only about xfail and should not affect this at all.

Hint: the bug is in `src/_pytest/skipping.py`,
the `pytest_runtest_makereport` hook.

(a) Issue statement.

```
def pytest_runtest_makereport(item: Item, ...):
    ...
    elif item.config.option.runxfail:
        pass  # don't interfere
    elif call.excinfo and isinstance(call.excinfo.value, ...):
        ...  # (3 lines)
    elif not rep.skipped and xfailed:
        ...  # (14 lines)
    elif ...:
        # calculate the line number
```

(b) Buggy code snippet.

```
--- a/src/_pytest/skipping.py
+++ b/src/_pytest/skipping.py
@@ -270,33 +270,34 @@ def pytest_runtest_makereport(item: Item, ...):
def pytest_runtest_makereport(item: Item, ...):
    ...
-    elif item.config.option.runxfail:
-        ... # (22 lines)
+    elif not item.config.option.runxfail:
+        if call.excinfo and isinstance(call.excinfo.value, ...):
+        ... # (20 lines)
+    if ...: # was elif
         # calculate the line number
+        # ... # (4 lines)
```

(c) Correct patch generated by SpecRover.

The patch correctly addresses the issue by modifying the logic in the pytest_runtest_makereport function. It ensures that the skip location is reported correctly regardless of whether the --runxfail option is used. The patch moves the handling of skip location correction outside the conditional block for --runxfail, which fixes the reported issue.

(d) Reviewer feedback.

Figure 4.7: pytest-7432 description and SpecRover artifacts.

Issue: Wrong Derivative kind attribute
I'm playing around with the `kind` attribute. The following is correct:

```
A = MatrixSymbol ('A', 2, 2)
i = Integral (A, x)
i.kind     # MatrixKind(NumberKind)
```

This one is wrong:

```
d = Derivative (A, x)
d.kind     # UndefinedKind
```

(a) Issue statement.

Buggy Locations
Class Derivative in
file sympy/core/function.py

Intended behavior:
Add a `kind` property to the `Derivative` class that returns the appropriate kind for derivatives. This may involve checking the kind of the expression being differentiated and the differentiation variables.

(b) Location and summary.

```
try:
    A = MatrixSymbol('A', 2, 3)
    i = Integral(A, x)
    assert i.kind == 'MatrixKind(NumberKind)',
    f"Expected 'MatrixKind(NumberKind)', got {i.kind}"
    d = Derivative(A, x)
    assert d.kind == 'MatrixKind(NumberKind)',
    f"Expected 'MatrixKind(NumberKind)', got {d.kind}"
except AssertionError as e:
    # issue reproduced
    print_stacktrace(e)
    raise
```

(c) Generated reproducer test.

```
--- a/sympy/core/function.py
+++ b/sympy/core/function.py
 ...
+    @property
+    def kind(self):
+        return self.expr.kind
+
     def __new__(cls, expr, *variables, **kwargs):
 ...
```

(d) Correct patch generated by SpecRover.

Patch is correct.
Reason: The patch correctly adds a `kind` property to the `Derivative` class that returns the `kind` of the expression being differentiated.

Test is wrong.
Reason: The test incorrectly compares the `kind` attribute to a string `MatrixKind(NumberKind)`. The `kind` attribute is an object, not a string, and should be compared directly to the expected `kind` object.

(e) Reviewer feedback.

Figure 4.8: sympy-21614 description and SpecRover artifacts.

example is pytest-7432[2]. The issue statement is shown in Figure 4.7a, which reports that pytest (a python testing framework) would miscalculate a line number in its output when an irrelevant option (`runxfail`) is enabled. The bug is caused by the code shown in Figure 4.7b. As can be seen, the calculation of the line number is wrongly placed in a branch that is mutually exclusive with the `runxfail` branch. Therefore, the calculation is wrongly skipped when the option is enabled.

To resolve the issue, SpecRover was able to locate the relevant code and produce the correct patch. An abridged version of the patch is shown in Figure 4.7c. It correctly addresses the issue by moving the line number calculation to a branch unaffected by the `runxfail` option. However, the patch might not be immediately understandable to a developer, because it changes as many as 51 lines in the original program (though most of the changes just involve the indentation level). Fortunately, the understanding of the patch can be eased by the reviewer feedback. The reviewer agent was able to identify the patch as correct and produced the feedback in Figure 4.7d. It properly summarized that the patch just moved the calculation to another branch. Using this summary, the developer would easily understand the patch and accept it.

Tolerance of Incorrect Tests Another advantage of the reviewer is enhanced tolerance of incorrect automatically generated tests. We illustrate this advantage with the example of sympy-21614[3], where SpecRover rejects an incorrectly written test while approving a correct patch. The issue statement and the buggy location identified by SpecRover are shown in Figure 4.8a and 4.8b. The issue mentioned an unexpected behavior of the `kind` attribute. After its context retrieval stage, SpecRover correctly identifies that the buggy location is in the `Derivative` class, and that its intended behavior is to have an additional `kind` property. Figure 4.8c and 4.8d show the automatically generated reproducer test and patch for this issue. In this case, the reproducer test is incorrect - the assertions compare an object with a string, which always evaluate to `False`. If this reproducer test is used solely to determine the correctness of the generated patches, any patch, even a correct one, will be rejected. However, since the reviewer agent in SpecRover simultaneously examines both the reproducer test and the patch without assuming either is correct, it is capable of rejecting the reproducer test while approving the patch. Figure 4.8e shows the reviewer agent's decision and comments towards the test and patch. The reviewer identifies that the assertions in the reproducer are written incorrectly, thereby rejecting the reproducer. On the other hand, the reviewer correctly approves the patch despite the presence of an unreliable test. The correct patch, along with the reasons for rejecting the reproducer, can be sent to the developer. The developer can then integrate the patch into the codebase. Additionally,

[2] https://github.com/pytest-dev/pytest/issues/7392

[3] https://github.com/sympy/sympy/issues/21604

the developer can revise the "almost correct" reproducer test based on the feedback provided by the reviewer agent.

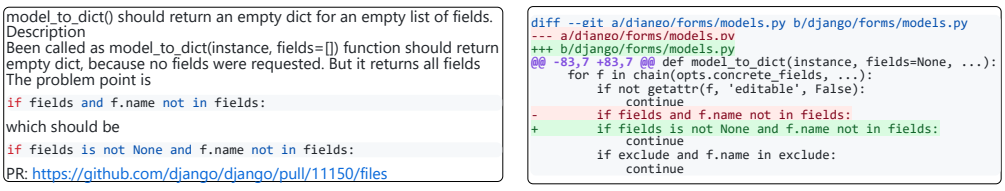
4.6 Threats to Validity

Randomness of LLMs LLMs are inherently random in their output generation, which may threaten the validity of LLM-based coding agents including SpecRover. We address this by setting the model temperature to 0, so that the model output is more deterministic.

Data Memorization An issue that can bias the evaluation of LLM agent-generated patches is *data memorization* [MS22]. Data memorization occurs when an LLM deals with a program that exists in its training set. This issue is rather common, since LLMs have been trained on a vast amount of publicly available code. In the presence of possible data memorization, when a correct patch is generated by an LLM agent, it may not be because of the capability of the LLM agent, but because the LLM has seen the repaired version of the program. In this way, the capability of the LLM agent is overestimated.

One way of gauging the threat of data memorization is to check whether the generated patch is syntactically equivalent to the ground-truth correct patch. Intuitively, the LLM is likely to generate a syntactically equivalent patch if it has seen the correct program. Therefore, the threat of data memorization is lower if the generated patch turns out not syntactically equivalent.

To gauge the data memorization threat and have a more accurate evaluation of the LLM agents, we perform this syntactic check of patches generated by agent systems on SWE-bench Lite. For each issue resolved by a tool, we compare the patch generated by the tool against the ground-truth developer-written patch, and check whether they are *syntactically equivalent*. The results are shown in Table 4.3. The column "# Resolved" shows the number of resolved issues by a tool; "# Syntactically Equiv." shows the number of resolved issues with the generated patch being syntactically equivalent to the ground-truth patch; "% Equiv. / Resolved" shows the ratio between these two numbers.



(a) Issue statement. **(b)** Ground-truth patch.

Figure 4.9: Issue django-11163. The issue statement contains the ground-truth patch.

On SWE-bench Lite, SpecRover generated a correct patch for 93 issues. Out of these, only 9 patches (10%) are syntactically identical with the ground-truth patch. Most other

Table 4.3: Number of resolved instances, and number of resolved instances with generated patches being syntactically equivalent to the ground-truth patch. Results are shown for each tool in SWE-bench Lite.

Tool	# Resolved	# Syntactically Equiv.	% Equiv. / Resolved
SpecRover	93	9	10%
Agentless	82	14	17%
Moatless Tools	80	12	15%
Aider	79	6	8%
OpenHands	78	0	0%
AppMap Navie	65	8	12%
AutoCodeRover	57	3	5%
SWE-agent	54	8	15%

tools show similar percentages of syntactically identical patches, ranging from 5% to 17%. OpenHands has 0% identical patches based on our computation, because OpenHands often creates backup files in the codebase, which are also included in its patch.

Overall, the threat of data memorization is minor in our evaluation of SpecRover. We note that the 10% identical patches represent an *upper bound* of the effect of potential data memorization. In some issues, the issue reporter has suggested the correct patch. An example of such issues is shown in Figure 4.9. For these issues, it is natural that the LLM can produce an identical patch, irrespective of data memorization. Moreover, even if an identical patch is produced, it can be that the correct patch is simple and thus easy for an LLM to produce.

4.7 Discussion

User study One of the important design goals of SpecRover is to reduce the cognitive load on the developer in vetting patches generated by coding agents and increase developer trust in the patches. We have partially validated this goal by empirically showing the increased signal-to-noise ratio of the patches generated by SpecRover. However, to fully validate this goal, we need to conduct a more comprehensive user study to compare the developer experience with SpecRover and the baseline coding agents, specifically in the time usage and ease of vetting the patches, and the subjective satisfaction with the generated patches, test cases and reviewer feedback.

Industrial Adoption Following Sonar's acquisition of the startup AutoCodeRover, co-founded by the thesis author[4], SpecRover was further developed into a commercial product, Sonar Remediation Agent, which remediates code quality issues identified by the static analyzer SonarQube [Son11]. The test-based workflow of SpecRover also informed the design of Sonar Foundation Agent[5], a general-purpose coding agent developed at Sonar by the thesis author together with Yuntong Zhang. Compared to SpecRover, Sonar Foundation Agent has more autonomy: the test-based workflow is described in the system prompt as a soft guideline rather than hardcoded in the agent's control flow, which is intended to retain the benefit of the workflow while leveraging the increasing capability of newer LLMs. Sonar Foundation Agent has ranked first on the SWE-bench leaderboard since December 2025 (as of November 2026)[6].

[4] https://www.forbes.com/sites/janakirammsv/2025/02/24/sonar-bets-on-ai-code-automation-with-autocoderover-acquisition/

[5] https://www.sonarsource.com/blog/introducing-sonar-foundation-agent/

[6] See the submission at https://github.com/SWE-bench/experiments/pull/394

Chapter 5

Symbolic Explanations in Coding Agents

5.1 Introduction

The previous chapters focus on improving the quality of code generated by automatic programming systems with imperfect specifications. As the quality of the generated code improves, automatic programming systems, especially LLM-based coding agents, are increasingly used. To manage the generated code, however, human developers need to understand how the code works and why it is correct or incorrect. This understanding helps steer the coding agent in the right direction, and is especially useful for debugging the code when failures occur in production. Without it, the generated code becomes increasingly difficult to manage and maintain over time.

To support this understanding, we envision that the coding agent provides a specification of the generated code. Unlike the specifications in the previous chapters, which state the expected behavior, this specification describes the actual behavior of the generated code. It is read by a human developer, so that the developer can understand the code, whereas earlier specifications are consumed by an automatic programming system. We call such a specification an *explanation*.

The need for trustworthy explanations is evident in real-world usage. When the Copilot agent proposed a fix for an issue in the public `.NET` repository, the lead developer was not satisfied with the patch alone and instead requested an explanation of the underlying bug, asking "what causes us to get into this situation in the first place?" [dot25]. More broadly, the demand for explanations is strongly supported by prior literature on automated debugging. Noller et al. [NSGR22] find that, beyond the patch itself, the most helpful artifact an automated program repair technique can provide is an explanation of the bug. Similarly, Kochhar et al. [KXLL16] report that over 85% of developers consider it important that a debugging technique be able to provide its rationale.

One possible form of explanation is natural language, which an LLM can generate easily. As noted in Chapter 3, however, a natural-language explanation is difficult to validate and often too ambiguous to describe the behavior of the code. We therefore propose the *symbolic explanation*, which describes the behavior of the code in terms of symbolic formulas. The

formulas describe the code precisely and can be checked by executing the code, which gives the developer a concrete reason to trust the explanation. Like an automated Sherpa[1], the symbolic explanation guides the developer through the key factors that lead to a bug.

Concretely, the symbolic explanation contains three conditions, each captured by a formula: the inputs that trigger the bug, the output behavior on those inputs, and the key intermediate program states that lead to that output. We generate it with a multi-agent framework, AutoCodeSherpa, and iteratively refine it through a feedback loop to improve its accuracy. The explanation is meant to support smoother human-agent collaboration. Together, the three conditions help developers answer the question "what causes us to get into this situation?".

We evaluate AutoCodeSherpa along five aspects: (i) the accuracy of its generated explanations, (ii) the assessment of patch correctness, (iii) assistance to other coding agents in resolving bugs, (iv) generalization across LLMs, and (v) scalability. The input, infection, and output conditions achieve accuracies of 85.7%, 79.7%, and 79.0%, respectively, which provides a basis for trust. Run on patches from coding agents, its property-based tests rejected 123% more incorrect patches than the best baseline. The explanations were also informative enough to help the issue-resolving technique Agentless [XDDZ25], improving its plausible patch generation rate by 60.7%. Explanation accuracy stayed consistent across LLMs. AutoCodeSherpa also scales to repositories of different sizes and remains robust on issues that require long patches.

Overall, we make the following contributions:

- We introduce executable symbolic explanations for coding agents. Such an explanation describes a software issue symbolically, in terms of program inputs and states, and can be executed.
- We present AutoCodeSherpa, an agentic tool that automatically generates these explanations.
- We show that the explanations are accurate, separate correct patches from incorrect ones more accurately than the baselines, and can improve the issue-resolution efficacy of other techniques.

5.2 Background

5.2.1 Bug Characterization

Our symbolic explanations are loosely inspired by the reachability, infection, and propagation (RIP) model of failure observation [AO16]. The RIP model notes that for a software bug

[1] A Sherpa is a mountain guide who assists trekkers in climbing mountains.

to be observable, the fault should be reached during execution (reachability), the state of the program should be incorrect (infection), and the infected state should be propagated to a statement making the state observable. This model of bugs has prominently been used in the analysis of mutation testing [JEF14, DPJ24]. Our tripartite formulation of input, infection, and output conditions has a rough correspondence to the RIP model.

Meanwhile, understanding bugs is an integral part of developers' day-to-day work and the first step to issue resolution. Despite this importance, there is little research on associating natural language descriptions of software issues with symbolic conditions. One naive approach to issue explanation is to prompt an LLM to generate a natural-language explanation from the issue description. Although an explanation so generated may appear coherent and help understanding to some extent, it is prone to errors from LLMs, even for issues in simple programs such as introductory-level programming assignments [BKPW23]. To the best of our knowledge, our work is the first to produce a *symbolic* explanation from natural-language issue reports.

5.2.2 Hoare Triple

An explanation for a bug is a description of how the bug affects the program state. A formal way of describing the propagation of program states through code is the *Hoare triple*, whose standard notation is $\{P\}\ C\ \{Q\}$, where P and Q are both properties about the program state, and C is a program. A Hoare triple is *partially correct*, if, whenever the program C starts with a state satisfying P (the *precondition*) *and* terminates, its terminal state will satisfy Q (the *postcondition*). In contrast to partial correctness, total correctness requires reasoning that the program C always terminates when the precondition P is met. As deciding program termination is beyond the scope of this chapter, all Hoare triples in this chapter are partially correct. In this chapter, we formalize the relationship between conditions with Hoare triples. For example, supposing the input condition is I, output condition is O, and program is P, we want to check that the partially correct Hoare triple $\{I\}\ P\ \{O\}$ holds.

5.2.3 Property-Based Testing

Property-based testing (PBT) is a possible way of checking whether a Hoare triple holds. PBT is a powerful software testing technique mainly used to find logical bugs. Beginning with the QuickCheck [CH00] framework for Haskell, PBT frameworks have been developed in popular programming languages like Python [MHD19] and Java [Hol20]; PBTs are mature enough to have had successes in uncovering real bugs in industrial contexts [BJA+21, HPAN16, AHNS15, GCD+24]. In PBT, there is a *property* to be checked, which is an executable specification of the program-under-test. The property often contains a *precondition* that specifies the valid domain of inputs. The PBT framework automatically checks the property on a large number of randomly or semi-randomly generated inputs, which are produced by a *generator* and filtered by the precondition. In our context, to check whether the Hoare triple

```
1 from hypothesis import given, assume
2 from hypothesis.strategies import floats
3
4 # PBT for a function `reciprocal()`
5 @given(floats(allow_nan=False, allow_infinity=False)) # generator
6 def test_reciprocal_property(x):
7     assume(x != 0) # precondition
8     # lines 9-10 are the property
9     result = reciprocal(x)
10    assert abs(x * result - 1.0) < 1e-9
```

Figure 5.1: An example PBT

$\{I\}\ P\ \{O\}$ holds, we use a PBT to repeatedly sample inputs that satisfy I, execute P, and check if O holds.

In Figure 5.1, we show a simple example PBT written in the Python Hypothesis [MHD19] framework. In the example, the program-under-test is a function `reciprocal` calculating the reciprocal of a floating point number. The property being tested is that a number multiplied by its reciprocal is equal to one. The precondition for the test is that the number is non-zero, and thus the generator simply produces all non-zero floating point numbers except `NaN` and infinity.

PBTs stand at a useful middle ground between traditional software testing and formal methods. On one hand, because PBTs execute a large number of inputs, they are generally more rigorous than the most commonly used example-based testing, which only checks a few inputs picked by the developer. On the other hand, PBTs are usable on a wider range of programs and can be efficiently computed. PBTs have the further benefit of being syntactically similar to example-based tests, making them accessible to developers. This is in contrast to deductive verification methods, which require specialized mathematical knowledge and thus have a higher entry barrier.

5.2.4 Coding Agents

In this chapter, we highlight the use of symbolic explanations for coding agents. Coding agents perform software engineering tasks by leveraging LLMs to autonomously invoke external tools. Since 2024, a variety of coding agents have emerged, differing in their levels of autonomy and their choices of tools. Representative examples include:

- AutoCodeRover [ZRFR24]. AutoCodeRover follows a two-stage workflow consisting of context retrieval and patch generation. During context retrieval, it autonomously invokes a tool to search the program's abstract syntax tree for relevant code snippets. In the patch generation stage, it produces a code change to resolve the issue.
- SpecRover (Chapter 4). SpecRover is an extension of AutoCodeRover which achieves high precision by writing a reproducer test and observing its execution result with its patch

applied. In this chapter, we use SpecRover as a baseline in patch validation capability due to its emphasis on precision.

- Agentless [XDDZ25]. Agentless is a coding agent with minimal autonomy. It employs a three-phase process consisting of localization, repair, and patch validation. In each phase, the prompts to the LLM follow fixed templates. Due to its simplicity, Agentless is sensitive to input variations. Therefore, we evaluate the quality of our symbolic explanations by providing them to Agentless and observing the resulting gains in issue resolution efficacy.
- OpenHands [WLS+25]. OpenHands is a flexible agent capable of using a variety of tools to execute diverse tasks. For coding tasks, it primarily uses a bash tool to execute terminal commands and a text editor tool to view and modify code. OpenHands has no predefined workflow, and thus has the highest level of autonomy among the agents in this list.

While coding agents are becoming increasingly effective at resolving software issues, as demonstrated by open benchmarks [SWE24], the code changes they produce may still be incorrect, even when validated against a limited number of test cases [WPL25]. This challenge is long-standing and is known in the automated program repair (APR) community as the overfitting problem [QLAR15]. To make coding agents more trustworthy, we propose symbolic explanations that provide coding agents with additional information to improve their effectiveness in resolving software issues. Moreover, these symbolic explanations can be executed to validate the code changes produced by coding agents. We evaluate both of these use cases (patch validation and improving issue resolution) in this chapter.

5.3 Overview

Figure 5.2 presents an overview of AutoCodeSherpa, along with a running example used throughout this section. As illustrated in the upper part of the figure, AutoCodeSherpa

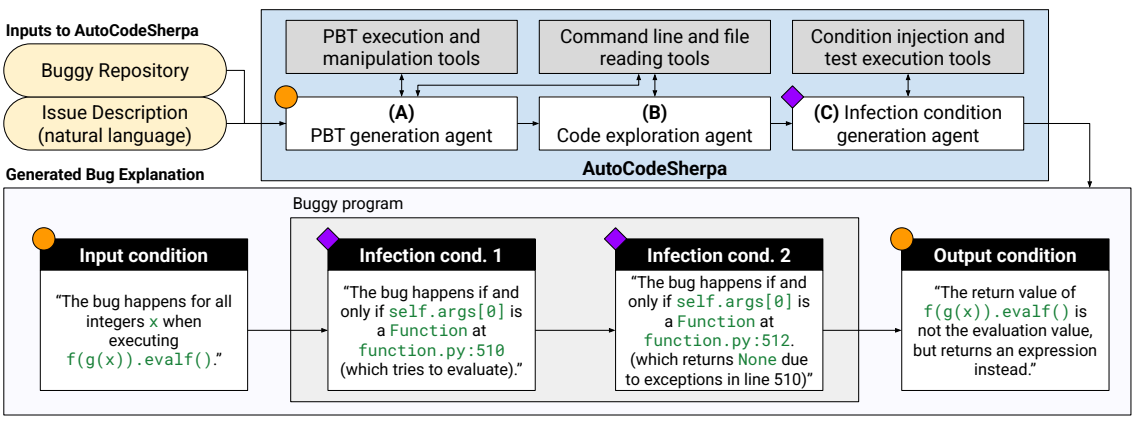


Figure 5.2: Overview of AutoCodeSherpa with a real example simplified for clarity; PBT is for property-based test.

```
500 def _eval_evalf(self, prec):
501     # Lookup mpmath function based on name
502     fname = self.func.__name__
503     try:
504         if not hasattr(mpmath, fname):
505             from sympy.utilities.lambdify import MPMATH_TRANSLATIONS
506             fname = MPMATH_TRANSLATIONS[fname]
507         func = getattr(mpmath, fname)
508     except (AttributeError, KeyError):
509         try:
510             return Float(self._imp_(*self.args), prec)  # <- BUG IS HERE
511         except (AttributeError, TypeError, ValueError):
512             return
513 ...
```

Figure 5.3: The buggy function involved in our running example.

takes as input a natural language issue description and the corresponding buggy program, and produces an explanation of the issue. Our goal is to explain why a bug occurs in a symbolic manner, inspired by the reachability–infection–propagation model described in Section 5.2.1. To this end, AutoCodeSherpa employs LLM agents to identify three complementary conditions:

- Input condition, which specifies the set of program inputs that trigger the issue;
- Infection condition, which describes buggy internal program states;
- Output condition, which describes an observable fault.

AutoCodeSherpa also ensures that the conditions are connected by program execution: on the buggy program, execution on an input satisfying the input condition reaches a state satisfying the infection condition and then exhibits the fault specified by the output condition. This connection is checked by executing a PBT consisting of the input and output conditions while observing the values of the infection condition. A formal definition of the conditions is provided in Definition 1.

To illustrate our approach, we present a running example highlighting the high-level operations of AutoCodeSherpa. The issue references a Stack Overflow post titled "Why can't I evaluate a composition of `implemented_functions` in SymPy at a point?" The root cause resides in line 510 of SymPy's `_eval_evalf` function, shown in Figure 5.3, while the symbolic explanation generated by AutoCodeSherpa is shown in the lower half of Figure 5.2.

To explain the issue, AutoCodeSherpa first runs a PBT generation agent (Figure 5.2(A)). The agent encodes the input condition as a Python function that generates a set of inputs, and the output condition as an assertion statement. It then checks that all generated inputs trigger the assertion failure. In our example, the PBT asserts that for all integers `x` within a certain range, `f(g(x))` produces an `Expression` object instead of an expected concrete value.

Further, to explain how the bug propagates inside the program, AutoCodeSherpa generates infection conditions—first-order logic formulae evaluated at specific locations to distinguish buggy states from normal ones. The code exploration agent (Figure 5.2(B)) identifies these locations by searching the code based on the issue description and the generated PBT. In our running example, the bug report already names the buggy function (Figure 5.3), allowing the agent to quickly locate it in the repository as a candidate location. We note that while this example issue has a single buggy location, the symbolic explanation technique applies to multi-location issues as well.

Within the identified buggy function, the infection condition generation agent (Figure 5.2(C)) then proceeds to produce symbolic expressions that evaluate to true only for bug-triggering inputs from the PBT. In this example, the agent generated conditions at lines 510 and 512 of the buggy function (Figure 5.3). Both conditions indicate that the bug occurs when the argument of a `Function` is itself a `Function`. They accurately capture the cause of the bug: the buggy line 510 fails to handle the argument, triggering the exception-handling code in line 512 that should not have been executed.

The symbolic explanations produced by AutoCodeSherpa have several practical uses. First, the PBT in the symbolic explanation can be executed against candidate patches to filter out incorrect ones, a use case evaluated in Section 5.7.2. Second, the infection conditions can enhance other software engineering techniques, such as coding agents, by generalizing beyond the issue description and revealing the internal logic of the bug. This use case is evaluated in Section 5.7.3.

5.4 Symbolic Explanations: Definition

In this section, we present the formal definition of our symbolic explanations of software issues, consisting of input, output, and infection conditions, as follows.

Definition 1 (Symbolic explanation)**.** *Given a program P whose input parameters are i, an issue X on the program, a symbolic explanation for issue X is a triple (I, F_L, O) that satisfies the following:*

- *Input condition I is a quantifier-free first-order logic formula over i, and for all inputs i that satisfy I, executing P with i triggers X;*
- *Output condition O is a quantifier-free first-order logic formula over the terminal state of P that holds only if X is triggered, and for which the partially correct Hoare triple $\{I\}\ P\ \{O\}$ holds; thus by starting P with inputs satisfying I, O is guaranteed;*
- *Supposing $P = C_1; C_2$ where C_1, C_2 are sequences of program statements, and L is the program location immediately after C_1, infection condition F_L is a quantifier-free first-order logic formula over the program state at L, for which the partially correct Hoare*

```
### Gist of the Report
The issue is that the `evalf` method in SymPy does not recursively evaluate the result of `_imp_`...

### Concrete Bug-Reproducing Inputs

1. **Input**: `f(g(2)).evalf()`
   **Expected**: `16.000000000000000`
   **Actual**: `f(g(2))` (fails, does not evaluate recursively)

2. **Input**: `g(f(2)).evalf()`
   **Expected**: `8.00000000000000`
   **Actual**: `g(f(2))` (fails, does not evaluate recursively)
...
```

Figure 5.4: Output from the generalization phase of PBT generation.

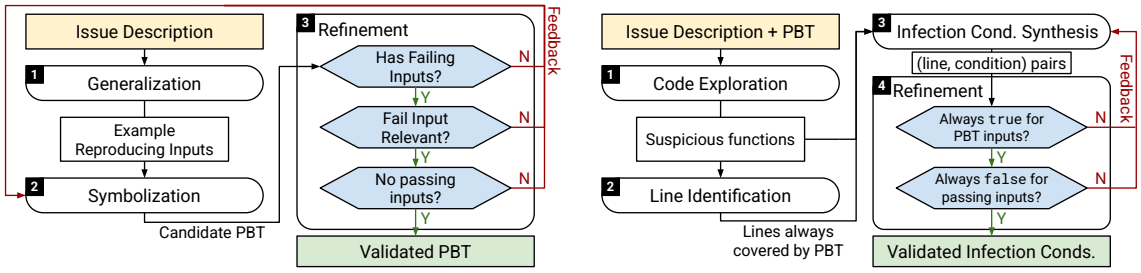
triples $\{I\}\ C_1\ \{F_L\}$ *and* $\{\neg I\}\ C_1\ \{\neg F_L\}$ *hold, i.e.,* F_L *is the result of symbolically propagating* I *to* L*.*

By defining symbolic explanations as above, our explanations are able to provide the following information. First, input conditions describe the set of inputs triggering a software issue. Literature on bug reports shows that providing inputs to trigger the issue is a key factor in bug report quality [BB09]. Second, output conditions describe how the issue influences program output, so that readers may recognize the issue when it happens. Third, infection conditions detail how the issue propagates within the program. This helps developers, as they frequently inspect program states to better comprehend issues [AL23, BSSZ18]. Furthermore, the three conditions are connected via program execution: on the buggy program, if the input condition holds, the program must reach a state satisfying the infection condition, from which it then reaches a state satisfying the output condition.

We implement each condition as executable artifacts, making our symbolic explanations executable, unlike natural language explanations. In this work, AutoCodeSherpa is implemented for Python. The input and output conditions I and O are implemented as a PBT. In particular, I is implemented as a function that generates a set of inputs, and O specifies an expected exception type. Infection conditions are implemented as first-order logic formulae, evaluated at specific lines in Python files via runtime monitoring. The PBT framework Hypothesis [MHD19] introduced in Section 5.2.3 is used to repeatedly sample inputs from the input set, then check that the Hoare triples defined in Definition 1 hold.

5.5 Symbolic Explanations: Construction

In this section, we present our agentic approach AutoCodeSherpa for generating the symbolic explanation, which involves generating a PBT for input and output conditions and program-internal expressions for infection conditions.



(a) PBT generation agent flowchart. **(b)** Infection condition generation agent flowchart.

Figure 5.5: Detailed condition generation flowcharts for AutoCodeSherpa.

5.5.1 PBT Generation

The first step in generating a symbolic explanation is to derive the input condition I and output condition O from an issue description, as illustrated in Figure 5.2. This process follows a three-step agentic workflow: generalization-symbolization-refinement, as shown in Figure 5.5a.

Generalization To encourage generalization from the issue description, the PBT agent is first prompted to generate multiple bug-reproducing inputs based on the description, along with their corresponding actual and expected program outputs. Figure 5.4 illustrates the agent's response at this stage for the running example in Section 5.3. While these input-output pairs may be imperfect, as they are generated by an LLM, they serve as a reasoning step from the specific input in the description to the general conditions I and O, supporting the subsequent symbolization and refinement steps.

Symbolization With example inputs generated, the PBT agent next abstracts over them to write a PBT to reproduce the issue, using symbolic expressions to represent the input and output conditions I and O. As shown in Figure 5.2, the agent is equipped with tools for command-line execution, file reading/writing, and PBT execution, enabling it to explore relevant files and perform trial-and-error before proposing a PBT. Once a PBT is proposed, it is executed to check if the exception specified by O is raised. If the exception is not raised, we prompt the PBT agent to retry. Note that this step only ensures the *existence* of an input i satisfying I that leads to O. The PBT is then refined in the next step by testing whether sampled inputs that satisfy I also lead to O.

Refinement The refinement step tests whether $\{I\}\ P\ \{O\}$ holds on sampled inputs. These tests provide empirical evidence but do not establish the triple for all inputs. To achieve this, two types of problems need to be detected and addressed. First, some failing inputs may fail for reasons unrelated to the described issue. To handle this, we either strengthen I to prevent the PBT from generating irrelevant inputs, or strengthen O so that these inputs

do not trigger spurious failures. Concretely, we execute the PBT to collect failing inputs and their corresponding exceptions, which are then presented to the PBT agent for review relative to the issue description. If the agent determines that any failing input i is irrelevant to the described issue, the PBT is routed back to the symbolization step to generate a new PBT. Additionally, the PBT generation prompt is enriched to exclude these irrelevant failing inputs.

The second potential problem is that, even after ensuring that all failing inputs are relevant, the PBT may still generate passing inputs. To address the problem, we strengthen I to filter out these inputs. While one could also attempt to fix the problem by weakening O, we avoid doing so, as this could incorrectly reject a correct program P', reducing the usefulness of the symbolic explanation. Concretely, we execute the PBT to identify passing inputs—those that satisfy I but do not lead to O. Any passing inputs are then presented to the agent, which is prompted to revise the input condition I to exclude the inputs. The input condition can be revised in any of the following three ways: (1) adding additional `assume` statements filtering out passing inputs; (2) changing the parameters of the generator (e.g. the `allow_nan` parameter in Figure 5.1), or (3) modifying the input generator function to change the sampling process itself. In practice, we find that parameter tuning and `assume` statement additions were the most common modifications. After revision, the PBT is executed again to check for passing inputs, and the process repeats until all inputs lead to the target failure.

5.5.2 Infection Condition Generation

After generating the input and output conditions, AutoCodeSherpa produces infection conditions F_L to aid understanding of the program itself. As shown in Figure 5.5b, infection condition generation consists of four steps, namely code exploration, line identification, infection condition synthesis, and refinement.

Code Exploration To generate infection conditions, which are defined at specific program lines, AutoCodeSherpa first finds functions relevant to the bug (suspicious functions). To do so, we reuse the context retrieval agent of AutoCodeRover [ZRFR24]. This agent finds likely buggy functions by invoking a tool that searches the abstract syntax tree of the program, e.g., searching for a class or for a function in a certain class. The search starts by the agent identifying keywords from the issue statement. The result of the search would reveal relevant parts of the program, and the agent would analyze the result in relation to the issue and possibly launch another search for interesting elements in the result. This process is repeated until the agent decides that the suspicious functions have been found. At this point, the identified buggy functions and the intermediate search results are passed to the infection condition agent.

Line Identification The infection condition agent receives the buggy functions identified by the code exploration agent and uses an LLM to select candidate code lines within these

functions. The agent checks each candidate line to ensure it is executed by all inputs generated from the input condition I. Specifically, for each candidate line, the program is instrumented by adding a statement raising a custom exception just before the candidate line. After this instrumentation, the generated PBT is executed to see if the custom exception is triggered for all sampled inputs. If the custom exception is not always triggered, indicating a candidate line is not always covered, the agent provides feedback to the LLM, which proposes revised lines. After up to five iterations, the agent produces a final set of lines covered by all inputs from I.

Condition Synthesis For each location L identified in the line identification stage, the agent presents an LLM with the buggy function and the issue description, through which the LLM generates a candidate symbolic expression F_L. Specifically, the concept of infection conditions is described to the LLM in its system prompt via the sentence "The conditions should evaluate to True for all bug-inducing inputs, and False for inputs that behave normally/are unrelated to the bug". During operation, the LLM is then prompted to "generate an infection condition at location [LOCATION]; you must generate general conditions that capture all bug-triggering inputs while excluding all passing tests." In response, the LLM generates a candidate infection condition; whether the candidate condition meets the definition of an infection condition is checked in the next step.

Refinement A candidate infection condition can fail to meet Definition 1 in the following two ways:

- Too strong: The condition is false for at least one PBT-sampled input. That is, there exists an input i satisfying the input condition I such that the infection condition F_L does not hold at location L for at least one visit to L during execution.
- Too weak: The condition is true for at least one test unrelated to the bug. That is, there exists an input i that does not satisfy the input condition I such that the infection condition F_L holds at location L for at least one visit to L during execution.

The refinement step checks whether a candidate infection condition F_L is too strong or too weak. If F_L passes both checks, it is accepted. Otherwise, the agent attempts to refine it by incorporating feedback about the failure, together with the issue description and surrounding source code. This refinement is attempted up to five times, after which the agent moves on to the next code location. When condition synthesis has been attempted on all code lines from the line identification stage, the agent returns the successfully identified infection conditions.

This completes the construction of a symbolic explanation, consisting of a PBT (representing input and output conditions) and infection conditions. As a final step to achieve easier presentation, one can direct an LLM to convert the symbolic explanations to a natural language report. One can trace information in this report back to the conditions identified by

AutoCodeSherpa, which have the properties outlined in Definition 1, unlike natural language explanations generated by LLM prompting alone. The report can be consumed by a human developer or another coding agent.

5.6 Experimental Setup

We evaluate the following research questions:

RQ1: How accurate are the input, output and infection conditions in the explanations?

RQ2: Are the generated PBTs useful for filtering incorrect patch candidates?

RQ3: Can the symbolic explanations improve the efficacy of issue resolution techniques?

RQ4: Does our symbolic explanation generation procedure generalize to different LLMs?

RQ5: How does symbolic explanation generation scale on repositories and bug complexity?

Benchmark To evaluate our approach, we use the SWE-bench Verified benchmark [CAS+24]. SWE-bench Verified is a subset of SWE-bench [JYW+24] and consists of 500 issues from 12 open source repositories that were labeled as solvable by humans. Despite this labeling, we observe that some issues in SWE-bench Verified are under-specified, in that the expected program behavior is not fully described in the issue text. Each issue includes a natural-language description, which AutoCodeSherpa uses as input to generate explanations.

SWE-bench Verified provides two types of test cases: fail-to-pass (F2P) and pass-to-pass (P2P). F2P test cases are intended to reproduce the issue: they fail on the buggy program and should pass on the fixed version. These tests are not available to LLM agents (including AutoCodeSherpa) and are used only for evaluation. In contrast, P2P test cases are regression tests that pass on both the buggy and fixed programs and are available to LLM agents.

RQ1: Accuracy of Conditions In RQ1, we report how often AutoCodeSherpa generates a symbolic explanation for issues in the benchmark. A symbolic explanation is not produced if the LLM fails to generate an input, output, or infection condition that passes the refinement steps described in Sections 5.5.1 and 5.5.2. For the explanations that are successfully generated, we evaluate the correctness of each condition using the following procedure:

- Input and output conditions are considered correct if the PBT they form fails on the buggy program and passes on the fixed program. If this is not the case, we manually examine the input and output conditions to decide their correctness.
- Infection conditions are evaluated using SWE-bench Verified test cases: an infection condition is considered correct if and only if it evaluates to true on all F2P tests and false on all P2P tests.

RQ2: Patch Validation Capability In RQ2, we investigate whether the symbolic explanations from AutoCodeSherpa can be used to filter out incorrect patches produced by coding agents. As an example, we evaluate patches from the precision-focused agent SpecRover (Chapter 4). We report how frequently AutoCodeSherpa could correctly distinguish correct patches from incorrect ones.

Ground-truth To determine patch correctness, we first run the P2P and F2P test cases provided by SWE-bench Verified. Patches that pass the tests are then manually checked for semantic equivalence with the human patch from the benchmark. To do this, two annotators (the thesis author and a co-author) independently compared the agent-generated and human patches, then discussed and resolved disagreements to reach a final verdict on the agent patch's correctness. Both are experienced Python developers (10+ and 5+ years) familiar with the SWE-bench repositories, which aided the assessment.

Prior to discussion, the two annotators achieved 92.5% agreement on the correctness of the 259 plausible SpecRover patches, with a Cohen's kappa of 0.81 (almost perfect agreement). Most comparisons were straightforward: 89 (34.3%) were syntactically identical or easily transformable to the developer patch, while most of the remaining 170 had small diffs (a median of 6 lines).

Baselines We compare AutoCodeSherpa against three baselines: SpecRover (Chapter 4), OpenHands [WLS+25], and OpenHands-PBT. SpecRover predicts patch correctness based on the issue description and an agent-generated example-based reproducer test. Patches produced by SpecRover, along with their validation results, are taken from its latest public data [SWE24]. OpenHands is a general-purpose coding agent with state-of-the-art efficacy in generating bug-reproducing tests in SWT-bench Verified [MMHV25] among publicly available data. In particular, the best publicly available results from OpenHands on the SWT-bench were obtained using GPT-5-mini, which we also use throughout RQ1–RQ3 for a fair comparison. An OpenHands test is deemed to validate a patch if it passes after the patch is applied. The third baseline, OpenHands-PBT, modifies the system prompt of OpenHands by adding the instruction "The tests you write must be property-based tests using the Hypothesis framework. If `hypothesis` is not already installed in the repo, `pip install hypothesis` before running the tests.", which led it to consistently generate PBTs. PBTs from OpenHands-PBT are executed the same way as those from AutoCodeSherpa. The classification accuracy of all three baselines is then evaluated against the ground-truth correctness labeled above.

RQ3: Efficacy Gains in Issue Resolution In RQ3, we evaluate an agent-agent interaction scenario, which may become more common as coding agents grow in prevalence. Specifically, we examine whether the symbolic explanations generated by AutoCodeSherpa improve the efficacy of Agentless in fault localization and issue resolution. For bugs where

input, infection, and output conditions were all generated, an LLM converted the symbolic explanation into a natural language report, as described in Section 5.5. This report was then supplied as additional information to Agentless, and efficacy was compared against default Agentless (without explanations) on the same set of bugs. We choose Agentless because its simple structure makes it more sensitive to the quality of explanations than other techniques. Issue resolution efficacy is measured via the plausible patch rate, defined as the proportion of patches that pass the developer-written reproducer test. We evaluate all ten patches that Agentless generates during its patch generation process.

To assess the impact of explanations, we append the explanation to the bug report as follows: "In addition, a trustworthy process has provided the following explanation for the bug: {AutoCodeSherpa explanation}". Agentless efficacy under this setting is compared with a no-explanation setting.

RQ4: Generalization across LLMs In RQ1–RQ3, we reported results for AutoCodeSherpa generated with GPT-5-mini, the same LLM used by our best baseline, OpenHands. In RQ4, we repeat the experiments from RQ1–RQ3 using several other LLMs to assess the generalizability of AutoCodeSherpa across different models. In particular, we evaluate AutoCodeSherpa on Claude-3.5-Sonnet, a closed-weight LLM from a different provider, and DeepSeek-V3.2, an open-weight model.

RQ5: Scalability across Repositories and Issue Complexity In RQ5, we evaluate whether AutoCodeSherpa performs consistently across repositories of different sizes (by lines of code) and issues of different patch complexities (by the number of changed lines and functions in the developer patch). Analyzing the RQ1 results, we track three indicators—PBT correctness, infection condition correctness, and total cost (USD)—and report the 95% confidence interval of each mean. For the binary indicators (PBT and infection condition correctness) we use the binomial standard error $\sqrt{\hat{p}(1-\hat{p})/n}$, and for continuous indicators (cost and runtime) the Bessel-corrected standard error of the mean, each scaled by 1.96.

Parameters The PBT agent was allowed at most 50 LLM calls to limit time and cost. For the code exploration agent, up to 50 invocations of the search tool were permitted, which is its default setting [Aut24]. The infection condition agent could suggest lines at most five times, and for each suggested line, the infection condition was iteratively improved up to five times. For fair comparison with OpenHands in RQ2, the experiments in RQ1–RQ3 were performed on GPT-5-mini. A temperature of 0 is used for DeepSeek-V3.2 and Claude-3.5-Sonnet (temperature cannot be set for GPT-5-mini). The programs under test were set up using the official Docker images of SWE-bench Verified, with a memory limit of 6 GB per container.

5.7 Evaluation

5.7.1 RQ1: Accuracy of Conditions

In RQ1, we evaluate the accuracy of the generated input, infection, and output conditions, as a necessary step in understanding the usability of the approach.

Figure 5.6 presents the ratio of successful PBT generations, including both input and output conditions, alongside the accuracy of the infection conditions. We note:

- First, the PBTs generated by AutoCodeSherpa were highly accurate. A PBT specifying both the input and output conditions was generated for 433 of the 500 issues (86.6%) in SWE-bench Verified. In these PBTs, 85.7% of the input conditions and 79.0% of the output conditions were correct. In 318 (73.4%) PBTs, both conditions were correct.
- The infection conditions generated by AutoCodeSherpa were similarly accurate. Out of the 433 issues with a PBT, a total of 1001 infection conditions were generated for 289 issues (an average of 3.46 infection conditions per issue), of which 798 (79.7%) were correct.
- The generated symbolic explanations are thus highly accurate, and would likely be sufficient for developers to trust AutoCodeSherpa's results. Prior studies on automated debugging suggest that roughly 75% accuracy is considered trustworthy by about 75% of developers [KXLL16].

Despite the high overall accuracy, we analyze the circumstances under which AutoCodeSherpa produces imperfect results. First, the infection condition generation rate leaves room for improvement: infection conditions were not generated for 33.3% of all issues with generated PBTs. One challenge is the opacity of objects in certain projects. For instance, the Django project—which accounts for 46% of the SWE-bench Verified benchmark—uses deeply nested objects, where buggy states can be subtle, making infection conditions harder to generate. Consequently, the infection condition generation rate for Django was lower compared to repositories such as matplotlib, which achieved a 90% generation rate. Nonetheless, AutoCodeSherpa still generated infection conditions for a non-trivial 62% of Django issues. Future work could improve infection condition generation by leveraging serialization libraries[2] to expose program state and guide condition synthesis.

Second, output conditions are the most difficult to generate accurately and have the lowest accuracy of 79.0% among the three condition types. Some issue descriptions only cover the buggy behavior, providing hints about the input and infection conditions but leaving the intended behavior implicit or incomplete, which makes it difficult to generate correct output conditions.

[2] https://github.com/explosion/srsly

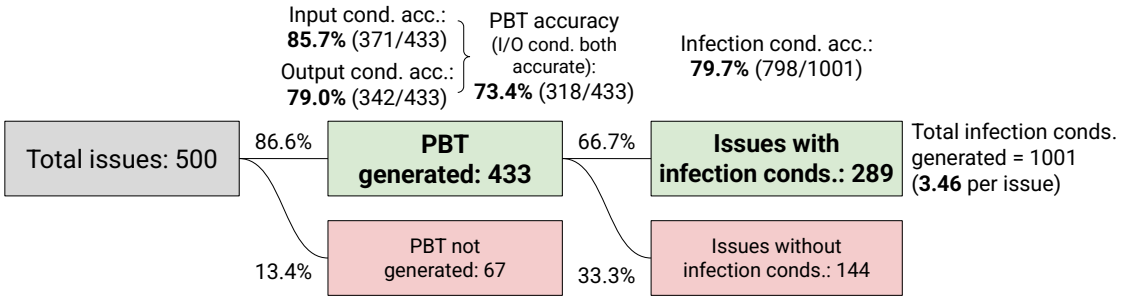
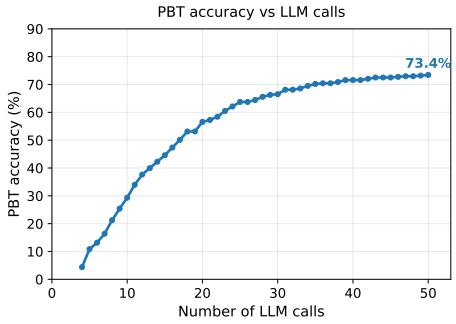
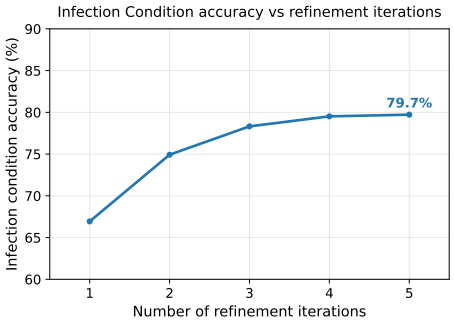


Figure 5.6: A plot of generation ratio and condition accuracy.

Impact of LLM Calls and Refinement Iterations We also examine how the number of LLM calls and refinement steps affects the accuracy of generated PBTs and infection conditions. As shown in Figure 5.7, the accuracy of generated PBTs increases steadily and plateaus after about 40 LLM calls, while the accuracy of generated infection conditions plateaus after 4 refinement steps. These results suggest that the chosen number of LLM calls and refinement steps for the PBT agent and the infection condition agent in Section 5.6 are sufficient for ensuring accuracy of the symbolic explanations.



(a) PBT accuracy across LLM calls, starting from the minimum number of calls the PBT agent needs to terminate.



(b) Infection condition accuracy across refinement iterations.

Figure 5.7: Effect of LLM calls and refinement iterations on explanation accuracy.

Answer to RQ1: AutoCodeSherpa is capable of generating accurate symbolic explanations for a large proportion of issues.

5.7.2 RQ2: Patch Validation Capability

Patch Validation Procedure RQ2 evaluates the capability of our symbolic explanations to validate agent-generated patches. To do so, a patch is first applied to the buggy program, and the PBT from the explanation is executed. The PBT framework repeatedly samples bug-triggering inputs and runs the PBT; if a test failure still occurs with the patch applied, the patch is considered invalid.

Metrics The symbolic explanation can be viewed as a binary classifier—patches that pass the PBT are classified as valid (positive), while those that fail are classified as invalid (negative). To assess its efficacy, we evaluate the confusion matrix of the classifier— i.e., true positive (TP), false positive (FP), true negative (TN), and false negative (FN)—along with derived metrics such as precision and recall. As mentioned in Section 5.6, evaluation is performed on patches generated for SWE-bench Verified [CAS+24] by SpecRover [RZR25], an LLM agent designed to improve patch precision.

Table 5.1: Confusion matrices of AutoCodeSherpa and baselines, where positive means correct patch.

	AutoCodeSherpa	SpecRover	OpenHands	OpenHands-PBT
#TP ↑	132 (33.3%)	**147 (37.1%)**	136 (34.3%)	134 (33.8%)
#TN ↑	**105 (26.5%)**	29 (7.3%)	47 (11.8%)	41 (10.4%)
#FP ↓	**140 (35.4%)**	216 (54.5%)	198 (50.0%)	204 (51.5%)
#FN ↓	19 (4.8%)	**4 (1.0%)**	15 (3.8%)	17 (4.3%)
Total		396		
False positive rate = FP / (TN+FP) ↓	**57.1**%	88.2%	80.8%	83.5%
Precision = TP / (TP+FP) ↑	**48.5**%	40.5%	40.7%	41.1%
Recall = TP / (TP+FN) ↑	87.4%	**97.4**%	90.1%	93.4%

Table 5.2: Reasons for FP patch categorization

PBT only checks for exception	53 (36.4%)
Issue description incomplete	51 (36.4%)
Developer patch goes beyond issue scope	36 (27.2%)

Table 5.1 shows the confusion matrix and related metrics for AutoCodeSherpa and the baselines. A total of 396 issues are included—those for which AutoCodeSherpa, SpecRover, OpenHands, and OpenHands-PBT all generated a test. Focusing on the 245 (= TN + FP) incorrect patches, AutoCodeSherpa invalidated 105 of the incorrect patches (TN), i.e., the PBT failed after their application. This number of true negatives is 262.0% higher than SpecRover, 123.4% higher than OpenHands, and 156.1% higher than OpenHands-PBT, indicating that AutoCodeSherpa also achieves a much lower false positive rate than all three baselines.

To understand the 140 false positives (FP)—i.e., incorrect patches not flagged by AutoCodeSherpa —we performed a manual analysis and summarized the reasons in Table 5.2. Among these 140 FPs, 53 PBTs only included checks for an exception, allowing any patch that resolved the exception to pass. This reflects the well-known overfitting problem in program repair, where weak test oracles can permit plausible but incorrect

patches [QLAR15]. For 51 issues, the issue description incompletely specifies the expected behavior—it describes only the buggy behavior but not the precise expected outputs when the bug is fixed. The PBT could not invalidate these patches because it only checks for the buggy behavior. Applying the regression tests of the program under test could further restrict patch behavior and potentially filter out 11 of these 51 incorrect patches. Finally, for 36 issues, the patch correctly handles the reported issue (and is thus flagged as correct by the PBT), but the developer patch goes beyond the reported issue and makes additional changes to the program. In these cases, AutoCodeSherpa correctly allows the patches to pass according to the issue description.

Meanwhile, among the 151 (=TP+FN) correct patches, AutoCodeSherpa produced 19 false negatives (FN). An examination of their issue descriptions shows that 3 issues are uninformative: they provide little or no description of the buggy behavior and instead directly describe a bug fix. Due to the insufficient information, generating accurate PBTs for such issues is difficult. Another issue describes the buggy and expected behaviors using images, whereas our agent processes text only. For the remaining 15 FN issues, the descriptions are informative, but the generated PBTs are incorrect. These errors are caused by LLM hallucinations and do not exhibit a common pattern. Despite these FN cases, AutoCodeSherpa achieves a recall comparable to that of the baselines.

Finally, we also calculate precision. Precision is important as it captures how much of developers' effort in reviewing agent-generated patches would be spent on actually correct patches. As shown in Table 5.1, AutoCodeSherpa has a clear margin in precision over the SpecRover, OpenHands, and OpenHands-PBT baselines as well.

Answer to RQ2: Compared to the SpecRover, OpenHands, and OpenHands-PBT baselines, AutoCodeSherpa is more effective at identifying incorrect patches while achieving higher precision.

5.7.3 RQ3: Efficacy Gains in Issue Resolution

In RQ3, we evaluate the efficacy gains in fault localization (FL) and issue resolution achieved by providing symbolic explanations to an example coding agent, Agentless.

Table 5.3: Agentless fault localization efficacy, with and without explanations

	File top-1	Element top-1	Element top-any
Without explanations	65.7%	27.3%	64.4%
With explanations	**78.2%** (+19.0%)	**42.6%** (+55.7%)	**76.8%** (+18.9%)

Table 5.3 shows how providing explanations from AutoCodeSherpa improves the FL accuracy of Agentless. At the file level, access to explanations increases top-1 localization accuracy (a true buggy file is ranked as most suspicious) by 19.0%. For *element*-level localization, where

Agentless identifies suspicious classes and methods, top-1 accuracy improves by 55.7% with explanations. For the *top-any* metric (i.e., whether the buggy element is identified at all), explanations enable the agent to correctly identify 18.9% of elements that it would otherwise have missed.

Agentless also benefited from explanations when generating patches, as shown in Table 5.4. As described in Section 5.6, Agentless generates ten patches per issue, both when it is and is not given explanations. Considering all ten patches for all issues, access to symbolic explanations increased the number of plausible patches by 60.7%, indicating that the explanations help guide Agentless toward effective fixes. When considering whether an issue was resolved with at least one plausible patch, Agentless successfully resolved 57.8% of all issues, a 32.5% improvement compared to when no explanations were provided.

To further understand what was driving the increase in plausible patch ratio and resolved issues, we experimented with a variant explanation that only used the PBT. We found that the plausible patch % increased by 9% in this PBT-only setting, in contrast to the 60.7% increase when infection conditions were provided. This suggests the critical role that infection conditions play in bug explanation: by providing concrete locations and conditions distinguishing buggy program states, they can help other agents generate effective patches.

Table 5.4: Agentless plausible patch and bug resolve rate, with and without explanations

	Plausible Patch %	Resolved Issues
Without explanations	29.2%	43.6%
With explanations	**47.0%** (+60.7%)	**57.8**% (+32.5%)

Despite the efficacy gains, some issues remain unresolved. To study the reasons for failure, we investigate a sample of 30 bugs where Agentless could not generate a plausible patch despite being given an explanation. We categorize the reasons for failure as follows. First, limitations in the benchmark itself can prevent Agentless from passing the developer test. In 30% of the sampled cases, the test checks behavior beyond what the issue describes. As a result, even when Agentless generates a patch consistent with the description, the developer test may still fail. For example, in `sympy__sympy-21930`, the issue description mentions only one class, but the developer test also tests other classes. For 46.7% of sampled cases, the explanation described how the bug occurred, but did not specify how the issue should be resolved. Remaining minor issues included (a) Agentless producing patches with syntax errors (10%); (b) the explanations being too vague (6.7%); and (c) the explanations being misleading (6.7%).

Answer to RQ3: Providing Agentless with explanations from AutoCodeSherpa considerably improved both its fault localization and patch generation efficacy.

Table 5.5: Evaluation of AutoCodeSherpa with different LLMs

Metric	GPT-5-mini	Claude-3.5-Sonnet	DeepSeek-V3.2
PBT Generation (RQ1) (%)	86.6	48.4	57.8
PBT Correctness (RQ1) (%)	73.4	74.4	72.2
Infection Condition Correctness (RQ1) (%)	79.7	70.3	85.1
Patch Validation Precision (RQ2) (%)	48.5	56.1	51.5
Agentless Plausible Improvement (RQ3) (%)	+60.7	+11.1	+50.2

Table 5.6: Effect of refinement in PBT agent. Refinement brings the correctness rate of different LLMs close. *Rectified* means the first PBT was incorrect but was changed into a correct one by the refinement step, while *Degraded* means a correct first PBT was changed into an incorrect one. *Discarded* means a PBT was initially generated but eventually not submitted, as it could not pass the refinement step.

	First PBT		Refinement			Final PBT	
	Generated	Correct	Rectified	Degraded	Discarded	Generated	Correct
GPT-5-mini	477	246 (51.6%)	83	2	44 (35 incorrect)	433	318 (73.4%)
DeepSeek-V3.2	462	181 (39.2%)	76	2	167 (125 incorrect)	295	213 (72.2%)

5.7.4 RQ4: Generalizability across LLMs

To demonstrate that AutoCodeSherpa generalizes to other LLMs, we ran it using different LLMs to evaluate performance, the results of which are shown in Table 5.5. AutoCodeSherpa could generate explanations with similar correctness, regardless of LLM: all models showed similar PBT and infection condition correctness, as well as in patch precision evaluated as in RQ2. However, the PBT generation rate differed between LLMs, indicating their capability difference. Taken together, these results suggest that even while LLMs have capability differences, the automated refinement of AutoCodeSherpa (illustrated in Figure 5.5) helps ensure similar quality for explanations.

To further understand the effect of the refinement steps, we analyze the difference in correctness between the *first* PBT suggested by LLMs in AutoCodeSherpa's execution trace, before being subjected to any refinement, and the *final* PBT which had been refined and passed the pipeline of AutoCodeSherpa, as shown in Table 5.6. Comparing the closed-weight LLM GPT-5-mini and the open-weight DeepSeek-V3.2, the first PBT correctness of the LLMs was different: 51.6% with GPT-5-mini, and 39.2% with DeepSeek-V3.2. From this lower accuracy, AutoCodeSherpa provided feedback to fix initially incorrect PBTs (Rectified in Table 5.6) while preventing the submission of incorrect PBTs (Discarded). Eventually, for

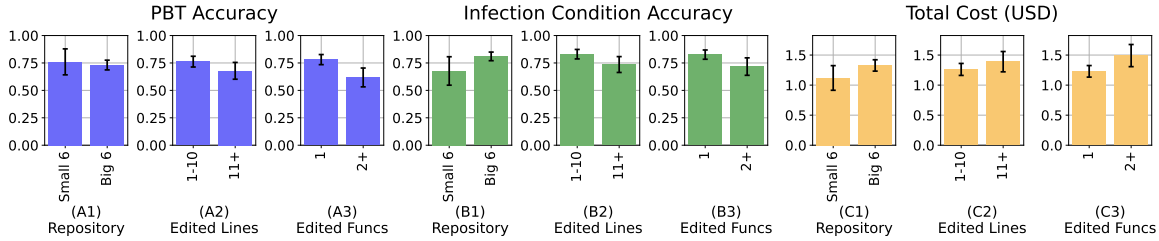


Figure 5.8: PBT and infection condition accuracy over different repositories and issue complexities. The bar charts are shown with 95% confidence intervals.

GPT-5-mini, 433 (=477-44) final PBTs were generated; among these, 246+83-2-(44-35)=318 were correct, where 44-35 corresponds to the number of correct first PBTs that were wrongly discarded, raising the correctness rate to 318/433 = 73.4%. For DeepSeek-V3.2, although the correctness of the first PBTs was lower than that of GPT-5-mini, the refinement process discarded as many as 125 incorrect first PBTs and rectified 76. As a result, there were 462-167=295 final PBTs, among which 462+76-2-(167-125)=213 were correct, raising the correctness rate to 213/295 = 72.2%, the same level as that of GPT-5-mini.

Meanwhile, all three models improved the plausible patch generation rate of Agentless. DeepSeek-V3.2 notably showed a strong improvement of 50.2%, similar to GPT-5-mini. However, Claude-3.5-Sonnet showed lower improvement than the other two models. This is due to two factors. First, Claude-3.5-Sonnet produced less accurate infection conditions than the other models as in Table 5.5, increasing the likelihood of explanations misguiding Agentless. Second, when converting symbolic explanations from AutoCodeSherpa into natural language, Claude-3.5-Sonnet's explanations were the shortest, at half the length of GPT-5-mini's explanations on average.

Robustness against LLM non-determinism We also reran AutoCodeSherpa to assess its robustness against LLM non-determinism. For cost reasons, we reran only GPT-5-mini and DeepSeek-V3.2. Table 5.7 compares the original and rerun results. For both LLMs, the rerun closely matches the original run across all metrics, indicating that AutoCodeSherpa can produce stable generation rates and explanation accuracy under LLM non-determinism.

Answer to RQ4: AutoCodeSherpa generalizes to different LLMs, generating symbolic explanations for all LLMs we evaluate. Furthermore, the refinement steps of AutoCodeSherpa ensure similar quality despite differences in LLM capability.

5.7.5 RQ5: Scalability across Repositories and Issue Complexity

In this research question, we evaluate how AutoCodeSherpa scales across repositories of different sizes and different issue complexities. Results are shown in Figure 5.8. We observe the following:

AutoCodeSherpa scales across repositories of different sizes. Subplots A1 and B1 of Figure 5.8 show the condition accuracies when running AutoCodeSherpa on issues from the smallest six and largest six repositories respectively. There was no significant relationship between repository size and accuracy. As the code exploration agent of AutoCodeSherpa retrieves relevant context from the repository, the impact of repository size on accuracy is limited.

AutoCodeSherpa is generally robust to issues of different complexity. The A2 and B2 subplots in Figure 5.8 show the condition accuracies when running AutoCodeSherpa on issues with small edits of 1-10 lines and large edits of 11+ changed lines. There is little difference in accuracy, indicating that AutoCodeSherpa can find accurate conditions even when the issue involves more buggy lines. Meanwhile, the A3 and B3 subplots of Figure 5.8 show the condition accuracies on issues on localized edits with 1 edited function and distributed edits with 2+ edited functions. The change in infection condition accuracy is not statistically significant. Analyzing the difference in PBT accuracy between issues with 1 and 2+ edited functions, we find the output condition is the primary culprit for the decline in accuracy: 75.9% of inaccurate PBTs have inaccurate output conditions. Issues where the developer changes many functions often have complex expected behavior which is not fully specified in the original issue report, leading to a decline in accuracy.

The cost of AutoCodeSherpa does not significantly change by repository or issue complexity. As the C1-C3 subplots in Figure 5.8 show, there is no statistically significant difference in cost observed in any category.

Answer to RQ5: AutoCodeSherpa scales to repositories of different sizes. AutoCodeSherpa is robust to issues requiring long patches.

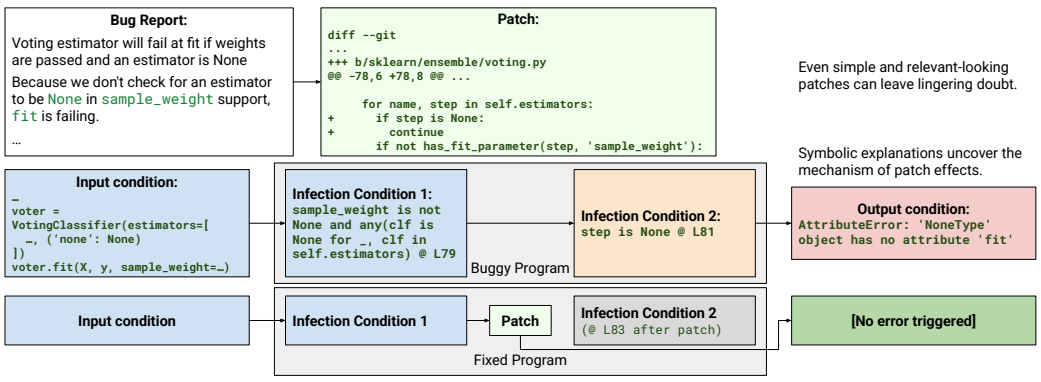


Figure 5.9: A schematic of a correct and helpful explanation.

5.8 Case Studies

To illustrate the symbolic explanations generated by AutoCodeSherpa, we discuss two examples, one helpful and one unhelpful, and analyze them in the context of their corresponding issues.

Helpful Example Figure 5.9 presents a schematic of the issue `scikit-learn-13779` from SWE-bench Verified and its symbolic explanation. The issue description, shown at the top-left, reports failure when the `estimator` parameter is `None`. A candidate patch, generated by SpecRover and displayed at the top, appears to handle the parameter correctly. However, in a real-world context, developers would likely need to inspect the code carefully to understand the patch's effects, as LLMs are known to produce plausible but incorrect patches [SGN+22].

On the bottom half of Figure 5.9, we illustrate how the symbolic explanation from AutoCodeSherpa can help understand both the issue and the patch. First, the input condition, shown on the left, accurately indicates that the issue arises when the `estimator` parameter is `None`. Second, infection condition 1, shown in the middle, further helps to understand the issue by indicating the values of key local variables in a buggy execution. Third, infection condition 2 is particularly helpful for understanding the patch by SpecRover: this condition indicates that the bug is triggered when the variable `step` is `None`, which is exactly the case handled by the patch. Finally, the PBT in the explanation can be executed to check automatically that the issue is resolved by the patch.

Unhelpful Example We present an unhelpful symbolic explanation generated for the issue `django-14539`. The issue description mentions that the `urlize` function has a bug. There are in fact two `urlize` functions in the `django` repository, and the bug report does not clarify which. In fact, only one of these two (defined in `html.py`) has real functionality; the second function (defined in `defaultfilters.py`) is simply calling the first. However, both the input condition and infection condition erroneously focus on the `defaultfilters.py` file. As a result, the symbolic explanation does not provide useful information about the root cause of the bug residing in the `html.py` file. This shows that our explanations can sometimes focus on correlated phenomena, rather than conducting an accurate analysis of the causal chain responsible for the bug.

5.9 Threats to Validity

While experimental results demonstrate the effectiveness of AutoCodeSherpa, we note the limitations of this study. Regarding threats to internal validity, in this study we performed manual analysis to assess the correctness of patches, which may contain mistakes. To mitigate this risk, the two annotators independently assessed the correctness of plausible

Table 5.7: Comparison of the original and rerun results of AutoCodeSherpa, showing its robustness.

Metric	GPT-5-mini		DeepSeek-V3.2	
	Original	Rerun	Original	Rerun
PBT Generation (%)	86.6	87.8	57.8	57.6
PBT Correctness (%)	73.4	76.5	72.2	73.6
Infection Cond. Generated (%)	66.7	66.5	46.0	50.3
Infection Cond. Correctness (%)	79.7	80.2	85.1	85.1
Patch Validation Precision (%)	48.5	48.7	51.5	50.0

patches, comparing them with the developer patch, and discussed to make a final decision. Meanwhile, there are threats to external validity: the results presented in this chapter may not generalize to other programming languages or bugs outside of SWE-bench Verified, and the Agentless performance improvement results may not generalize to other issue resolution techniques. Despite this, we note that the concept of bug characterization in the form of symbolic expressions, as well as its instantiation in the form of property-based tests and within-program first-order predicates, is general.

Another threat unique to LLM-based systems is the non-determinism of LLMs. This threat is reduced in three ways. First, we set the sampling temperature to 0 where applicable, which reduces output variance. Second, to empirically assess run-to-run stability, we reran the experiments with GPT-5-mini and DeepSeek-V3.2 and observed results similar to the original runs (Table 5.7). Finally, the refinements of AutoCodeSherpa filter out low-quality LLM outputs, as shown in Table 5.6. Beyond this, one could further reduce the variance of generated conditions by aggregating multiple samples through self-consistency [WWS+23, CAA+23], which we leave for future work.

5.10 Discussion

Validation of Hoare triples In Definition 1, we introduce the Hoare triples that the input, output, and infection conditions should satisfy. In the current methodology, we check the Hoare triples with a PBT engine, which performs a biased random search in the input space; this search is incomplete but often works well in practice. However, to guarantee the satisfaction of the Hoare triples, formal verification would be needed. We leave the formal verification of the Hoare triples on the large, real-world Python programs involved in our experiments as future work.

User study One important design goal of AutoCodeSherpa is to enable smoother human-agent collaboration, specifically for a developer to better understand the generated code and provide feedback to the coding agent. We have empirically shown that the understanding of the generated patch encoded in the symbolic explanation is useful for guiding a coding agent to generate a better patch. However, to fully examine this goal, a user study is needed to evaluate the developer experience with the symbolic explanation. For example, the user study can compare the time usage of developers in manually debugging the issue with and without the symbolic explanation.

Chapter 6

Related Work

This chapter surveys prior work through the central concern of this thesis: how to obtain and strengthen oracles for the use of automatic programming systems when formal specifications are unavailable. The survey starts with test-based automated program repair, followed by LLM-based code generation and specification inference, and then coding agents that can work at the repository scale. Finally, it surveys work related to executable explanations in agentic programming.

6.1 Automatic Program Repair and Test-Based Specifications

This first group of related work puts EvoRepair in perspective. It establishes the pre-LLM setting of the thesis: tests are the available but incomplete specification, so rather than merely search for a plausible patch, the repair process should also actively enhance the test suite.

Test-based automated program repair (APR) [LGPR19, Mon17] is a well studied research area in software engineering. Given a buggy program P, and a test-suite T, automated program repair attempts to (minimally) modify P to a program P' which passes the given test-suite T. APR techniques involve metaheuristic search [WNGF09], semantic analysis [NQRC13], machine learning [LR16], or a combination of different techniques. APR can also be used to rectify automatically generated code from LLMs, see e.g. [FGM+23].

The related work includes automated coevolutionary searches, especially for tests and patches, testing and repair (in particular, the approaches that leverage evolutionary searches), and test generation for patch testing.

6.1.1 Coevolution of Tests and Patches

Most related to EvoRepair is the approach by Arcuri and Yao [AY08]. It takes a formal specification and a buggy program as input and co-evolves test cases and programs using genetic programming with the goal of repairing the buggy program. The formal specification allows them to generate as many unit tests as they want, which is an essential part of their technique. Moreover, the designed fitness function that drives the evolution is derived from

the formal specification. With the general lack of formal specifications in practice, EvoRepair instead focuses on available bug reports, extracts generally valid constraints on the expected behavior, and uses them as test oracles. The work of Fix2Fit [GMR19] generates new tests (to prune overfitting patches) but does not evolve existing ones. EvoRepair is distinct in that it evolves the test and the patch pool by reusing the mutation-based search in EvoSuite. In the past, EvoSuite has been used to perform related adaptations, e.g., to evolve assertions [TJTP20, JCHT18]. Contrary to these works, EvoRepair does not invalidate the oracles. Instead, it uses the oracles for invalidating and evolving patches.

Concolic Program Repair (CPR) [SNGR21] proposes generating new tests to prune overfitting patches. CPR still needs a lightweight, user-provided constraint to reason about the generated patches. CPR does not mutate the patches after test generation; their proposed refinement only removes overfitting patches from the patch pool. The limitation in CPR is the assumption of having the correct patch in the patch pool, whereby the correct patch can be identified after the removal of overfitting patches. EvoRepair makes no such assumptions.

6.1.2 Evolutionary Repair

EvoRepair is most related to evolutionary repair algorithms such as GenProg [WNGF09] that operate under the redundancy assumption, also known as the plastic surgery hypothesis [BBD+14]. The assumption is that the repair ingredients can be extracted from elsewhere in the buggy program itself. Empirical studies [DMMA19] show that this assumption does not hold true for most of the bugs in the popular Defects4J benchmark [JJE14]. Repair techniques that restrict the search space to the buggy program itself have not been able to successfully find a plausible patch for many of the bugs in the considered benchmarks. ARJA-e [YB20b] is a more recent study on more fine-grained changes that extends the search space beyond the redundancy assumption, allowing it to fix a much higher number of bugs in the Defects4J benchmark. Similar to SimFix [JXZ+18], it combines the search space with repair templates [LKKB19, KNSK13] and extends the redundancy assumption by allowing more fine-grained modifications to AST node elements, thereby expanding the search space. Modifying AST node elements by replacing them with similar alternatives provides ARJA-e the capability to generate new code, which allows fixing bugs previously not fixed by its predecessors ARJA [YB20a] and GenProg [WNGF09]. Since ARJA-e represents the state of the art in evolutionary repair, EvoRepair builds on the patch enumeration in ARJA-e. Note that approaches like VarFix [WSKLG21], which exploit similarities between test executions to speed up the exploration, are complementary to EvoRepair.

6.1.3 Evolutionary Testing

EvoRepair builds on EvoSuite [FA11], the state-of-the-art evolutionary framework for whole test suite generation. Driven by coverage metrics, the test generation thereby handles test data instantiation, method call sequences, and retrieval of regression oracles via mutation testing.

In the program repair context, EvoRepair has no access to a reference solution and hence cannot use the regression oracles generated by EvoSuite. Instead, it has to generate its own test oracles. EvoSuite interfaces with numerous genetic algorithms such as MOSA [PKT15], DynaMOSA [PKT18], and NSGA-II [DPAM02]. EvoRepair also incorporates these well-researched search strategies for multiple objectives in each domain.

6.1.4 Test Generation for Patch Testing

Existing works in patch testing aim at alleviating the overfitting issue [SNGR21, GMR19, YMD+19]. For example, CPR [SNGR21] co-explores tests and abstract patches with the goal of alleviating overfitting patches. It generates new test cases with a lightweight, user-provided constraint as a test oracle. However, CPR does not mutate the patches after test generation but simply refines the abstract patches to exclude patches that violate the test oracle. Fix2Fit [GMR19] identifies overfitting patches by using grey-box fuzzing to generate new tests and using crash-freedom as an oracle. It is a post-processing technique that can enhance any APR technique. Differently, UnsatGuided [YMD+19] targets input data of synthesis-based repair techniques like Nopol [XMD+17] and SemFix [NQRC13]. It aims to strengthen repair constraints by generating additional tests that can supplement the developer test suite. They show that their approach can indeed help to prune patches that are overfitting because of regression-introducing edits. However, they also acknowledged that due to the oracle problem, UnsatGuided is unsuccessful in pruning other kinds of overfitting patches. Other works in patch testing focus on the identification of regression errors [SFMO13, PKC16, NPB+20]. In contrast to these techniques, EvoRepair evolves the test pool as well as the patch pool via re-use of the mutation-based search in EvoSuite, and goes beyond regression errors with test oracles derived from bug reports. EvoRepair further integrates test generation as an integral element of the coevolution, instead of having it as a pre- or post-process of APR.

6.2 LLM-Based Code Generation and Program Selection

The test-based APR literature above treats tests as the primary specification, and the main difficulty is that this specification is incomplete. LLM-based code generation relieves the difficulty, because an LLM can use natural language as the specification, which often carries more information about the program's intended behavior than a few test cases.

However, compared to test-based APR, the challenge in LLM-based code generation is that it is difficult to validate the generated program against a natural-language specification. Therefore, there is one line of work focusing on selecting the correct program from multiple LLM-generated candidates, mostly based on self-consistency of the LLM. In contrast, LLMCodeChoice leverages differential testing to distinguish between candidates first, making the selection process more accurate and cost-efficient.

Code Generation Code generation (program synthesis) is the task of automatically generating programs that satisfy some form of user specification. In existing works, the specification can be a logical formula [ABJ+13] or input-output examples [GHS12, OZ15]. In recent years, neural networks have been trained to generate programs from a natural language description of the desired program [SZM+19, YN17]. Most recently, a large number of LLMs have been developed, e.g., Codex [CTJ+21], AlphaCode [LCC+22], ChatGPT [Ope22], and GPT-4 [Ope23]. They have been pretrained on a massive amount of data and can generalize to various tasks given an appropriate prompt, without fine-tuning. In particular, LLMs go beyond previous techniques by generating programs from natural language descriptions. LLMCodeChoice takes advantage of ChatGPT to generate initial program samples.

Prompting for Code Generation A widely used prompting technique for code generation is the few-shot prompting introduced with GPT-3 [BMR+20]. In a few-shot prompt, the actual problem to be solved is preceded by several example question-answer pairs. These examples result in improved quality of generated code. Several other prompting techniques have been inspired by few-shot prompting, e.g., chain-of-thought [WWS+22] and structured chain-of-thought [LLLJ25]. LLMCodeChoice studies the problem of selecting correct LLM-generated code, instead of generating code. The selection technique can be applied on top of programs generated with various prompting techniques, to enhance user trust in the generated programs.

Program Differentiation An important part of program selection is to expose the semantic differences between programs. One way to do so is *differential testing*, which aims to generate concrete inputs on which two programs exhibit different behavior (distinguishing inputs). The inputs can be generated via a biased random search under the guidance of some feedback (e.g., program coverage [CSS+16]) or generated by symbolic execution [PDEP08]. Besides differential testing, *regression verification* [BPRT13, BALR20] can also be used to expose the inequivalence between programs. LLMCodeChoice uses differential testing for program differentiation, because the generated distinguishing inputs enable the LLM to be used more effectively as a test oracle.

Program Selection from LLMs LLMs are non-deterministic [OZHW23], and many generation attempts may result in the generation of incorrect programs. To tackle this issue, several techniques have been proposed to select the correct program from multiple samples. AlphaCode [LCC+22] divides programs into clusters based on program output on a set of LLM-generated test inputs. Programs with the same outputs are put in the same cluster, and larger clusters are prioritized for selection. Speculyzer [KLE22] and CodeT [CZN+23] perform a similar clustering, where programs passing the same set of LLM-generated tests are clustered. Apart from the cluster size, Speculyzer and CodeT also take into account the number of passed test cases when doing selection. LLMCodeChoice clusters programs

by differential testing. This produces distinguishing inputs that better expose differences between programs than LLM-generated test cases. Moreover, instead of selecting the largest cluster, it selects the cluster that the LLM chooses as an oracle. The selection literature therefore makes the validation challenge explicit: after candidates have been generated, the system still needs evidence for choosing one behavioral cluster over another.

Repair of Code from LLMs Another possible mitigation to the lack of correctness guarantee is to repair LLM-generated code. The repair process can be aided with different artifacts, e.g., LLM-generated feedback on the code [MTG+23] and error information produced by code execution [JWW23, CLSZ24, FGM+23]. LLMCodeChoice focuses on program selection, which complements repair of LLM-generated code.

6.3 LLM for Specification Inference

Apart from code generation, the LLM itself can also infer specifications. The specification-inference capability appears in both LLMCodeChoice and SpecRover. Related to LLMCodeChoice are the works that generate tests with an LLM, and related to SpecRover are the works that infer formal specifications with an LLM. This section positions LLMCodeChoice and SpecRover relative to these works.

Test Generation with LLMs AthenaTest [TDS+20] generates unit tests for Java programs with a transformer. It uses the program-under-test as the oracle. ChatUniTest [XCZ+23] prompts ChatGPT to generate Java unit tests with context information, including method name, class name, etc. ChatTester [YLD+24] queries ChatGPT about the "intention" of the method-under-test, which is then used to prompt ChatGPT to generate tests. CodaMosa [LILS23] improves coverage of search-based testing by leveraging an LLM to generate example test cases for under-covered functions. Fuzz4All [XPT+24] automatically prompts LLMs to generate fuzzing inputs. Contrary to these works, LLMCodeChoice does not use LLMs to generate tests. It uses the LLM as partial guidance to infer the intended behavior of test inputs generated via differential testing.

Function-Level Specification Inference Previous works have studied function-level specification inference by means of LLMs [MSW+24, MLL+25, EFCL24]. However, these works either focus on generating specifications for simple one-function or one-class programs [MLL+25, MSW+24], or assume that a target function is provided [EFCL24]. In contrast, SpecRover targets large programs and does not assume the target functions to generate specifications for are given.

6.4 Repository-Level Coding Agents

The preceding works show that LLMs can produce specifications at the level of unit tests or individual functions. Coding agents expand the capability of LLMs to the repository

scale. Additionally, coding agents have to tackle a new challenge: a natural-language issue provides only global intent, while the agent must recover local intent across a codebase. This is the setting of SpecRover, where inferred specifications guide patch generation and reviewer agents judge patches in the presence of unreliable generated tests.

6.4.1 Coding Agents

Coding agents are autonomous software systems that allow LLMs to invoke tools to perform software engineering (SE) tasks. Early examples include SWE-agent [YJW+24] and AutoCodeRover [ZRFR24], which focus on resolving software issues. The capability of SE agents has expanded to test generation [MMHV24], bug finding [ZWL+25], and more [AZL+26]. To tackle these tasks, SE agents typically make use of program analysis tools such as code search [ZRFR24], code edit, test execution, and command-line execution [YJW+24]. Some agentic systems can employ multiple agents to collaborate on a complex task, with each agent specializing in one part or one step in the task, e.g., SpecRover (Chapter 4).

6.4.2 Patch Review under Unreliable Tests and Patches

Since both tests and patches generated in an autonomous workflow can be unreliable, SpecRover additionally considers the natural-language issue description when judging their correctness. This judgment is implemented as the reviewer agent. Compared to the previous works that discover additional specifications based on user intent [FNS+24], SpecRover does not rely on interactive user feedback. Instead, it utilizes the high-level natural-language description as additional feedback. Furthermore, compared to previous works that evaluate the generated code with an LLM-based reviewer [ZYH+23], the reviewer agent in SpecRover is designed for the setup where both natural-language instructions and unreliable tests are present.

6.4.3 Code Intent Extraction via LLMs

Among LLM-based approaches for solving software issues, SpecRover is thematically closest to the work of AutoCodeRover [ZRFR24]. Like AutoCodeRover, SpecRover takes the position that program modifications like bug fixing are best aided by inference of the developer intent. AutoCodeRover infers the developer intent only from the software project structure. In contrast, SpecRover is more general and is capable of inferring specifications from different sources including program structure, program behavior, tests and so on. Furthermore, SpecRover focuses on giving an explanation of the produced patches.

6.5 Symbolic Explanations in Coding Agents

The specification can guide agents in code generation. An equally important question is what evidence, or explanation, the agent can return, for the generated code to be trusted. This final group of related work positions AutoCodeSherpa: it moves from using specifications

to guide or judge code toward producing executable, symbolic explanations that make the agent's reasoning checkable.

6.5.1 Issue Reproduction

One line of work closely related to AutoCodeSherpa is issue reproduction, which aims to write a test case to reproduce a given issue. Since the symbolic explanation in AutoCodeSherpa is executable, it is akin to a reproducer test. However, unlike existing works on issue reproduction [KYY23, WGM+24, MMHV24], which only try to find example-based reproducing tests, AutoCodeSherpa's explanation concisely represents a large, potentially infinite number of buggy program executions with symbolic expressions. In other words, AutoCodeSherpa's symbolic explanation is unique in that it *generalizes* from the issue description.

6.5.2 Input Conditions

No existing techniques appear to jointly generate input, infection, and output conditions in a manner similar to AutoCodeSherpa. However, there have been efforts to characterize the input space. Avicenna [ESS+23] seeks to identify and explain the inputs that cause a fault. It is limited to inputs conforming to a predefined grammar. This is unlike AutoCodeSherpa, which uses the expressiveness of PBTs to represent general input and output conditions. As most tasks in the benchmark involve the construction and use of complex objects, direct comparison with Avicenna would be difficult. Daikon [ECGN99] generates invariants for program states, but requires these invariants to fit pre-defined property templates.

6.5.3 Automatic Bug Explanation

Prior research shows developers are skeptical of automatically generated patches provided without evidence and that they seek explanations for the root cause of a bug [KXLL16, NSGR22]. This observation has prompted researchers to propose techniques that explain bugs. Bugsplainer [MSR23] uses a neural machine translation technique to generate natural language explanations for specific software lines. AutoSD [KCYL25] uses LLMs to follow the scientific process to generate and validate hypotheses about the bug. SpecRover (Chapter 4) analyzes the intended behavior for software components and generates a natural language explanation based on these findings. In contrast to all such approaches, AutoCodeSherpa generates a *symbolic* bug explanation, which is executable and therefore can be automatically evaluated.

Chapter 7

Conclusion

In this thesis, we have approached the recurring problem of insufficient specifications in automatic programming through four studies.

7.1 Summary

In the pre-LLM era, using test cases for specification results in patch overfitting. EvoRepair presents a practical solution with the coevolution of patches and tests. We addressed the oracle problem in test generation not by assuming a formal specification but by considering which information and requirements a user and bug reporter can provide. Therefore, we extracted test oracles from bug reports in Defects4J. We implemented the proposed workflow in our tool EvoRepair as an extension of EvoSuite, a well-known tool for test generation. We hope that our practical assumption of test oracles and the integration into EvoSuite lower the entrance barrier for software developers to apply APR in practice. Furthermore, the coevolution concept is not restricted to EvoSuite and can be ported to other patch and test generators. Our evaluation, including the comparison with the state-of-the-art techniques, shows that EvoRepair is highly competitive and that the coevolution improves the patch and test quality.

Moving from the pre-LLM setting to LLM-driven code generation, the specification becomes a natural-language description and the question becomes how to derive an oracle that better discriminates between candidate programs from this ambiguous specification. In LLMCodeChoice, we propose to improve the trustworthiness of LLM-generated programs with a new perspective of retrieving program specifications from natural language descriptions. LLMCodeChoice curates distinguishing inputs to reveal differences between LLM-generated programs and further utilizes the LLM to select the correct output for those distinguishing inputs from all possibilities. Our evaluation on the generation of standalone programs within HumanEval and MBPP demonstrates the effectiveness of LLMCodeChoice.

Additionally, LLMCodeChoice points to two broader perspectives. First, semantic-analysis-based test generation for LLM-generated programs is important: the byproducts from semantic analysis, in our case the distinguishing tests, can potentially serve as evidence or an

explanation of "correctness" of LLM-generated programs to enhance developers' trust when using AI programming assistants. Second, automated program specification generation from LLMs is also promising: despite not being perfect, they can be valuable hints to alert users of AI programming assistants about potential untrustworthiness in the automatically generated programs, and they can support low-effort, human-assisted gradual oracle inference.

In the agentic era, where coding agents are capable of improving code at repository scale, SpecRover studies how LLM agents can produce patches with evidence based on a global specification. SpecRover combines specification inference from multiple sources with a reviewer agent that vets candidate patches against the issue description, inferred intent, and available tests. In this way, the system is designed not merely to maximize patch generation efficacy, but to improve the quality and interpretability of the suggested fixes. Owing to the growth of LLM-based automatic programming, there is increasing interest in autonomous program improvement technologies, and SpecRover reflects the perspective that such systems should aim to produce patches that developers can accept with confidence and understand through accompanying explanations. Looking ahead, we expect that LLM agents will need to improve both precision and recall, while also strengthening their vetting of generated patches through richer test generation and more reliable evidence. This suggests evaluating future agents not only by efficacy and cost, but also by the signal-to-noise ratio of their outputs from the viewpoint of the developer who must decide what to trust.

Despite their capabilities, however, human oversight and trust-building are still required for successful deployment of SE agents. To that end, AutoCodeSherpa investigates the promise of symbolic explanations of issue reports. By producing these symbolic explanations, we enhance developer understanding, improve the quality of output produced by other agents, and most importantly provide evidence of correctness for code and patches generated by agents. Because these explanations are executable, they help filter out incorrect patches produced by agents and thereby improve the quality of automatically generated code. As SE agents become more commonplace, and coding undergoes further automation, we expect this line of work to contribute to mechanisms for enhancing trust in auto-coding. In this sense, it supports the broader vision of *trusted automatic programming* and its integration into future workflows.

Looking further ahead, as LLM agents generate code with ever increasing speed and quality, specifications will likely take a more central role in the software development workflow. In such a workflow, human developers communicate a specification, most often an incomplete one, to an LLM agent in natural language, while the LLM agent communicates the actual specification underlying the generated code, or in other words an explanation, back to the human developer. This explanation can take various forms, including natural language, executable tests, or a symbolic formula. It enables human developers to understand, trust, and govern the generated code, and to collaborate more effectively with the LLM agent. From this perspective, specifications and explanations go beyond being merely a way to

improve generated code or an afterthought used to justify it. Instead, they become the main interface through which LLM agents are used and governed.

7.2 Future Work

7.2.1 Seeding Differential Testing with LLM-Generated Inputs

Open question. Differential testing is a critical step in the workflow of Chapter 3. In differential testing, two systems-under-test are executed with the same input to reveal differences between their outputs. Previous works on seeding differential testing with LLM-generated inputs [EMML+25, DXP+23] focus on satisfying complex syntax and semantics constraints of the inputs, e.g., when the input is a C program [EMML+25] or a deep learning program [DXP+23]. Beyond seeds satisfying the constraints, there is a lack of studies on the use of LLM-generated seeds for the purpose of revealing differences between outputs.

Possible approach. Given two complex systems-under-test, the LLM can be used to generate inputs that are more likely to reveal differences between outputs, with its understanding of the intent of the systems. The inputs can serve as seeds that are further mutated to reveal more discrepancies.

Success measure. To evaluate the effectiveness of the approach, we can measure the ratio of seeds that reveal a discrepancy directly, and the average number of discrepancies revealed by all the mutants of a seed. The baseline can be random seeds.

7.2.2 Resolving Ambiguities Revealed by Conflicting Candidate Outputs

Open question. In Chapter 3, we observe that the distinguishing input-output examples for LLM-generated programs can reveal ambiguities in the natural-language description of the desired program. In order for the correct program to be selected in this case, the ambiguity needs to be resolved by a human developer.

Possible approach. In practice, a developer can examine the involved input-output examples and the problem description to resolve the ambiguity. Furthermore, they can enrich the problem description to remove the ambiguity.

Success measure. To evaluate this workflow, a user study can be conducted, where the developers try to resolve the ambiguities revealed by the conflicting candidate outputs and to enrich the problem description. We can measure the time usage of developer resolution and whether ambiguity still remains after the enrichment.

7.2.3 Patch Correctness Estimation

Open question. In Chapter 4, we observe that coding agents should have a high signal-to-noise ratio to reduce cognitive load on developers. While SpecRover successfully improves the signal-to-noise ratio by employing a reviewer agent that reconciles different specifications, the reviewer agent's decision is still a coarse-grained binary signal. In this case, if there are a large number of issues with a patch accepted by the coding agent itself, it is difficult for a developer to prioritize among the issues. Ideally, there should be a more fine-grained signal, e.g., an estimate of the probability that the patch is correct. The estimation can be used to prioritize issues in different ways, e.g., automatically approving the ones with high confidence value and manually examining the ones with low confidence value, or manually examining every issue from high to low confidence. The most relevant work to this direction is SWE-Gym [PWN+25], which fine-tunes an LLM to label trajectories of a coding agent as successful or unsuccessful; a continuous signal is then obtained by extracting the internal log probability for both labels. There are also works on general agent trajectory calibration [ZXW26, KBB+26], whose goal is to estimate the likelihood that an agent's trajectory will succeed. The caveat of these works is that they do not explicitly analyze software-engineering-specific evidence, such as patch location and test coverage.

Possible approach. Software-engineering-specific evidence can be extracted from the trajectory as features for training a model. For example, the agent may have identified multiple potential patch locations; whether the final patch location was decided with or without evaluating the alternatives may provide a useful signal. For another example, the strength of the reproducer test may also help the estimate, and one can check the rigor by doing additional analysis like mutation testing. Other possible signals include the agent's action patterns, i.e., alternation among inspecting code, writing a patch, running a test case, etc. A predictor model can be trained on these signals to estimate the likelihood of the patch being correct.

Success measure. The effectiveness of the approach can be measured with three standard metrics: Expected Calibration Error (ECE) [GPSW17], which measures the agreement between the estimates and actual success rates; the Brier Score (BS), a strictly proper scoring rule assessing probabilistic predictions in terms of both calibration and discrimination; and AUROC, which measures the ability to distinguish between successful and failed trajectories.

7.2.4 Executable Explanations in Patch Review

Open question. In Chapter 5, a design goal of AutoCodeSherpa is to help developers understand the agent-generated patches better. However, to validate this goal, a user study would be needed.

Possible approach. In the user study, we can present developers with a natural-language description of the issue, an agent-generated patch, and with or without an executable explanation of the patch generated by AutoCodeSherpa. The developer can then decide whether to accept the patch.

Success measure. We can measure the time usage and the accuracy of patch-acceptance decisions by the developer, with and without the executable explanations.

References

[ABC+13] Saswat Anand, Edmund K. Burke, Tsong Yueh Chen, John Clark, Myra B. Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, Phil McMinn, Antonia Bertolino, J. Jenny Li, and Hong Zhu. An orchestrated survey of methodologies for automated software test case generation. *Journal of Systems and Software*, 86(8):1978–2001, 2013.

[ABJ+13] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo MK Martin, Mukund Raghothaman, Sanjit A Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*, pages 1–8, 2013.

[AHM06] John Anvik, Lyndon Hiew, and Gail C. Murphy. Who should fix this bug? In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 361–370. Association for Computing Machinery, 2006.

[AHNS15] Thomas Arts, John Hughes, Ulf Norell, and Hans Svensson. Testing AUTOSAR software with QuickCheck. In *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pages 1–4, 2015.

[AL23] Abdulaziz Alaboudi and Thomas D LaToza. What constitutes debugging? An exploratory study of debugging episodes. *Empirical Software Engineering*, 28(5):117, 2023.

[Ama23] Amazon Web Services. Amazon CodeWhisperer. https://aws.amazon.com/codewhisperer/, 2023.

[AO16] Paul Ammann and Jeff Offutt. *Introduction to software testing*. Cambridge University Press, 2016.

[AON+21] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[App24] AppMap. AppMap Navie. https://appmap.io/product/appmap-navie, 2024.

[ARU17] Rajeev Alur, Arjun Radhakrishna, and Abhishek Udupa. Scaling enumerative program synthesis via divide and conquer. In *International conference on tools and algorithms for the construction and analysis of systems*, pages 319–336, 2017.

[Aut24] AutoCodeRoverSG. AutoCodeRoverSG/auto-code-rover. https://github.com/AutoCodeRoverSG/auto-code-rover, 2024.

[AY08] Andrea Arcuri and Xin Yao. A novel co-evolutionary approach to automatic software bug fixing. In *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*, pages 162–168, 2008.

[AZL+26] L. Applis, Y. Zhang, S. Liang, N. Jiang, L. Tan, and A. Roychoudhury. Unified software engineering agent as AI software engineer. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026.

[BALR20] Sahar Badihi, Faridah Akinotcho, Yi Li, and Julia Rubin. ARDiff: scaling program equivalence checking via iterative abstraction and refinement of common code. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 13–24, 2020.

[BB09] Adrian Bachmann and Abraham Bernstein. Software process data quality and characteristics: a historical view on open and closed source projects. In *Proceedings of the joint international and annual ERCIM workshops on Principles of software evolution (IWPSE) and software evolution (Evol) workshops*, pages 119–128, 2009.

[BBD+14] Earl T. Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. The plastic surgery hypothesis. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 306–317. Association for Computing Machinery, 2014.

[BGE+21] Arianna Blasi, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Antonio Carzaniga. MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation. *Journal of Systems and Software*, 181:111041, 2021.

[BGK+18] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 242–253. Association for Computing Machinery, 2018.

[BHM+15] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[BJA+21] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. Using lightweight formal methods to validate a key-value storage node in Amazon S3. *Amazon Science*, 2021.

[BKPW23] Rishabh Balse, Viraj Kumar, Prajish Prasad, and Jayakrishnan Madathil Warriem. Evaluating the quality of LLM-generated explanations for logical errors in CS1 student programs. In *Proceedings of the 16th Annual ACM India Compute Conference*, pages 49–54. Association for Computing Machinery, 2023.

[BMR+20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1877–1901, 2020.

[BPRT13] John Backes, Suzette Person, Neha Rungta, and Oksana Tkachuk. Regression verification using impact summaries. In *Proceedings of the International SPIN Symposium on Model Checking of Software (SPIN)*, pages 99–116, 2013.

[BSSZ18] Moritz Beller, Niels Spruit, Diomidis Spinellis, and Andy Zaidman. On the dichotomy of debugging behavior among programmers. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 572–583, 2018.

[CAA+23] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311*, 2023.

[CAS+24] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/, 2024.

[CH00] Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *SIGPLAN Not.*, 35(9):268–279, 2000.

[CLSZ24] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

[Cog24] Cognition Labs. Introducing Devin, the first AI software engineer. https://www.cognition-labs.com/introducing-devin, 2024.

[CRPA12] José Campos, André Riboira, Alexandre Perez, and Rui Abreu. GZoltar: an Eclipse plug-in for testing and debugging. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 378–381. ACM, 2012.

[CSS+16] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. Coverage-directed differential testing of JVM implementations. In *proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 85–99, 2016.

[CTJ+21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[CZN+23] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[DMMA19] Thomas Durieux, Fernanda Madeiral, Matias Martinez, and Rui Abreu. Empirical review of Java program repair tools: A large-scale experiment on 2,141 bugs and 23,551 repair attempts. In *Proceedings of the ACM Joint*

European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pages 302–313. ACM, 2019.

[dot25] Comment on dotnet/runtime pull request #115733. https://github.com/dotnet/runtime/pull/115733#issuecomment-2892088377, 2025.

[DPAM02] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[DPJ24] Hang Du, Vijay Krishna Palepu, and James A Jones. Ripples of a mutation—An empirical study of propagation effects in mutation testing. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1–13, 2024.

[DRML22] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. TOGA: A neural method for test oracle generation. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2130–2141. Association for Computing Machinery, 2022.

[DW10] Vidroha Debroy and W. Eric Wong. Using mutation to automatically suggest fixes for faulty programs. In *2010 Third International Conference on Software Testing, Verification and Validation*, pages 65–74, 2010.

[DXP+23] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, pages 423–435, 2023.

[ECGN99] Michael D Ernst, Jake Cockrell, William G Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 213–224, 1999.

[EFCL24] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proc. ACM Softw. Eng.*, 1(FSE), 2024.

[EMML+25] Karine Even-Mendoza, Héctor D Menéndez, William B Langdon, Aidan Dakhama, Justyna Petke, and Bobby R Bruce. Search+ LLM-based testing for ARM simulators. In *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 469–480, 2025.

[ESS+23] Martin Eberlein, Marius Smytzek, Dominic Steinhöfel, Lars Grunske, and Andreas Zeller. Semantic debugging. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 438–449, 2023.

[FA11] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 416–419. Association for Computing Machinery, 2011.

[FA15] Gordon Fraser and Andrea Arcuri. Achieving scalable mutation-based generation of whole test suites. *Empirical Software Engineering*, 20(3):783–812, 2015.

[FGM+23] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1469–1481. IEEE, 2023.

[FNS+24] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Trans. Software Eng.*, 50(9):2254–2268, 2024.

[Gau24] Paul Gauthier. Aider. https://github.com/paul-gauthier/aider, 2024.

[GCD+24] Harrison Goldstein, Joseph W Cutler, Daniel Dickstein, Benjamin C Pierce, and Andrew Head. Property-based testing in practice. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1–13, 2024.

[GHS12] Sumit Gulwani, William R. Harris, and Rishabh Singh. Spreadsheet data manipulation using examples. *Communications of the ACM*, 55(8):97–105, 2012.

[Gil24] Kevin Gilpin. How AppMap Navie solved the SWE bench AI coding challenge. https://dev.to/appmap/how-appmap-navie-solved-the-swe-bench-ai-coding-challenge-20an, 2024.

[Git23] GitHub. GitHub Copilot. https://github.com/features/copilot, 2023.

[GM22] Ali Ghanbari and Andrian Marcus. Patch correctness assessment in automated program repair based on the impact of patches on production and test code. In *Proceedings of the ACM SIGSOFT International Symposium on*

Software Testing and Analysis (ISSTA), pages 654–665. Association for Computing Machinery, 2022.

[GMR19] Xiang Gao, Sergey Mechtaev, and Abhik Roychoudhury. Crash-avoiding program repair. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 8–18. ACM, 2019.

[GPSW17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330, 2017.

[Gul11] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

[Hen08] Joerg Henning. MATH-227 bug, 2008. https://issues.apache.org/jira/browse/MATH-227.

[Hol20] Paul Holser. JUnit-QuickCheck: Property-based testing, JUnit-style. https://pholser.github.io/junit-quickcheck/site/1.0/, 2020.

[HPAN16] John Hughes, Benjamin C. Pierce, Thomas Arts, and Ulf Norell. Mysteries of DropBox: Property-based testing of a distributed synchronization service. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 135–145, 2016.

[IVTJ22] Ali Reza Ibrahimzada, Yigit Varli, Dilara Tekinoglu, and Reyhaneh Jabbarvand. Perfect is the enemy of test oracle. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 70–81. Association for Computing Machinery, 2022.

[JCHT18] Gunel Jahangirova, David Clark, Mark Harman, and Paolo Tonella. OASIs: Oracle assessment and improvement tool. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 368–371. Association for Computing Machinery, 2018.

[JEF14] René Just, Michael D Ernst, and Gordon Fraser. Efficient mutation analysis by propagating and partitioning infected execution states. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 315–326, 2014.

[JGST10] Susmit Jha, Sumit Gulwani, Sanjit A Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 215–224, 2010.

[JH11] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[JJE14] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440. Association for Computing Machinery, 2014.

[JWW23] Shuyang Jiang, Yuhao Wang, and Yu Wang. SelfEvolve: A code evolution framework via large language models. *arXiv preprint arXiv:2306.02907*, 2023.

[JXZ+18] Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. Shaping program repair space with existing patches and similar code. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 298–309. Association for Computing Machinery, 2018.

[JYW+24] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[KBB+26] Brendan King, Farima Fatahi Bayat, Jean-Flavien Bussotti, Pouya Pezeshkpour, and Estevam Hruschka. Confidence reasoning graphs: Structured confidence estimation for LLM agents. *arXiv preprint arXiv:2610.07948*, 2026.

[KCYL25] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering*, 30(2):1–28, 2025.

[KLB+19] Anil Koyuncu, Kui Liu, Tegawendé F. Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. iFixR: Bug report driven program repair. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 314–325. Association for Computing Machinery, 2019.

[KLE22] Darren Key, Wen-Ding Li, and Kevin Ellis. I speak, you verify: Toward trustworthy neural program synthesis. *arXiv preprint arXiv:2210.00848*, 2022.

[KNSK13] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. Automatic patch generation learned from human-written patches. In *Proceedings of the*

IEEE/ACM International Conference on Software Engineering (ICSE), pages 802–811, 2013.

[KXLL16] Pavneet Singh Kochhar, Xin Xia, David Lo, and Shanping Li. Practitioners' expectations on automated fault localization. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 165–176. Association for Computing Machinery, 2016.

[KYY23] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2312–2323. IEEE, 2023.

[LAD+22] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3843–3857, 2022.

[LCC+22] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[LGPR19] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019.

[LILS23] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023.

[LKCSL17] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. QuixBugs: A multi-lingual program repair benchmark set based on the Quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, pages 55–56. Association for Computing Machinery, 2017.

[LKKB19] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. TBar: Revisiting template-based automated program repair. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 31–42. Association for Computing Machinery, 2019.

[LLLJ25] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. Structured chain-of-thought prompting for code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(2):37:1–37:23, 2025.

[LR16] Fan Long and Martin C. Rinard. Automatic patch generation by learning correct code. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 298–312. ACM, 2016.

[LXWZ23] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[Mas87] Henry Massalin. Superoptimizer: a look at the smallest program. *ACM SIGARCH Computer Architecture News*, 15(5):122–126, 1987.

[MB23] Manish Motwani and Yuriy Brun. Better automatic program repair by using bug reports and tests together. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

[McM04] Phil McMinn. Search-based software test data generation: a survey. *Software Testing, Verification and Reliability*, 14(2):105–156, 2004.

[MHD19] David Maciver and Zac Hatfield-Dodds. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4:1891, 2019.

[MLL+25] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 16–28. IEEE, 2025.

[MM16] Matias Martinez and Martin Monperrus. Astor: A program repair library for Java (demo). In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 441–444. Association for Computing Machinery, 2016.

[MM18] Matias Martinez and Martin Monperrus. Ultra-large repair search space with automatically mined templates: The Cardumen mode of Astor. In *Proceedings of the International Symposium on Search-Based Software Engineering (SSBSE)*, volume 11036, pages 65–86. Springer, 2018.

[MMHV24] Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[MMHV25] Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. SWT-bench: Can your model write reproduction tests for real-world issues? https://swtbench.com/?results=verified, 2025.

[Mon17] Martin Monperrus. Automated software repair: a bibliography. *ACM Computing Surveys*, 51, 2017.

[MS22] Inbal Magar and Roy Schwartz. Data contamination: From memorization to exploitation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 157–165. Association for Computational Linguistics, 2022.

[MSR23] Parvez Mahbub, Ohiduzzaman Shuvo, and Mohammad Masudur Rahman. Explaining software bugs leveraging code structures in neural machine translation. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 640–652. IEEE Press, 2023.

[MSW+24] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proc. ACM Softw. Eng.*, 1(FSE):2332–2354, 2024.

[MTG+23] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[MUMM19] F. Madeiral, S. Urli, M. Maia, and M. Monperrus. BEARS: An extensible Java bug benchmark for automatic program repair studies. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 468–478. IEEE Computer Society, 2019.

[MYR16] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. Angelix: scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 691–701. ACM, 2016.

[NPB+20] Yannic Noller, Corina S. Păsăreanu, Marcel Böhme, Youcheng Sun, Hoang Lam Nguyen, and Lars Grunske. HyDiff: Hybrid differential software

analysis. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1273–1285. Association for Computing Machinery, 2020.

[NQRC13] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 772–781, 2013.

[NSGR22] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. Trust enhancement issues in program repair. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2228–2240. Association for Computing Machinery, 2022.

[Och57] Akira Ochiai. Zoogeographic studies on the soleoid fishes found in Japan and its neighbouring regions. *Bulletin of Japanese Society of Scientific Fisheries*, 22:526–530, 1957.

[Ope22] OpenAI. Introducing ChatGPT. https://openai.com/blog/chatgpt, 2022.

[Ope23] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[Örw24] Albert Örwall. Moatless Tools. https://github.com/aorwall/moatless-tools, 2024.

[OZ15] Peter-Michael Osera and Steve Zdancewic. Type-and-example-directed program synthesis. *ACM SIGPLAN Notices*, 50(6):619–630, 2015.

[OZHW23] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. LLM is like a box of chocolates: the non-determinism of ChatGPT in code generation. *arXiv preprint arXiv:2308.02828*, 2023.

[PBWDJ12] Elena Popovici, Anthony Bucci, R. Paul Wiegand, and Edwin D. De Jong. *Coevolutionary Principles*, pages 987–1033. Springer Berlin Heidelberg, 2012.

[PDEP08] Suzette Person, Matthew B Dwyer, Sebastian Elbaum, and Corina S Păsăreanu. Differential symbolic execution. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 226–237, 2008.

[PKC16] Hristina Palikareva, Tomasz Kuchta, and Cristian Cadar. Shadow of a doubt: Testing for divergences between software versions. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1181–1192. Association for Computing Machinery, 2016.

[PKT15] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. Reformulating branch coverage as a many-objective optimization problem. In

2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), pages 1–10, 2015.

[PKT18] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering*, 44(2):122–158, 2018.

[PM02] Ludo Pagie and Melanie Mitchell. A comparison of evolutionary and coevolutionary search. *International Journal of Computational Intelligence and Applications*, 02(01):53–69, 2002.

[PWN+25] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 267. PMLR / OpenReview.net, 2025.

[QLAR15] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 24–36. Association for Computing Machinery, 2015.

[RCV+15] José Miguel Rojas, José Campos, Mattia Vivanti, Gordon Fraser, and Andrea Arcuri. Combining multiple coverage criteria in search-based unit test generation. In *Search-Based Software Engineering*, pages 93–108. Springer International Publishing, 2015.

[RGG+23] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[RPPR26] Abhik Roychoudhury, Corina Pasareanu, Michael Pradel, and Baishakhi Ray. Agentic AI software engineer: Programming with trust. *Communications of the ACM*, 69(5), 2026.

[RZR25] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. SpecRover: Code intent extraction via LLMs. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 963–974. IEEE, 2025.

[SBLGB15] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? Overfitting in automated program repair. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 532–543. Association for Computing Machinery, 2015.

[Sch23] Philip Schanely. CrossHair: An analysis tool for Python that blurs the line between testing and type systems. https://github.com/pschanely/CrossHair, 2023.

[SFMO13] Sina Shamshiri, Gordon Fraser, Phil McMinn, and Alessandro Orso. Search-based propagation of regression faults in automated regression testing. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation Workshops*, pages 396–399, 2013.

[SGN+22] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. What is it like to program with artificial intelligence? In *Proceedings of the 33rd Annual Conference of the Psychology of Programming Interest Group (PPIG 2022)*, 2022.

[SLTB+06] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th international conference on Architectural support for programming languages and operating systems*, pages 404–415, 2006.

[SNGR21] Ridwan Shariffdeen, Yannic Noller, Lars Grunske, and Abhik Roychoudhury. Concolic program repair. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 390–405. Association for Computing Machinery, 2021.

[Son11] SonarSource. SonarQube. https://github.com/SonarSource/sonarqube, 2011.

[SWE24] SWE-bench Team. SWE-bench Verified leaderboard. https://www.swebench.com/#verified, 2024.

[SZM+19] Zeyu Sun, Qihao Zhu, Lili Mou, Yingfei Xiong, Ge Li, and Lu Zhang. A grammar-based structural CNN decoder for code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 7055–7062. AAAI Press, 2019.

[TDS+20] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.

[TJTP20] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. Evolutionary improvement of assertion oracles. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1178–1189. Association for Computing Machinery, 2020.

[TYPR16] Shin Hwei Tan, Hiroaki Yoshida, Mukul R. Prasad, and Abhik Roychoudhury. Anti-patterns in search-based program repair. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 727–738. Association for Computing Machinery, 2016.

[Wey82] Elaine J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4):465–470, 1982.

[WGM+24] Xinchen Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. AEGIS: An agent-based framework for general bug reproduction from issue descriptions. *arXiv preprint arXiv:2411.18015*, 2024.

[WLS+25] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, et al. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025.

[WNGF09] Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. Automatically finding patches using genetic programming. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2009.

[WPL25] You Wang, Michael Pradel, and Zhongxin Liu. Are "solved issues" in SWE-bench really solved correctly? An empirical study. *arXiv preprint arXiv:2503.15223*, 2025.

[WSKLG21] Chu-Pan Wong, Priscila Santiesteban, Christian Kästner, and Claire Le Goues. VarFix: Balancing edit expressiveness and search effectiveness in automated program repair. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 354–366. Association for Computing Machinery, 2021.

[WWS+22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in

large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[WWS+23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[XCZ+23] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. ChatUniTest: a ChatGPT-based automated unit test generation tool. *arXiv preprint arXiv:2305.04764*, 2023.

[XDDZ25] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proc. ACM Softw. Eng.*, 2(FSE):801–824, 2025.

[XLZ+18] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. Identifying patch correctness in test-based program repair. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 789–799. Association for Computing Machinery, 2018.

[XMD+17] Jifeng Xuan, Matias Martinez, Favio DeMarco, Maxime Clément, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus. Nopol: Automatic repair of conditional statement bugs in Java programs. *IEEE Transactions on Software Engineering*, 43(1):34–55, 2017.

[XPT+24] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 126:1–126:13. ACM, 2024.

[YB20a] Yuan Yuan and Wolfgang Banzhaf. ARJA: Automated repair of Java programs via multi-objective genetic programming. *IEEE Transactions on Software Engineering*, 46(10):1040–1067, 2020.

[YB20b] Yuan Yuan and Wolfgang Banzhaf. Toward better evolutionary program repair: An integrated approach. *ACM Trans. Softw. Eng. Methodol.*, 29(1), 2020.

[YGM+22] He Ye, Jian Gu, Matias Martinez, Thomas Durieux, and Martin Monperrus. Automated classification of overfitting patches with statically extracted code features. *IEEE Transactions on Software Engineering*, 48(8):2920–2938, 2022.

[YJW+24] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces

enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 50528–50652, 2024.

[YLD+24] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving ChatGPT for unit test generation. *Proc. ACM Softw. Eng.*, 1(FSE):1703–1726, 2024.

[YMD+19] Zhongxing Yu, Matias Martinez, Benjamin Danglot, Thomas Durieux, and Martin Monperrus. Alleviating patch overfitting with automatic test generation: a study of feasibility and effectiveness for the Nopol repair system. *Empirical Software Engineering*, 24(1):33–67, 2019.

[YMM22] He Ye, Matias Martinez, and Martin Monperrus. Neural program repair with execution-based backpropagation. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1506–1518. Association for Computing Machinery, 2022.

[YN17] Pengcheng Yin and Graham Neubig. A syntactic neural model for general-purpose code generation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 440–450. Association for Computational Linguistics, 2017.

[ZRFR24] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[ZWL+25] Mingwei Zheng, Chengpeng Wang, Xuwei Liu, Jinyao Guo, Shiwei Feng, and Xiangyu Zhang. An LLM agent for functional bug detection in network protocols. *arXiv preprint arXiv:2506.00714*, 2025.

[ZXW26] Jiaxin Zhang, Caiming Xiong, and Chien-Sheng Wu. Agentic confidence calibration. *arXiv preprint arXiv:2601.15778*, 2026.

[ZYH+23] Tianyi Zhang, Tao Yu, Tatsunori Hashimoto, Mike Lewis, Wen-Tau Yih, Daniel Fried, and Sida Wang. Coder reviewer reranking for code generation. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 202, pages 41832–41846. PMLR, 2023.